天链机械臂 SDK 使用说明文档

Py 开发接口手册

支持平台：Linux(x86-64、ARM)、Windows

（Ver.2.6.0） 天链机器人有限责任公司 销售联系（Tel）：18090080872

版本号	修改内容	修改日期	修改人
V1.0	完成说明文档初步全部撰写	2026.4.25	何向阳
V2.0	文档内容修改完善	2026.8.07	曹芩滔
V2.5.1	按 V2.5.1 接口面裁剪：移除 V3 专有接口章节、SDK 扩展（实验性）接口章节	2026.9.16	ShiHongchao
V2.5.2	接口面未变更，版本号随 SDK V2.5.2 同步	2026.9.21	ShiHongchao
V2.5.3	分发包结构补充 Windows 调试版 C++ 库说明；接口面未变更，版本号随 SDK V2.5.3 同步	2026.10.8	ShiHongchao
V2.6.0	新增周期同步速度/扭矩（CSV/CST）与拖动轨迹管理接口说明；`DragParam` 字段随协议更新；版本号随 SDK V2.6.0 同步	2026.10.23	ShiHongchao

目录

- 一．概述
- 二．支持的操作系统与软件版本
- 三．SDK结构
- 四．SDK接口说明
- 4.1 接口通用规则
- 4.2 连接管理接口
    - 4.2.1 connect_robot - 创建机械臂连接控制句柄
    - 4.2.2 disconnect_robot - 关闭所有机械臂连接
    - 4.2.3 get_connection_status - 获取控制器连接状态
    - 4.2.4 get_controller_network_config - 获取控制器网络配置
    - 4.2.5 set_controller_network_config - 设置控制器网络配置
- 4.3 日志管理接口
    - 4.3.1 log_download_by_quantity - 保存日志到文件

    - 4.3.2 set_log_level - 设置日志级别
- 4.4 基础状态查询接口
    - 4.4.1 get_current_position - 获取机械臂当前坐标信息
    - 4.4.2 get_robot_running_state - 获取作业文件运行状态
    - 4.4.3 get_servo_state - 获取伺服状态
    - 4.4.4 get_current_mode - 获取机器人当前模式
    - 4.4.5 get_robot_dh_param - 获取机械臂 DH 参数
    - 4.4.6 get_library_version - 读取机械臂软件信息
    - 4.4.7 get_robot_joint_param - 获取关节参数
    - 4.4.8 query_joint_software_version - 查询指定关节（轴）的软件版本号
    - 4.4.9 get_nexmotion_lib_version - 获取底层 nexmotion 库版本信息
    - 4.4.10 get_robot_state - 获取机器人整体状态
    - 4.4.11 get_robot_type - 获取机器人类型
    - 4.4.12 get_current_line_speed_and_joint_speed - 获取当前线速度与关节速度
    - 4.4.13 get_pos_reachable - 查询点位可达性
    - 4.4.14 robot_state_callback - 注册机器人状态回调
    - 4.4.15 get_speed - 获取速度
    - 4.4.16 get_cartesian_params - 获取笛卡尔参数
    - 4.4.17 get_joint_position - 查询指定关节位置
- 4.5 伺服状态与系统参数设置接口
    - 4.5.1 clear_error - 清除系统错误/清除关节错误代码
    - 4.5.2 set_servo_state - 设置伺服就绪状态(使能)
    - 4.5.3 set_servo_poweron - 设置伺服上电状态(先使能)
    - 4.5.4 set_servo_poweroff - 设置伺服下电状态
    - 4.5.5 set_speed - 设置速度
    - 4.5.6 restore_default_param_DH - 恢复机械臂默认DH参数
    - 4.5.7 set_cartesian_params - 设置机器人笛卡尔参数
    - 4.5.8 restore_network_factory_settings - 恢复网络出厂设置
    - 4.5.9 set_robot_joint_param - 设置当前机器人关节参数
    - 4.5.10 set_controller_ip - 配置控制器有线网口IP
    - 4.5.11 set_robot_dh_param - 设置机器人 DH 参数
    - 4.5.12 set_zero_pos_deviation - 设置零位偏差
    - 4.5.13 set_axis_zero_position - 标定当前状态下的零位/指定关节当前位置为零位
    - 4.5.14 set_receive_error_or_warnning_message_callback - 设置错误警告回调
- 4.6 六维力传感器查询接口
    - 4.6.1 set_six_dimensional_force_communication_params - 配置六维力传感器数据
    - 4.6.2 get_six_dimensional_force_communication_params - 获取六维力传感器通讯参数
    - 4.6.3 get_sensor_base_param - 获取六维力传感器的基础参数
    - 4.6.4 mark_base_sensor - 执行六维力传感器标定
    - 4.6.5 get_sensor_6d_data - 获取六维力传感器数据

    - 4.6.6 get_payload_param_by_sensor - 获取六维力传感器负载参数
- 4.7 碰撞检测接口
    - 4.7.1 get_collision_detection_param - 查询碰撞防护等级
    - 4.7.2 set_collision - 设置碰撞防护参数
    - 4.7.3 get_collision - 获取碰撞防护参数
- 4.8 电机与关节监控接口
    - 4.8.1 get_current_motor_speed - 获取各关节电机当前转速
    - 4.8.2 get_current_motor_torque - 获取各关节电机当前扭矩
    - 4.8.3 get_current_motor_current - 获取各关节电机当前电流
    - 4.8.4 get_current_motor_payload - 获取各关节电机当前负载
    - 4.8.5 get_joint_voltage - 获取各关节电压
    - 4.8.6 get_joint_temperature - 获取关节电机温度
- 4.9 作业运动控制接口
    - 4.9.1 job_insert_moveJ - 作业里添加关节运动关节空间运动
    - 4.9.2 job_insert_moveL - 作业里添加关节运动笛卡尔空间直线运动
    - 4.9.3 job_insert_moveC - 作业里添加关节运动笛卡尔空间圆弧运动
    - 4.9.4 job_open - 打开指定作业文件
    - 4.9.5 job_create -- 创建作业文件名
    - 4.9.6 job_get_all_jobfile_name -- 获取全部作业文件名
    - 4.9.7 job_run --启动指定作业文件
- 4.10 实时运动控制接口
    - 4.10.1 robot_movej - 关节空间运动到目标位姿
    - 4.10.2 robot_movel - 笛卡尔空间直线运动到目标位姿
    - 4.10.3 robot_go_home - 机器人回原点（home）
    - 4.10.4 robot_go_to_reset_position - 机器人回到复位位置
- 4.11 坐标系与用户坐标管理接口
    - 4.11.1 set_user_coordinate_data - 设置用户坐标
    - 4.11.2 set_user_coord_number - 设置用户坐标编号
    - 4.11.3 get_user_coord_para - 获取当前使用的用户坐标的参数
    - 4.11.4 get_user_coord_number - 获取当前使用的用户坐标编号
    - 4.11.5 get_current_coord - 获取当前工作坐标系
    - 4.11.6 set_current_coord - 切换示教运动坐标系
    - 4.11.7 calibration_oxy - 三点法自动设置用户坐标系
    - 4.11.8 calculate_user_coordinate - 计算用户坐标
    - 4.11.9 get_target_coord_pos_value - 获取指定坐标下的位置
    - 4.11.10 get_local_pos_p_value - 获取本地位置变量 P 值
    - 4.11.11 set_default_cartesian_params - 设置笛卡尔参数为默认值
- 4.12 工具手相关接口
    - 4.12.1 get_tool_hand_number - 获取当前使用的工具手编号
    - 4.12.2 set_tool_hand_number - 设置工具手编号

    - 4.12.3 set_tool_hand_param - 设置工具手参数
    - 4.12.4 get_tool_hand_param - 获取当前工具手参数
    - 4.12.5 set_tool_coordinate_range - 设置工具包络球参数（工具坐标范围参数）
    - 4.12.6 get_tool_coordinate_range - 获取工具包络球参数（工具坐标范围参数）
- 4.13 示教操作接口
    - 4.13.1 robot_start_jogging - 点动
    - 4.13.2 robot_stop_jogging -- 停止点动
- 4.14 全局路点管理接口
    - 4.14.1 set_global_position - 新增全局路点/设置全局路点
    - 4.14.2 get_global_position - 查询全局 GP 点位
    - 4.14.3 set_global_string - 设置全局字符串变量
    - 4.14.4 get_global_string - 获取全局字符串变量
    - 4.14.5 get_global_sync_position - 获取全局同步位置
    - 4.14.6 get_global_variant - 获取全局变量
- 4.15 模式与 IO 控制接口
    - 4.15.1 set_current_mode - 设置机器人模式（切换示教模式和运动模式）
    - 4.15.2 set_digital_output - 设置数字 IO 输出
    - 4.15.3 get_digital_output - 获取所有数字 IO 输出
    - 4.15.4 get_digital_input - 获取所有 IO 输入状态
    - 4.15.5 set_analog_output - 设置模拟量输出
    - 4.15.6 get_analog_input - 获取模拟量输入
    - 4.15.7 get_analog_output - 获取模拟量输出
    - 4.15.8 get_io_type - 获取 IO 类型
    - 4.15.9 get_IO_reset_function - 获取 IO 复位功能
- 4.16 算法接口
    - 4.16.1 get_origin_coord_to_target_coord -- 坐标转换算法
    - 4.16.2 get_quat2rpy - 四元数 转 欧拉角
    - 4.16.3 get_rpy2quat - 欧拉角 转 四元数
    - 4.16.4 get_rpy2r - 欧拉角 转 旋转矩阵
    - 4.16.5 get_tr2r- 位姿 转 旋转矩阵
    - 4.16.6 get_r2tr - 旋转矩阵 转 位姿矩阵
- 4.17 modbus通信接口
    - 4.17.1 modbus_set_master_parameter - 设置 Modbus RTU 主站参数
    - 4.17.2 modbus_open_master - 打开 Modbus 主站
    - 4.17.3 modbus_write_multiple_holding_registers - 写多个保持寄存器
    - 4.17.4 modbus_read_holding_registers - Modbus RTU 读取多个保持寄存器
    - 4.17.5 modbus_write_signal_coil_status - Modbus RTU协议写线圈
    - 4.17.6 modbus_write_signal_holding_registers - Modbus RTU协议写保持寄存器
    - 4.17.7 modbus_read_coil_status -- Modbus读线圈状态
    - 4.17.8 modbus_read_input_status - Modbus读取输入状态

    - 4.17.9 modbus_read_input_registers - Modbus 读输入寄存器
- 4.18 队列运动接口
    - 4.18.1 queue_motion_set_status - 打开/关闭队列运动模式
    - 4.18.2 queue_motion_push_back_moveJ队列运动模式的本地队列最后插入一条moveJ运动
    - 4.18.3 queue_motion_push_back_moveL队列运动模式的本地队列最后插入一条move L运动
    - 4.18.4 queue_motion_push_back_moveC - 队列运动本地队列末尾插入一条 MoveC 运动
    - 4.18.5 queue_motion_push_back_moveS - 队列运动本地队列末尾插入一条 MoveS 运动
    - 4.18.6 queue_motion_send_to_controller - 将本地队列的前size个数据发送到控制器
    - 4.18.7 queue_motion_get_status -- 查询队列模式状态
    - 4.18.8 queue_motion_size - 查询当前运动队列的长度
    - 4.18.9 queue_motion_get_queuelen - 查询当前运动队列剩余的指令数量
    - 4.18.10 queue_motion_suspend - 暂停连续运动（轨迹暂停）
    - 4.18.11 queue_motion_restart - 暂停后再次运行运动队列（继续当前轨迹运动）
    - 4.18.12 queue_motion_stop - 停止队列运动
    - 4.18.13 queue_motion_stop_not_power_off - 停止连续运动保持上电状态（轨迹缓停）
    - 4.18.14 queue_motion_clear_Data - 清空本地队列数据
- 4.19 拖拽示教与轨迹记录回放
    - 4.19.1 set_darg_mode - 设置拖拽示教的拖拽方式
    - 4.19.2 set_teach_type - 设置示教模式类型
    - 4.19.3 get_teach_type -- 查询示教模式类型
    - 4.19.4 set_current_teach_sensitivity - 设置电流环拖动示教灵敏度
    - 4.19.5 get_current_teach_sensitivity - 获取电流环拖动示教灵敏度
    - 4.19.6 get_drag_thread_is_end -- 查询拖拽是否结束
    - 4.19.7 track_record_start - 轨迹记录开始
    - 4.19.8 get_track_record_status - 轨迹记录开启状态查询
    - 4.19.9 track_record_stop - 轨迹记录关闭
    - 4.19.10 track_record_save - 保存拖动示教轨迹
    - 4.19.11 track_record_playback - 轨迹回放
    - 4.19.12 track_record_delete - 轨迹清除
    - 4.19.13 drag_trajectory_export - 导出拖动轨迹
    - 4.19.14 drag_trajectory_import - 导入拖动轨迹
    - 4.19.15 drag_trajectory_playback - 回放指定拖动轨迹
    - 4.19.16 drag_trajectory_delete - 删除指定拖动轨迹
- 五．7000端口接口
- 5.1 以下功能在连接7000端口后使用
    - 5.1.1 open_servoJ - 打开关节跟踪模式
    - 5.1.2 close_servoJ - 关闭关节跟踪模式
    - 5.1.3 set_servoJ_pos - 设置实时跟踪关节位置

    - 5.1.4 open_servo_csv_mode - 打开周期同步速度模式
    - 5.1.5 close_servo_csv_mode - 关闭周期同步速度模式
    - 5.1.6 set_servo_csv_velocity - 下发周期同步速度
    - 5.1.7 open_servo_cst_mode - 打开周期同步扭矩模式
    - 5.1.8 close_servo_cst_mode - 关闭周期同步扭矩模式
    - 5.1.9 set_servo_cst_torque - 下发周期同步扭矩
- 六． 使用示例
- 6.1Modbus 主站通信综合示例
- 6.2关节实时跟踪servoJ使用示例
- 6.3关节运动到目标点robot_movej直线运动到目标位置robot_movel综合使用示例
- 6.4点动使用示例
- 6.5作业文件指令使用示例
- 6.6队列运动指令使用示例
- 6.7坐标变换算法示例
- 6.8模式与 IO 控制接口示例
- 6.9拖拽示教与轨迹记录/回放示例
- 附录（一）

一．概述

本说明文档为四川天链机器人股份有限公司研发生产的超轻量协作机器人TCB系列Python开发包SDK使用说明文档，通过该文档，用户能够更好使用SDK控制TCB机械臂执行相应任务，更加便捷地进行二次开发。

二．支持的操作系统与软件版本

1. 操作系统：目前支持Linux（x86和arm），Windows。
2. 软件版本：Python3.10及以上，本SDK基于Python3.10版本开发，使用前确保不同版本之间的兼容性。

三．SDK结构

SDK结构如下：

```
| LICENSE
| README.md
|
├─docs                # 说明文档
|
├─examples            # 示例程序
```

```
│ ├─cpp
│ └─py
│
├─include           # 对外头文件（namespace tl）
│ ├─tl_api.h
│ ├─tl_interface.h
│ ├─tl_types.h
│ └─...              # tl_force / tl_servo_ext 等
│
├─lib               # 平台动态库 + Python 绑定
│ ├─linux
│ │ ├─x86           # libtl_host.so + python（_tl_host + tl_interface.py）
│ │ └─arm64         # libtl_host.so + python（_tl_host + tl_interface.py）
│ ├─windows         # tl_host.dll + tl_host.lib + python（tl_host.pyd）
│ │ └─debug         # 调试版 C++ 库：tl_host.dll + tl_host.lib + tl_host.pdb
│
└─src
  ├─tl_host         # tl_host 合并库源码
  └─python_bind     # nanobind Python 绑定
```

四．SDK接口说明

Sdk接口说明提供python参考用法，具体参数类型可以在对应cpp中文件查看.sdk支持c++,python进行二次开发。

4.1 接口通用规则

1. **导入方式**：本文档接口基于python通过import tl_interface as tl导入
2. **句柄要求**：所有接口第一个参数必须是connect_robot返回的有效 socket 句柄
3. **返回值约定**：
    - 0：执行成功
    - 非 0：执行失败
4. **数据类型**：
    - 向量类型：使用tl.VectorDouble()或tl.VectorInt()创建
    - 结构体类型：使用tl.MoveCmd()、tl.RobotDHParam()等创建
5. **坐标系约定**：
    - 0：关节坐标系

- 1：直角坐标系
- 2：工具坐标系
- 3：用户坐标系

6. **工具函数说明**：为保证示例代码输出格式统一、结果清晰规整，本文档所有python示例统一封装了调试打印工具函数，用于标准化接口返回状态、向量、矩阵、分隔标题的输出展示。
    - print_result：统一打印接口调用结果，区分成功 / 失败并输出错误码；
    - print_separator：打印章节分隔标题，用于区分多组测试案例；
    - print_vector：格式化打印一维浮点向量，固定 4 位小数，适用于欧拉角、四元数、关节角度等数据；
    - print_matrix：按行主序格式化打印多维矩阵，适配 3×3 旋转矩阵、4×4位姿矩阵输出；
    - print_ok：打印单条成功提示，无返回码，用于非接口调用类的普通校验通过提示；
    - print_fail：打印单条失败提示，附带错误码，用于非接口返回的自定义逻辑失败告警；
    - print_title：打印大段测试模块标题，使用长分割线，用于标记整套测试模块的起始位置

```
def print_result(msg, ret):
    if ret == SUCCESS:
        print("  [成功] " + msg)
    else:
        print("  [失败] " + msg + ", 错误码: " + str(ret))
def print_separator(title):
    print("\n========== " + title + " ==========")
def print_vector(name, v):
    """打印向量 --- 格式: name [size] = {v1, v2, ...}, 保留4位小数"""
    formatted = ["{:.4f}".format(x) for x in v]
    print("  " + name + " [" + str(len(v)) + "] = {" + ", ".join(formatted) + "}")
def print_matrix(name, m, rows, cols):
    """打印矩阵（行主序），保留4位小数"""
    print("  " + name + " [" + str(len(m)) + "]:")
```

```
    for r in range(rows):
        line = "  "
        for c in range(cols):
            idx = r * cols + c
            if idx < len(m):
                line += "{:10.4f} ".format(m[idx])
        print(line)
def print_ok(msg):
    print("  [OK] " + msg)
def print_fail(msg, ret):
    print("  [FAIL] " + msg + ", 错误码: " + str(ret))
def print_title(title):
    print("\n" + "=" * 50)
    print("  " + title)
    print("=" * 50)
```

4.2 连接管理接口

头文件：include/tl_interface.h

相关示例代码：examples/py/ex_connect.py

4.2.1 connect_robot - 创建机械臂连接控制句柄

方法：connect_robot(const std::string& ip, const std::string& port);

功能描述：建立与机械臂控制器的网络通信通道。需要分别连接指令端口（6001）和实时端口（7000）以获得两个独立的句柄。

参数：

- ip：机械臂控制器 IP 地址，例如 \"192.168.1.13\"。
- port：端口号字符串，指令通道 \"6001\"，实时跟踪通道 \"7000\"。

返回值：

- int：已连接的句柄
  - 1：连接成功，返回有效的 socket 句柄
  - ≤ 0：连接失败

使用示例：

```
import tl_interface as tl

# 创建机械臂连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

if socket_fd > 0:

    print(f"机械臂连接成功！控制器句柄：{socket_fd}")

else:

    print(f"连接失败，控制器返回句柄：{socket_fd}")
```

4.2.2 disconnect_robot - 关闭所有机械臂连接

方法：disconnect_robot(socket_fd);

功能描述：断开控制器。

参数：

- socket_fd：已连接的句柄，由connect_robot返回

返回值：

- None

使用示例：

```
import tl_interface as tl

# 先建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# 断开连接

tl.disconnect_robot(socket_fd)
```

```
print("已成功断开与控制器的连接")
```

最佳实践：

- 建议使用try-finally结构确保断开连接
- 断开连接前建议先下使能

4.2.3 get_connection_status - 获取控制器连接状态

方法：get_connection_status(socketFd);

功能描述：获得控制器连接状态。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

ret = tl.get_connection_status(socket_fd)

if ret == 0:
    print("控制器已连接")
else:
    print(f"控制器连接状态查询失败，错误码：{ret}")
```

4.2.4 get_controller_network_config - 获取控制器网络配置

方法：get_controller_network_config(socketFd);

功能描述：获取控制器当前的网络配置信息。

参数：

- socketFd：已连接的句柄。

返回值：

- tuple：(ret, name, address, gateway, dns)，name 网络接口名称、address IP 地址、gateway 网关、dns DNS
- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

ret, name, address, gateway, dns = tl.get_controller_network_config(socket_fd)

if ret == 0:
    print(f"IP：{address}，网关：{gateway}")
```

4.2.5 set_controller_network_config - 设置控制器网络配置

方法：set_controller_network_config(socketFd, name, ip, gateway, dns);

功能描述：设置控制器网络配置。

参数：

- socketFd：已连接的句柄。
- name：要修改的网络接口名称。
- ip：新的 IP 地址。
- gateway：网关地址，传空字符串表示不配置网关。
- dns：DNS 服务器地址，传空字符串表示不配置 DNS。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")
```

```
ret = tl.set_controller_network_config(socket_fd, "eth0", "192.168.1.14",
"192.168.1.1", "")
```

4.3 日志管理接口

头文件：include/tl_interface.h

相关示例代码：examples/py/ex_log_download.py

4.3.1 log_download_by_quantity - 保存日志到文件

方法：log_download_by_quantity(SOCKETFD socketFd, int counts,const std::string& directoryPath);

参数：

- socketFd：已连接的句柄。
- counts：文件数量。
- directoryPath：目录的完整路径，字符串格式。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
print("[INFO] 开始下载日志...")

ret = log_download_by_quantity(socket_fd, count, log_path)

if ret != SUCCESS:

    print("[ERROR] 日志下载失败，程序退出!")

    return -1

print("[INFO] 日志下载完成")
```

4.3.2 set_log_level - 设置日志级别

方法：set_log_level(level);

功能描述：设置日志级别。

参数：

- level：日志级别，0=DEBUG 1=INFO 2=WARNING 3=ERROR。

返回值：

- 无

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

tl.set_log_level(1)  # 设置为 INFO 级别
```

4.4 基础状态查询接口

头文件：include/tl_interface.h

4.4.1 get_current_position - 获取机械臂当前坐标信息

方法：get_current_position(socketFd, coord, pos);

功能描述：获取机器人当前位置。

参数：

- socketFd：已连接的句柄。
- coord：坐标系类型。
    - 0：关节坐标系。
    - 1：直角坐标系。
    - 2：工具坐标系。
- pos：输出参数，用于存储当前位置的向量。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```python
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# 获取关节坐标系下的当前位置

current_pos = tl.VectorDouble()

coord = 0 # 关节坐标系

ret = tl.get_current_position(socket_fd, coord, current_pos)

if ret == 0:

    pos_list = list(current_pos)

    print(f"获取位置成功：{pos_list}")

else:

    print(f"获取位置失败，错误码：{ret}")
```

4.4.2 get_robot_running_state - 获取作业文件运行状态

方法：get_robot_running_state(SOCKETFD socketFd, int& status);

功能描述：获取当前作业文件的运行状态，执行前需要延时0.1秒。

参数：

- socketFd：已连接的句柄。
- status：程序运行状态，0 停止 1 暂停 2 运行。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```python
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")
```

```
time.sleep(0.1)

status = 0

ret, status = tl.get_robot_running_state(socket_fd, status)

if ret != SUCCESS:

    print("[ERROR] 获取机械臂运行状态失败，程序退出!")

else:

    if status == 0:

        print("[INFO] 机械臂处于停止状态 (status = " + str(status) + ")")

    elif status == 1:

        print("[INFO] 机械臂处于暂停状态 (status = " + str(status) + ")")

    elif status == 2:

        print("[INFO] 机械臂处于运行状态 (status = " + str(status) + ")")
```

4.4.3 get_servo_state - 获取伺服状态

方法：get_servo_state(SOCKETFD socketFd, int& status);

功能描述：获取机械臂当前伺服状态信息。

参数：

- socketFd：已连接的句柄。
- status 接收获取结果 0：停止状态 1：就绪状态 2：报警状态 3：运行状态。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
 # 2. 检查伺服状态

state = -1

ret, state = get_servo_state(socket_fd, state)

if ret != SUCCESS:
```

```
    print(f"[ERROR] 获取伺服状态失败: {ret}")
    return
print(f"  [信息] 当前伺服状态: {state} (0=停止 1=就绪 2=报警 3=运行)")
if state == 2:
    print("[ERROR] 伺服处于报警状态，无法上电，请排查报警原因后重试")
    return
if state == 3:
    print("[INFO] 机械臂已在使能上电状态")
    return
```

4.4.4 get_current_mode - 获取机器人当前模式

方法：get_current_mode(SOCKETFD socketFd, int& mode);

功能描述：获取机械臂当前模式。

参数：

- socketFd：已连接的句柄
- mode：当前模式 0：示教 1：远程 2：运行。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# ---- 检查模式并确保伺服运行 ----
# 先获取当前模式
current_mode = -1
ret, current_mode = get_current_mode(sock, current_mode)
if ret != SUCCESS or current_mode < 0:
    print("[ERROR] 获取当前模式失败: " + str(ret))
```

```
    disconnect_robot(sock)

    return -1

print("  [信息] 当前模式: " + str(current_mode) + " (0=示教 1=远程 2=运行)")
```

4.4.5 get_robot_dh_param - 获取机械臂 DH 参数

方法：get_robot_dh_param(SOCKETFD socketFd, RobotDHParam& param);

参数：

- socketFd：已连接的句柄。
- param：输出参数，RobotDHParam 结构体，包含 alpha/a/theta/d（标准 DH 参数双精度数组）、eulerAngle（欧拉角）、mountingAngle（安装角）等成员。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

dh_params = tl.RobotDHParam()

ret = tl.get_robot_dh_param(socket_fd, dh_params)

if ret == 0:

    print("alpha=" + str(list(dh_params.alpha)))

    print("a=" + str(list(dh_params.a)))

    print("theta=" + str(list(dh_params.theta)))

    print("d=" + str(list(dh_params.d)))

    print("eulerAngle=" + str(dh_params.eulerAngle))

    print("mountingAngle=" + str(dh_params.mountingAngle))

else:
```

```
    print(f"获取机械臂DH参数失败，错误码：{ret}")
```

4.4.6 get_library_version - 读取机械臂软件信息

方法：get_library_version();

功能描述：获取 SDK 库版本信息。

参数：

- None

返回值：

- str：成功返回版本信息字符串，格式 "(Base:)"，如 "V2.5.1-0-gabc1234-20260916(Base: 2.0.4-20260914-6cf4a3b)"。

使用示例：

```
import tl_interface as tl

# 获取库版本

version = tl.get_library_version()

print(f"机器人库版本 = {version}")
```

4.4.7 get_robot_joint_param - 获取关节参数

方法：get_robot_joint_param(socketFd, id, param);

功能描述：获取指定关节参数。

参数：

- socketFd：已连接的句柄。
- id：关节轴号，1\~7 代表第 1\~7 轴。
- param：输出参数，用于存储关节参数的结构体。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# 创建关节参数对象

joint_param = tl.RobotJointParam()

# 获取第 1 轴参数

ret = tl.get_robot_joint_param(socket_fd, 1, joint_param)

if ret == 0:

    print(f"返回值(ret) = {ret}")

    print(f"减速比 = {joint_param.reduceRatio}")

    print(f"额定转速 = {joint_param.ratedRotSpeed}")

    print(f"最大加速度 = {joint_param.maxAcc}")

    print(f"电机方向 = {joint_param.motorDirection}")

    print(f"正向软限位 = {joint_param.maxPos}")

    print(f"负向软限位 = {joint_param.minPos}")

else:

    print(f"获取关节参数失败，错误码：{ret}")
```

4.4.8 query_joint_software_version - 查询指定关节（轴）的软件版本号

方法：query_joint_software_version(socketFd, axisNum);

功能描述：查询指定关节（轴）的软件版本号。

参数：

- socketFd：已连接的句柄。
- axisNum：轴编号

返回值：

- tuple：(ret, version)，ret 为执行结果，version 为查询到的软件版本号字符串。
- 0：成功。
- 非 0：失败。

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# 查询 J1~J7 关节软件版本号

for i in range(1, 8):

    ret, version = tl.query_joint_software_version(socket_fd, i)

    if ret == 0:

        print(f"关节{i}软件版本: {version}")

    else:

        print(f"查询关节{i}软件版本失败，错误码: {ret}")
```

4.4.9 get_nexmotion_lib_version - 获取底层 nexmotion 库版本信息

方法：get_nexmotion_lib_version(socketFd);

功能描述：获取算法库版本。

参数：

- socketFd：已连接的句柄。

返回值：

- tuple：(ret, version)，version 为算法库版本号字符串
- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

ret, version = tl.get_nexmotion_lib_version(socket_fd)
```

4.4.10 get_robot_state - 获取机器人整体状态

方法：get_robot_state(socketFd, param);

功能描述：7000 端口查询状态。

参数：

- socketFd：已连接的句柄（7000 端口）。
- param：RobotState 结构体，查询参数（channel 查询通道，最多 9 个通道；ioState 查询 IO；position 0-关节坐标 1-基坐标；dataildmotionpos 机械臂运动点位；programRunStatus 运行状态；servoStatus 伺服状态；operationMode 操作模式；globalSpeed 全局速度等）。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 7000 端口建立连接（机器人状态查询走 7000）

socket_fd_7000 = tl.connect_robot("192.168.1.13", "7000")

# 构造查询参数（单次回复）

param = tl.RobotState()

param.channel = 1

param.stop = False

param.mode = 0                # 0-查询只回复一次 1-持续回复

param.interval = 1000

param.position = 0            # 0-关节坐标 1-直角坐标

param.servoStatus = True
```

```
param.programRunStatus = True

ret = tl.get_robot_state(socket_fd_7000, param)

if ret == 0:

    print("机器人状态查询请求已发送，等待回调")

# 停止持续推送并断开

stop_param = tl.RobotState()

stop_param.channel = 1

stop_param.stop = True

tl.get_robot_state(socket_fd_7000, stop_param)

tl.disconnect_robot(socket_fd_7000)
```

4.4.11 get_robot_type - 获取机器人类型

方法：get_robot_type(socketFd, type);

功能描述：获取机器人类型（返回机器人类型编号）。

参数：

- socketFd：已连接的句柄。
- type：传入占位值（`RobotType` 枚举成员或 `int`），实际类型以返回值为准。

RobotType 枚举说明：

枚举值	数值	说明
SIX_AXIS_SERIAL	1	六轴串联多关节
FOUR_AXIS_SCARA	2	四轴 SCARA
FOUR_AXIS_PALLETIZING	3	四轴码垛
FOUR_AXIS_SERIAL	4	四轴串联多关节
SINGLE_AXIS	5	单轴
FIVE_AXIS_SERIAL	6	五轴串联多关节
SIX_AXIS_COLLABORATIVE	7	六轴协作
TWO_AXIS_SCARA	8	二轴 SCARA
THREE_AXIS_SCARA	9	三轴 SCARA

枚举值	数值	说明
THREE_AXIS_CARTESIAN	10	三轴直角
THREE_AXIS_SPECIAL	11	三轴异形
SEVEN_AXIS_SERIAL	12	七轴串联多关节
SCARA_SPECIAL	13	SCARA 异形一
FOUR_AXIS_PALLETIZING_LEAD	14	四轴码垛丝杆

返回值：

- tuple：(ret, type)，ret 为执行结果，type 为机器人类型（`RobotType` 枚举）
- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# 第 2 个参数为占位值，实际类型以返回值为准
ret, robot_type = tl.get_robot_type(socket_fd, tl.RobotType.SIX_AXIS_SERIAL)

if ret == 0:

    print(f"机器人类型：{robot_type}，编号：{int(robot_type)}")
```

4.4.12 get_current_line_speed_and_joint_speed - 获取当前线速度与关节速度

方法：get_current_line_speed_and_joint_speed(socketFd, joint_speed, joint_speed_sync);

功能描述：获取当前末端线速度和关节速度。

参数：

- socketFd：已连接的句柄。
- joint_speed：输出参数，关节速度，长度 7，单位 度/s（需传入 VectorDouble 实例）。
- joint_speed_sync：输出参数，外部轴关节速度，长度 5，单位 度/s（需传入 VectorDouble 实例）。

返回值：

- tuple：(ret, line_speed)，ret 为执行结果、line_speed 为末端线速度 mm/s

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# 出参必须传 VectorDouble 实例

joint_speed = tl.VectorDouble()        # 关节速度，长度 7

joint_speed_sync = tl.VectorDouble()   # 外部轴关节速度，长度 5

ret, line_speed = tl.get_current_line_speed_and_joint_speed(socket_fd, joint_speed, join
t_speed_sync)

if ret == 0:

    print(f"末端线速度: {line_speed} mm/s")

    print(f"关节速度: {list(joint_speed)}")

    print(f"外部轴关节速度: {list(joint_speed_sync)}")
```

4.4.13 get_pos_reachable - 查询点位可达性

方法：get_pos_reachable(socketFd, pos, movetype, result);

功能描述：检测目标点位是否可达。

参数：

- socketFd：已连接的句柄。
- pos：目标点位坐标数据，vector 长度 14（[0]坐标系 0:关节 1:直角 2:工具 3:用户；[1]单位 0:角度 1:弧度；[2]形态；[3]工具手坐标序号；[4]用户坐标序号；[5][6]备用；[7-13]点位信息）。
- movetype：移动方式，"MOVJ" 或 "MOVL"。
- result：输出参数，点位是否可达（true=可达）。

返回值：

- tuple：(ret, result)，ret 为执行结果、result 为点位是否可达
- 0：成功

- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# 目标点位 14 位: [7-13] 为点位值

pos = tl.VectorDouble([0, 0, 0, 0, 0, 0, 0, 20.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0])

ret, reachable = tl.get_pos_reachable(socket_fd, pos, "MOVJ", False)

if ret == 0:

    print(f"点位可达：{reachable}")
```

4.4.14 robot_state_callback - 注册机器人状态回调

方法：robot_state_callback(socketFd, function);

功能描述：7000 端口状态返回的回调函数，状态变化时自动调用 function。

参数：

- socketFd：已连接的句柄（7000 端口）。
- function：回调函数，接收状态字符串参数。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 7000 端口建立连接（状态回调走 7000）

socket_fd_7000 = tl.connect_robot("192.168.1.13", "7000")

# 定义状态回调（回调在独立线程触发，不能做耗时操作）

def on_robot_state(state):

    print(f"[回调] 收到状态：{state}")
```

```
# 注册回调

ret = tl.robot_state_callback(socket_fd_7000, on_robot_state)

if ret == 0:

    print("状态回调注册成功")
```

4.4.15 get_speed - 获取速度

方法：get_speed(socketFd, speed);

功能描述：获得当前模式的速度。

参数：

- socketFd：已连接的句柄。
- speed：速度大小，0~100。

返回值：

- tuple：(ret, speed)，ret 为执行结果、speed 为当前速度 0~100
- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

speed = 0

ret, speed = tl.get_speed(socket_fd, speed)

if ret == 0:

    print(f"当前速度: {speed} (0<speed≤100 百分比)")

else:

    print(f"获取速度失败，错误码：{ret}")
```

4.4.16 get_cartesian_params - 获取笛卡尔参数

方法：get_cartesian_params(socketFd, param);

功能描述：获取笛卡尔空间参数。

参数：

- socketFd：已连接的句柄。
- param：CartesianParam 输出参数（maxSpeed 最大线速度 mm/s；maxAcc 最大线加速度 mm/s^2；maxDec 最大线减速度 mm/s^2；maxAttitudeVel 最大姿态角速度 deg/s；speedLimitMode 速度限制方式；posResolution 绝对位置分辨率；maxJerk 最大线加加速度 mm/s^3 等）。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

param = tl.CartesianParam()

ret = tl.get_cartesian_params(socket_fd, param)

if ret == 0:

    print(f"最大线速度：{param.maxSpeed} mm/s")
```

4.4.17 get_joint_position - 查询指定关节位置

方法：get_joint_position(socketFd, axisNum, pos);

功能描述：查询指定关节在基坐标系中的位置。

参数：

- socketFd：已连接的句柄。
- axisNum：关节轴号（如 1 表示 J1 轴）。
- pos：输出参数，关节位置坐标（需传入 VectorDouble 实例）。

返回值：

- 0：成功。
- 非 0：失败。

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

pos = tl.VectorDouble()        # 输出参数，需传入 VectorDouble 实例

# 查询 J1 轴位置

ret = tl.get_joint_position(socket_fd, 1, pos)

if ret == 0:

    print(f"J1 位置: {list(pos)}")

else:

    print(f"查询失败，错误码：{ret}")
```

4.5 伺服状态与系统参数设置接口

头文件：include/tl_interface.h

4.5.1 clear_error - 清除系统错误/清除关节错误代码

方法：clear_error(SOCKETFD socketFd);

功能描述：复位控制器的当前报警状态（清除红色错误）。在机械臂发生碰撞、超限位等错误后，排除物理故障后调用此接口清除报警。清错后需要手动进行下电操作，释放控制器的占用状态才可以继续上电（清错后不能直接上电，先下电再上电）。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# 1. 先清除控制器错误和报警，再下电复位确保状态同步
```

```
clear_error(socket_fd)

time.sleep(0.3)

set_servo_poweroff(socket_fd)

time.sleep(0.3)
```

4.5.2 set_servo_state - 设置伺服就绪状态(使能)

方法：set_servo_state(SOCKETFD socketFd, int state);

功能描述：控制电机抱闸与使能。state=1=使能（解锁抱闸），state=0=去使能（抱闸锁死）。

参数：

- socketFd：已连接的句柄。
- state：1使能，0去使能。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# 设置伺服就绪（READY=使能抱闸）

ret = tl.set_servo_state(socket_fd, tl.ServoState.READY)

if ret == 0:

    print("  [信息] 伺服就绪设置成功")

else:

    print(f"[ERROR] 设置伺服就绪失败: {ret}")
```

4.5.3 set_servo_poweron - 设置伺服上电状态(先使能)

方法：set_servo_poweron(SOCKETFD socketFd);

功能描述：机器人上电。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
def enable_servo(socket_fd):

    """使能上电 ------
先清错，再检查伺服状态，报警则报错退出，最后上电"""

    print("--- 检查伺服状态 ---")

    # 1. 先清除控制器错误和报警，再下电复位确保状态同步

    ret = clear_error(socket_fd)

    time.sleep(0.3)

    set_servo_poweroff(socket_fd)

    time.sleep(0.3)

    # 2. 检查伺服状态

    state = -1

    ret, state = get_servo_state(socket_fd, state)

    if ret != SUCCESS:

        print(f"[ERROR] 获取伺服状态失败: {ret}")

        return

    print(f"  [信息] 当前伺服状态: {state} (0=停止 1=就绪 2=报警 3=运行)")

    if state == 2:

        print("[ERROR]伺服处于报警状态，无法上电，请排查报警原因后重试")

        return

    if state == 3:
```

```
        print("[INFO] 机械臂已在使能上电状态")
        return
    # 3. 上电流程
    if state == 0:
        print("  [信息] 伺服处于停止状态，设置就绪...")
        ret = set_servo_state(socket_fd, 1)
        if ret != SUCCESS and ret != OPERATION_NOT_ALLOWED:
            print(f"[ERROR] 设置伺服就绪失败: {ret}")
            return
        time.sleep(0.5)
    print("  [信息] 上电使能...")
    ret = set_servo_poweron(socket_fd)
    if ret != SUCCESS:
        print(f"[ERROR] 上电失败: {ret}")
        return
    time.sleep(2)
    # 4. 确认结果
    _, state = get_servo_state(socket_fd, state)
    if state == 3:
        print("[INFO] 机械臂使能上电成功（servo_state = " + str(state) + "）")
    else:
        print("[ERROR] 机械臂使能上电失败（servo_state = " + str(state) + "）")
```

4.5.4 set_servo_poweroff - 设置伺服下电状态

方法：set_servo_poweroff(SOCKETFD socketFd);

功能描述：机器人下电。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
def disable_servo(socket_fd):
    """使能伺服下电"""
    state = -1
    _, state = get_servo_state(socket_fd, state)
    if state == 1:
        print("[INFO] 机械臂已使能下电")
    elif state == 3:
        set_servo_poweroff(socket_fd)
        _, state = get_servo_state(socket_fd, state)
        print("[INFO] 机械臂使能下电成功（servo_state = " + str(state) + "）")
        return
    print("[INFO] 机械臂使能下电失败（servo_state = " + str(state) + "）")
```

4.5.5 set_speed - 设置速度

方法：set_speed(SOCKETFD socketFd, int speed);

功能描述：设置当前模式的速度。

参数：

- socketFd：已连接的句柄。
- speed :速度大小0-100。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
test_speeds = [50, 80, 30]

for spd in test_speeds:

    print("\\n[步骤 5] 设置全局速度: " + str(spd) + "%")

    ret = set_speed(sock, spd)

    print("set_speed", ret)

    time.sleep(0.2)

    current_speed = -1

    ret, current_speed = get_speed(sock, current_speed)

    if ret == SUCCESS:

        print("  [信息] 读取确认: 当前速度 = " + str(current_speed) + "%")
```

4.5.6 restore_default_param_DH - 恢复机械臂默认DH参数

方法：restore_default_param_DH(SOCKETFD socketFd, int robotNum);

功能描述：恢复机械臂默认DH参数。

参数：

- socketFd：已连接的句柄。
- robotNum：机器人编号(1-4)。

返回值：

- 0：成功
- 非 0：失败
- DHparam DH：参数

使用示例：

```python
# 该功能不建议测试，请在厂家指导下使用

def demo_restore_default_param_DH(sock):

    """恢复机械臂默认DH参数"""

    print("恢复默认DH参数")

    robot_num = 1  # TODO: 根据实际机器人编号调整（1-4）

    ret = restore_default_param_DH(sock, robot_num)

    print("restore_default_param_DH", ret)

    print()
```

4.5.7 set_cartesian_params - 设置机器人笛卡尔参数

方法：set_cartesian_params(SOCKETFD socketFd, const CartesianParam& param);

功能描述：配置笛卡尔空间运动动力学约束参数，限定末端执行器最大直线速度、直线加速度、旋转角速度与角加速度，实现轨迹平滑度与运行安全管控。

参数：

- socketFd：已连接的句柄。
- param 包含笛卡尔空间运动参数的结构体。

返回值：

- 0：成功。
- 非 0：失败

使用示例：

```python
 # 以下功能不建议测试，请在厂家指导下使用

def demo_set_cartesian_params(sock):

    """设置笛卡尔参数（最大线速度/线加速度/角速度）"""

    print("设置笛卡尔参数")

    print("  [信息] 以下数值需根据真实机械臂手册调整")

    param = CartesianParam()

    param.maxSpeed = 500.0                # TODO: 最大线速度(mm/s)
```

```
    param.maxAcc = 1000.0                # TODO: 最大线加速度(mm/s²)
    param.maxDec = 1000.0                # TODO: 最大线减速度(mm/s²)
    param.maxAttitudeVel = 100.0         # TODO: 最大姿态角速度(°/s)
    param.speedLimitMode = 0             # TODO: 速度限制方式（0=位姿，1=位置）
    ret = set_cartesian_params(sock, param)
    print("set_cartesian_params", ret)
    print()
```

4.5.8 restore_network_factory_settings - 恢复网络出厂设置

方法：restore_network_factory_settings(SOCKETFD socketFd);

功能描述：执行控制器网络参数恢复出厂操作，重置通讯 IP 地址为默认固定地址，属于高危配置操作。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功。
- 非 0：失败

使用示例：

```
def demo_restore_network_factory_settings(sock):
    """恢复网络出厂设置（危险操作，会重置所有网络参数并可能断连）"""
    print("恢复网络出厂设置")
    print("  [警告] 恢复网络出厂设置会重置控制器所有网络参数！")
    print("  [警告] 执行后需要重新配置网络才能连接！")
    ret = restore_network_factory_settings(sock)
    print("restore_network_factory_settings", ret)
    print()
```

4.5.9 set_robot_joint_param - 设置当前机器人关节参数

方法：set_robot_joint_param(SOCKETFD socketFd, int id, RobotJointParam& param);

功能描述：对指定单关节进行运动与硬限位参数配置，设定各关节最大运行速度、加速度及正负向机械位置软限位，约束关节运动边界保障本体运行安全。

参数：

- socketFd：已连接的句柄。
- id：关节参数[1,6]。
- param：结构体参数。

返回值：

- 0：成功。
- 非 0：失败

使用示例：

```
 # 以下功能不建议测试，请在厂家指导下使用

def demo_set_robot_joint_param(sock):

    """设置关节参数（最大速度/加速度/位置限位）"""

    print("设置关节参数")

    print("  [信息] 遍历设置关节1~6的参数，以下数值需根据真实机械臂手册调整")

    for axis_id in range(1, 7):

        param = RobotJointParam()

        # --- 运动学参数（不可随意修改，需按实际减速比/编码器设置）---

        param.reduceRatio = 100.0          # TODO: 减速比

        param.encoderResolution = 17       # TODO: 编码器分辨率

        param.motorDirection = 1           # TODO: 电机方向（1=正向，-1=反向）

        # --- 位置限位（单位：度）---

        param.maxPos = 180.0               # TODO: 轴正限位(°)

        param.minPos = -180.0              # TODO: 轴反限位(°)
```

```
        # --- 速度参数（单位：度/秒）---
        param.ratedRotSpeed = 100.0         # TODO: 电机额定转速
        param.maxRotSpeed = 150.0           # TODO: 电机最大转速
        # --- 加减速度 ---
        param.maxAcc = 100.0                # TODO: 最大加速度
        param.maxDec = 100.0                # TODO: 最大减速度
        ret = set_robot_joint_param(sock, axis_id, param)
        print("set_robot_joint_param (关节%d)" % axis_id, ret)
    print()
```

4.5.10 set_controller_ip - 配置控制器有线网口IP

方法：set_controller_ip(SOCKETFD socketFd, std::string name, std::string address, std::string gateway, std::string dns);

功能描述：配置控制器有线网口 IP。

参数：

- socketFd：已连接的句柄。
- name：网卡设备名称。
- address：ip地址。
- gateway：网关地址。
- dns：DNS服务器地址。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")
```

```
# 危险操作：修改控制器 IP 会立即生效并导致 TCP 断连，请确认新 IP 与电脑在同一网段

# TODO: 根据实际网络环境修改以下参数

name = "eth0"             # 网口名称

address = "192.168.1.14"  # 目标 IP 地址

gateway = "192.168.1.1"   # 网关地址

dns = "8.8.8.8"           # DNS 服务器

ret = tl.set_controller_ip(socket_fd, name, address, gateway, dns)

if ret == 0:

    print("set_controller_ip 成功")

else:

    print(f"set_controller_ip 失败，错误码：{ret}")
```

4.5.11 set_robot_dh_param - 设置机器人 DH 参数

方法：set_robot_dh_param(socketFd, param);

功能描述：设置当前机器人 DH 参数。

参数：

- socketFd：已连接的句柄。
- param：RobotDHParam 结构体（alpha[6] 1-6轴DH参数alpha 单位 deg；a[6] 1-6轴DH参数a 单位 mm；theta[6] 1-6轴DH参数theta 单位 deg；d[6] 1-6轴DH参数d 单位 mm；eulerAngle 欧拉角模式；mountingAngle 安装角度 单位 deg）。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

param = RobotDHParam()
```

```
param.alpha = [0, 0, 0, 0, 0, 0]

ret = tl.set_robot_dh_param(socket_fd, param)
```

4.5.12 set_zero_pos_deviation - 设置零位偏差

方法：set_zero_pos_deviation(socketFd, axis, shift);

功能描述：设置零点偏移。

参数：

- socketFd：已连接的句柄。
- axis：轴号（1-7）。
- shift：偏移量（-360 < shift < 360）。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

ret = tl.set_zero_pos_deviation(socket_fd, 1, 0.5)
```

4.5.13 set_axis_zero_position - 标定当前状态下的零位/指定关节当前位置为零位

方法：set_axis_zero_position(SOCKETFD socketFd, int axis);

功能描述：将当前关节位置记录为零位。axis=0 表示所有轴，axis=1\~6 表示单轴。

警告：仅在机械臂物理对齐零刻度线后调用，否则将导致坐标系错乱，引发撞机风险！

参数：

- socketFd：已连接的句柄。
- axis：轴号，1\~7 代表第 1\~7 轴，0 表示全部轴。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

import time

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# 设置所有轴原点

axis = 0 # 0表示全部关节

ret = tl.set_axis_zero_position(socket_fd, axis)

print(f"设置轴原点返回值(ret) = {ret}")

time.sleep(1)
```

4.5.14 set_receive_error_or_warnning_message_callback - 设置错误警告回调

方法：set_receive_error_or_warnning_message_callback(SOCKETFD socketFd,void(*function) (int messageType, const char* message, int messageCode));

功能描述：注册异步错误/警告回调函数。当控制器产生错误或警告时，会异步调用该回调。

参数：

- socketFd：已连接的句柄。
- function：指向回调函数的指针。该回调函数在接收到错误或警告信息时被调用。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接
```

```
socket_fd = tl.connect_robot("192.168.1.13", "6001")

# 定义错误/警告回调（在独立线程触发，不能做耗时操作）

def error_warning_callback(message_type, message, code):

    print(f"[回调] type={message_type} message={message} code={code}")

# 注册回调

ret = tl.set_receive_error_or_warnning_message_callback(socket_fd, error_warning_callbac
k)

if ret == 0:

    print(" [成功] 回调注册成功，后续收到错误/警告会自动触发")

else:

    print(f" [失败] 回调注册失败，错误码：{ret}")
```

4.6 六维力传感器查询接口

头文件：include/tl_interface.h

4.6.1 set_six_dimensional_force_communication_params - 配置六维力传感器数据

方法：set_six_dimensional_force_communication_params(SOCKETFD socketFd, SixDimensionalForceCommunicationParams& param);

功能描述：配置机械臂末端六维力传感器。

参数：

- socketFd：已连接的句柄。
- commParams：输入参数，用于存储六维力传感器通讯参数。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```python
# 1. 设置六维力传感器通讯参数

commParams = SixDimensionalForceCommunicationParams()

commParams.sensorCommunicationType = 0 # 0: EtherCAT

commParams.startupAutoConnectSensor = True # 启动自动连接传感器

commParams.sensorDragEnable = True # 传感器拖拽使能

commParams.etherCat_mapNum = 1 # EtherCAT 参数

commParams.YDirection = -1 # y反方向

commParams.ZDirection = -1 # z反方向

ret = set_six_dimensional_force_communication_params(socketFd, commParams)

if ret == SUCCESS:

    print("设置六维力传感器通讯参数成功!")

else:

    print(f"设置六维力传感器通讯参数失败! ret = {ret}")
```

4.6.2 get_six_dimensional_force_communication_params - 获取六维力传感器通讯参数

方法：get_six_dimensional_force_communication_params(socketFd, params);

功能描述：获取六维力传感器通讯参数。

参数：

- socketFd：已连接的句柄。
- params：SixDimensionalForceCommunicationParams 输出参数（sensorCommunicationType 通讯类型；sensorDragEnable 传感器拖拽使能；originDataInitialPara 原始数据初始参数；startupAutoConnectSensor 启动自动连接传感器；YDirection Y 方向；ZDirection Z 方向；etherCat_mapNum EtherCAT 等）。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

params = SixDimensionalForceCommunicationParams()

ret = tl.get_six_dimensional_force_communication_params(socket_fd, params)
```

4.6.3 get_sensor_base_param - 获取六维力传感器的基础参数

方法：get_sensor_base_param(SOCKETFD socketFd, SensorBaseParam& baseParam);

功能说明：获取六维力传感器的基础参数（质量、质心、标零状态）。

参数：

- socketFd：已连接的句柄。
- baseParam 用于接收传感器基础参数的结构体。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# 查询标定后的基础参数

base_param = SensorBaseParam()

ret = get_sensor_base_param(socket_fd, base_param)

if ret == SUCCESS:

    print("  传感器质量: " + str(base_param.sensorMass))

    print("  质心 - X: " + str(base_param.sensorMassCenterX)

          + ", Y: " + str(base_param.sensorMassCenterY)

          + ", Z: " + str(base_param.sensorMassCenterZ))
```

4.6.4 mark_base_sensor - 执行六维力传感器标定

方法：mark_base_sensor(SOCKETFD socketFd, bool& success);

功能描述：完成六维力传感器零点偏移与正交耦合误差校准，消除自重及安装偏差带来的力信号零漂，保证后续力控采集数据精度。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# ----标定六维力传感器零位/重心----

# 操作前请确保:

#   1. 机械臂末端无负载或已安装标准负载

#   2. 机械臂处于竖直姿态（关节各轴在合理位置）

#   3. 周围安全，标定过程中机械臂会缓慢运动

print("[INFO]**********标定六维力传感器零位/重心**********")

print("[INFO] 即将执行六维力传感器标定，请确认:")

print("  1. 机械臂末端无负载（或已安装标准负载）")

print("  2. 机械臂周围安全无人")

confirm = input("  是否继续? (y/n): ")

if confirm.lower() == 'y':

    success = False

    ret, success = mark_base_sensor(socket_fd, success)

    if ret != SUCCESS:

        print("[ERROR] 标定六维力传感器失败，错误码: " + str(ret))

    elif success:

        print("[INFO] 标定六维力传感器成功")

        # 查询标定后的基础参数
```

```
        base_param = SensorBaseParam()
        ret = get_sensor_base_param(socket_fd, base_param)
        if ret == SUCCESS:
            print("  传感器质量: " + str(base_param.sensorMass))
            print("  质心 - X: " + str(base_param.sensorMassCenterX)
                  + ", Y: " + str(base_param.sensorMassCenterY)
                  + ", Z: " + str(base_param.sensorMassCenterZ))
            print("  标零状态: " + ("已标零" if base_param.saveZero else "未标零"))
    else:
        print("[ERROR] 标定六维力传感器失败，标定结果为 false")
else:
    print("  [信息] 用户取消标定")
print("")
```

4.6.5 get_sensor_6d_data - 获取六维力传感器数据

方法：get_sensor_6d_data(socketFd, sensorData);

功能描述：获取六维力传感器数据。

参数：

- socketFd：已连接的句柄。
- sensorData：输出参数，用于存储力和力矩的结构体。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# 2. 获取六维力传感器数据
sensorData = Sensor6DData()
ret = get_sensor_6d_data(socketFd, sensorData)
```

```
if ret == SUCCESS:
    print("获取六维力传感器数据成功!")
    print(f"传感器连接状态: {'已连接' if sensorData.sensorConnected else '未连接'}")
    print("--- 传感器原始数据 ---")
    print(f"Fx = {sensorData.fxData:.3f} N")
    print(f"Fy = {sensorData.fyData:.3f} N")
    print(f"Fz = {sensorData.fzData:.3f} N")
    print(f"Mx = {sensorData.mxData:.3f} N·m")
    print(f"My = {sensorData.myData:.3f} N·m")
    print(f"Mz = {sensorData.mzData:.3f} N·m")
    print("--- 去皮后数据 ---")
    print(f"Fx = {sensorData.fxDataSubBase:.3f} N")
    print(f"Fy = {sensorData.fyDataSubBase:.3f} N")
    print(f"Fz = {sensorData.fzDataSubBase:.3f} N")
    print(f"Mx = {sensorData.mxDataSubBase:.3f} N·m")
    print(f"My = {sensorData.myDataSubBase:.3f} N·m")
    print(f"Mz = {sensorData.mzDataSubBase:.3f} N·m")
else:
    print(f"获取六维力传感器数据失败! ret = {ret}")
```

4.6.6 get_payload_param_by_sensor - 获取六维力传感器负载参数

方法：get_payload_param_by_sensor(socketFd, param);

功能描述：获取传感器负载参数。

参数：

- socketFd：已连接的句柄。
- param：PayloadParamBySensor 输出参数（payloadMass 负载质量 kg；payloadMassCenterX/Y/Z 负载质心 mm）。

返回值：

- tuple：(ret, param)
- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

param = PayloadParamBySensor()

ret, param = tl.get_payload_param_by_sensor(socket_fd, param)
```

4.7 碰撞检测接口

头文件：include/tl_interface.h

4.7.1 get_collision_detection_param - 查询碰撞防护等级

方法：get_collision_detection_param(SOCKETFD socketFd, CollisionPara& param);

功能描述：获取机械臂末端六维力传感器数据。

参数：

- socketFd：已连接的句柄。
- param：输入参数， CollisionPara 结构体。

返回值：

- 0：成功
- 非 0：失败
- param：结构体内部参数填充

使用示例：

```
"""查询碰撞防护等级"""

print("[INFO]**********查询碰撞防护等级**********")
```

```
param = CollisionPara()

ret = get_collision_detection_param(socket_fd, param)

if ret != SUCCESS:

    print("[ERROR] 获取碰撞防护等级失败，错误码: " + str(ret))

    return

print("[INFO] 碰撞检测阈值（指令）: ", end="")

for val in param.collisionDetection_run:

    print(str(val) + " ", end="")

print("")

print("[INFO] 碰撞检测阈值（点动）: ", end="")

for val in param.collisionDetection_teach:

    print(str(val) + " ", end="")

print("")

print("[INFO] 指令位置响应时间: " + str(param.position_delay_time_ms_value) + " ms")

print("[INFO] 误差允许时间: " + str(param.error_enable_time_ms_value) + " ms")

print("[INFO] 机器人轴数: " + str(param.axisum))
```

4.7.2 set_collision - 设置碰撞防护参数

方法：set_collision(socketFd, param);

功能描述：设置碰撞防护参数。

参数：

- socketFd：已连接的句柄。
- param：CollisionPara 结构体（collisionDetection_run 碰撞检测阈值-指令；collisionDetection_teach 碰撞检测阈值-点动；position_delay_time_ms_value 指令位置响应时间；error_enable_time_ms_value 误差允许时间；axisum 机器人轴数，默认 6）。

返回值：

- 0：成功

- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

param = CollisionPara()

param.axisum = 6

ret = tl.set_collision(socket_fd, param)
```

4.7.3 get_collision - 获取碰撞防护参数

方法：get_collision(socketFd, param);

功能描述：获取碰撞防护参数。

参数：

- socketFd：已连接的句柄。
- param：CollisionPara 输出参数。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

param = CollisionPara()

ret = tl.get_collision(socket_fd, param)
```

4.8 电机与关节监控接口

头文件：include/tl_interface.h

4.8.1 get_current_motor_speed - 获取各关节电机当前转速

方法：get_current_motor_speed(socketFd, motor_speed, motor_speed_sync);

功能描述：获取当前电机转速。

参数：

- socketFd：已连接的句柄。
- motor_speed：输出参数，机器人本体电机转速，长度 7，单位 [RPM]（需传入 VectorInt 实例）。
- motor_speed_sync：输出参数，外部轴电机转速，长度 5，单位 [RPM]（需传入 VectorInt 实例）。

返回值：

- 0：成功。
- 非 0：失败。

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

motor_speed = tl.VectorInt()         # 机器人本体电机转速，长度 7

motor_speed_sync = tl.VectorInt()    # 外部轴电机转速，长度 5

ret = tl.get_current_motor_speed(socket_fd, motor_speed, motor_speed_sync)

# 打印结果

print(f"返回值: {ret}")

print(f"本体电机转速: {list(motor_speed)}")

print(f"外部轴电机转速: {list(motor_speed_sync)}")
```

4.8.2 get_current_motor_torque - 获取各关节电机当前扭矩

方法：get_current_motor_torque(socketFd, motor_torque, motor_torque_sync);

功能描述：获取当前电机扭矩。

参数：

- socketFd：已连接的句柄。
- motor_torque：输出参数，机器人本体电机扭矩，长度 7，单位 [%]（需传入 VectorInt 实例）。
- motor_torque_sync：输出参数，外部轴电机扭矩，长度 5，单位 [%]（需传入 VectorInt 实例）。

返回值：

- 0：成功。
- 非 0：失败。

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

motor_torque = tl.VectorInt()        # 机器人本体电机扭矩，长度 7

motor_torque_sync = tl.VectorInt()   # 外部轴电机扭矩，长度 5

ret = tl.get_current_motor_torque(socket_fd, motor_torque, motor_torque_sync)

# 打印结果

print(f"返回值: {ret}")

print(f"本体电机扭矩: {list(motor_torque)}")

print(f"外部轴电机扭矩: {list(motor_torque_sync)}")
```

4.8.3 get_current_motor_current - 获取各关节电机当前电流

方法：get_current_motor_current(socketFd, motor_current, motor_current_sync);

功能描述：获取当前电机电流。

参数：

- socketFd：已连接的句柄。
- motor_current：输出参数，机器人本体电机电流，长度 7，单位 ‰（需传入 VectorDouble 实例）。

- motor_current_sync：输出参数，外部轴电机电流，长度 5，单位 ‰（需传入 VectorDouble 实例）。

返回值：

- 0：成功。
- 非 0：失败。

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

motor_current = tl.VectorDouble()        # 机器人本体电机电流，长度 7

motor_current_sync = tl.VectorDouble()   # 外部轴电机电流，长度 5

ret = tl.get_current_motor_current(socket_fd, motor_current, motor_current_sync)

# 打印结果

print(f"返回值: {ret}")

print(f"本体电机电流: {list(motor_current)}")

print(f"外部轴电机电流: {list(motor_current_sync)}")
```

4.8.4 get_current_motor_payload - 获取各关节电机当前负载

方法：get_current_motor_payload(socketFd, motor_payload, motor_payload_sync);

功能描述：获取当前电机负载。

参数：

- socketFd：已连接的句柄。
- motor_payload：输出参数，机器人本体电机负载，长度 7，单位 [%]（需传入 VectorDouble 实例）。
- motor_payload_sync：输出参数，外部轴电机负载，长度 5，单位 [%]（需传入 VectorDouble 实例）。

返回值：

- 0：成功。
- 非 0：失败。

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

motor_payload = tl.VectorDouble()        # 机器人本体电机负载，长度 7

motor_payload_sync = tl.VectorDouble()   # 外部轴电机负载，长度 5

ret = tl.get_current_motor_payload(socket_fd, motor_payload, motor_payload_sync)

# 打印结果

print(f"返回值: {ret}")

print(f"本体电机负载: {list(motor_payload)}")

print(f"外部轴电机负载: {list(motor_payload_sync)}")
```

4.8.5 get_joint_voltage - 获取各关节电压

方法：get_joint_voltage(socketFd, joint_voltage, positioner_voltage);

功能描述：获取关节电压。

参数：

- socketFd：已连接的句柄。
- joint_voltage：输出参数，机器人本体各关节电压（需传入 VectorDouble 实例）。
- positioner_voltage：输出参数，外部轴各关节电压（需传入 VectorDouble 实例）。

返回值：

- 0：成功。
- 非 0：失败。

使用示例：

```python
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

joint_voltage = tl.VectorDouble()        # 机器人本体各关节电压

positioner_voltage = tl.VectorDouble()   # 外部轴各关节电压

ret = tl.get_joint_voltage(socket_fd, joint_voltage, positioner_voltage)

# 打印结果

print(f"返回值: {ret}")

print(f"本体关节电压: {list(joint_voltage)}")

print(f"外部轴关节电压: {list(positioner_voltage)}")
```

4.8.6 get_joint_temperature - 获取关节电机温度

方法：get_joint_temperature(socketFd, temperatures);

功能描述：获取关节温度。

参数：

- socketFd：已连接的句柄。
- temperatures：输出参数，用于存储输出的信息。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```python
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

joint_temperature = tl.VectorDouble()

ret = tl.get_joint_temperature(socket_fd, joint_temperature)
```

```
# 打印结果

print(f"返回值: {ret}")

print(f"关节温度: {list(joint_temperature)}")
```

4.9 作业运动控制接口

头文件：include/tl_interface.h

4.9.1 job_insert_moveJ - 作业里添加关节运动关节空间运动

方法：job_insert_moveJ(SOCKETFD socketFd, int line, MoveCmd moveCmd);

功能描述：在作业文件指定行插入一条关节空间运动（MoveJ）指令，需使用P 变量模式。

参数：

- socketFd：已连接的句柄。
- line：插入的位置行号。
- moveCmd：运动指令结构体MoveCmd，包含目标位置和运动参数
    - targetPosType：设为 PosType::PType。
    - targetPosName：点位变量名，如 \"P0001\"。
    - coord：0（关节坐标系）。
    - targetPosValue：std::vector\<double>，长度至少6，存储目标角度（度）。
    - velocity：速度百分比（1\~100）。
    - acc、dec：加速度/减速度百分比。
    - pl：平滑度（0表示精确到达）。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

```

```
# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

cmd = tl.MoveCmd()

cmd.targetPosType = tl.PosType.PType

cmd.targetPosName = "P0001"

cmd.coord = 0                # 关节坐标系

cmd.velocity = 40

cmd.acc = 10

cmd.dec = 10

cmd.pl = 0

cmd.targetPosValue = tl.VectorDouble([0.0] * 14)

ret = tl.job_insert_moveJ(socket_fd, 1, cmd)

if ret == 0:

    print("[INFO] 作业文件插入MoveJ成功")

else:

    print(f"[ERROR] 作业文件插入MoveJ失败，错误码：{ret}")
```

4.9.2 job_insert_moveL - 作业里添加关节运动笛卡尔空间直线运动

方法：job_insert_moveL(SOCKETFD socketFd, int line, MoveCmd moveCmd);

功能描述：插入笛卡尔空间直线运动（MoveL）指令。

参数：

- socketFd：已连接的句柄。
- line：插入的位置行号。
- moveCmd：运动指令结构体MoveCmd，包含目标位置和运动参数。
    - targetPosType：设为 PosType::PType。
    - targetPosName：点位变量名，如 \"P0001\"。
    - coord：1（直角坐标系）。

- targetPosValue：[X(mm), Y(mm), Z(mm), RX(rad), RY(rad),RZ(rad)]。
- velocity：速度百分比（1\~100）。
- acc、dec：加速度/减速度百分比。
- pl：平滑度（0表示精确到达）。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

cmd = tl.MoveCmd()

cmd.targetPosName = "P0002"

cmd.coord = 1                        # 直角坐标系

cmd.velocity = 40

cmd.acc = 10

cmd.dec = 10

cmd.pl = 0

cmd.targetPosValue = tl.VectorDouble([280, 0.0, 380, 3.14, 0.0, 0.0])

ret = tl.job_insert_moveL(socket_fd, 2, cmd)

if ret == 0:

    print("[INFO] 作业文件插入MoveL成功")

else:

    print(f"[ERROR] 作业文件插入MoveL失败，错误码：{ret}")
```

4.9.3 job_insert_moveC - 作业里添加关节运动笛卡尔空间圆弧运动

方法：job_insert_moveC(SOCKETFD socketFd, int line, MoveCmd moveCmd);

功能描述：插入笛卡尔空间圆弧运动（MoveC）指令。通常需要连续插入两条 MoveC 指令，第一条为圆弧中间点，第二条为终点。

参数：

- socketFd：已连接的句柄。
- line：插入的位置行号。
- MoveCmd：运动指令结构体，包含目标位置和运动参数。
    - targetPosType：设为 PosType::PType。
    - targetPosName：点位变量名，如 \"P0001\"。
    - coord：1（直角坐标系）。
    - targetPosValue：为终点坐标，[X(mm), Y(mm), Z(mm), RX(rad),RY(rad), RZ(rad)]。
    - velocity：速度百分比（1\~100）。
    - acc、dec：加速度/减速度百分比。
    - pl：平滑度（0表示精确到达）。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

cmd = tl.MoveCmd()

cmd.targetPosName = "P0003"

cmd.coord = 2                        # 工具坐标系

cmd.velocity = 40
```

```
cmd.acc = 10

cmd.dec = 10

cmd.pl = 0

cmd.targetPosValue = tl.VectorDouble([270, 0.0, 380, 3.14, 0.0, 0.0])

ret = tl.job_insert_moveC(socket_fd, 3, cmd)

if ret == 0:

    print("[INFO] 作业文件插入MoveC成功")

else:

    print(f"[ERROR] 作业文件插入MoveC失败，错误码：{ret}")
```

4.9.4 job_open - 打开指定作业文件

方法：job_open(SOCKETFD socketFd, const std::string& jobName);

功能描述：打开已存在的作业文件，使其成为当前激活程序（后续插入指令将写入该文件）。

参数：

- socketFd：已连接的句柄。
- jobName：作业文件名称。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

jobName = "TEST_PY"

ret = tl.job_open(socket_fd, jobName)

if ret == 0:
```

```
    print("[INFO] 打开作业文件成功")

else:

    print(f"[ERROR] 打开作业文件失败，错误码：{ret}")
```

4.9.5 job_create -- 创建作业文件名

方法：job_create(SOCKETFD socketFd, const std::string& jobName);

功能描述：在控制器内新建一个空的作业文件（自动防止重名）。

参数：

- socketFd：已连接的句柄。
- jobName：作业文件名称。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# ----创建作业文件----

jobName = "TEST_PY"

print("[INFO] **********创建作业文件**********")

ret = job_create(socket_fd, jobName)

if ret != SUCCESS:

    print("[ERROR] 创建作业文件失败，程序退出!")

    return -1

print("[INFO] 创建作业文件成功")
```

4.9.6 job_get_all_jobfile_name -- 获取全部作业文件名

方法：job_get_all_jobfile_name(SOCKETFD socketFd,std::vector\<std::vector\<std::string>>& robotsFile);

功能描述：获取控制器内所有作业文件列表（按机器人分组，通常只有一组）。

参数：

- socketFd：已连接的句柄。
- robotsFile：作业文件名称，二维数组，一维长度是4，对应4个机器人。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

all_job_names = tl.VectorVectorString()

ret = tl.job_get_all_jobfile_name(socket_fd, all_job_names)

if ret == 0:

    i = 1

    for job_names in all_job_names:

        for name in job_names:

            print(f"作业文件 {i} 名称: {name}")

            i += 1

else:

    print(f"获取作业文件名失败，错误码：{ret}")
```

4.9.7 job_run --启动指定作业文件

方法： job_run(SOCKETFD socketFd, const std::string& jobName);

功能描述：启动运行已打开的作业文件（**该功能需要在运行模式下使用，调用该接口时，控制器会自动进行伺服模式切换**）。

参数：

- socketFd：已连接的句柄。

- jobName：作业文件名称。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# ----运行作业文件----

print("[INFO] **********运行作业文件**********")

jobName = "TEST_PY"

ret = job_run(socket_fd, jobName)

if ret != SUCCESS:

    print("[ERROR] 运行作业文件失败，程序退出!")

    return -1

print("[INFO] 运行作业文件成功")

print()
```

4.10 实时运动控制接口

头文件：include/tl_interface.h

4.10.1 robot_movej - 关节空间运动到目标位姿

方法：robot_movej(SOCKETFD socketFd,MoveCmd moveCmd);

功能描述：立即下发关节角度数据，驱动机械臂实时运动到目标位置（在线点对点控制）。不依赖作业文件（**该功能需要在示教模式下使用**）。

参数：

- socketFd：已连接的句柄。
- moveCmd：运动指令结构体，包含目标位置和运动参数。
- targetPosValue 点位数组，n个轴就赋值前n位数组，其余置0。
- vel 速度，参数范围：0\<vel≤100 单位 %。

- coord 坐标系，参数范围：0≤coord≤3。
- acc 加速度，参数范围：0\<acc≤100。
- dec 减速度，参数范围：0\<dec≤100。
- isSync 是否同步模式 true同步 false不同步。

返回值：

- 0：成功
- 非 0：失败

使用示例：注意使用时要先上电

```
# ---- MoveJ运动控制 ----

print("[INFO]**********MoveJ运动控制**********")

cmd = MoveCmd()

cmd.targetPosType = PosType_PType

cmd.targetPosName = ""

cmd.coord = 0 # 关节坐标系

cmd.velocity = 40 # 关节速度百分比 (1,100]

cmd.acc = 10

cmd.dec = 10

cmd.pl = 0 # 平滑参数 [0,5]

cmd.targetPosValue = VectorDouble([

20.0, 10.0, -10.0,

0.0, 0.0, 0.0, 0.0]) # 关节坐标下单位为度

ret = robot_movej(socket_fd, cmd)

if ret != SUCCESS:

    print("[ERROR] MoveJ运动控制失败，程序退出!")

    return -1

# 等待运动到位
```

```
wait_motion_done(socket_fd, 30)

print("[INFO] MoveJ运动控制成功")
```

4.10.2 robot_movel - 笛卡尔空间直线运动到目标位姿

方法：robot_movel(SOCKETFD socketFd, MoveCmd moveCmd);

功能描述：在笛卡尔空间中，控制机械臂从当前位姿沿直线运动至目标位姿。（**该功能需要在示教模式下使用**）。

参数

- socketFd：已连接的句柄（机器人连接成功后返回的 ID）。
- move_cmd：运动指令结构体，包含笛卡尔目标位姿及运动参数。
- coord = 1（直角坐标系）。
- targetPosValue 7维：[X(mm), Y(mm), Z(mm), RX(rad), RY(rad),RZ(rad), 0]。
- vel 速度，参数范围：0\<vel≤100 单位 %。
- coord 坐标系，参数范围：0≤coord≤3。
- acc 加速度，参数范围：0\<acc≤100。
- dec 减速度，参数范围：0\<dec≤100。

返回值

- 0：成功
- 非 0：失败

使用示例：注意使用时要先上电

```
# ---- MoveL运动控制 ----

print("[INFO] **********MoveL运动控制**********")

cmd.targetPosName = ""

cmd.coord = 1 # 直角坐标系

cmd.targetPosValue = VectorDouble([

280, 90.0, 270,

3.14, 0.0, 0.0]) # 直角坐标下位移单位是mm，旋转单位是弧度
```

```
ret = robot_movel(socket_fd, cmd)

if ret != SUCCESS:

    print("[ERROR] MoveL运动控制失败，程序退出!")

    return -1

# 等待运动到位

wait_motion_done(socket_fd, 30)

print("[INFO] MoveL运动控制成功")
```

4.10.3 robot_go_home - 机器人回原点（home）

方法：robot_go_home(socketFd);

功能描述：回到设定的零点。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

ret = tl.robot_go_home(socket_fd)

if ret == 0:

    print("机器人已回原点")
```

4.10.4 robot_go_to_reset_position - 机器人回到复位位置

方法：robot_go_to_reset_position(socketFd);

功能描述：回到控制器预设的复位点（若控制器未配置复位点，接口可能返回错误码）。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

ret = tl.robot_go_to_reset_position(socket_fd)
```

4.11 坐标系与用户坐标管理接口

头文件：include/tl_interface.h

4.11.1 set_user_coordinate_data - 设置用户坐标

方法：set_user_coordinate_data(SOCKETFD socketFd,int userNum, std::vector\<double> pos);

功能描述：标定用户坐标。

参数：

- socketFd：已连接的句柄。
- userNum：用户坐标系编号。
- pos：坐标系数据结构体。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# ----配置用户坐标系----

print("[INFO] **********配置用户坐标系**********")
```

```
userNum = 1

user_coord = [10, 20, 30, 0, 0, 0, 0]

ret = set_user_coordinate_data(socket_fd, userNum, user_coord)

if ret != SUCCESS:

    print("[ERROR] 配置用户坐标系失败，程序退出!")

    sys.exit(-1)

print("[INFO] 配置用户坐标系成功")

print()
```

4.11.2 set_user_coord_number - 设置用户坐标编号

方法：set_user_coord_number(SOCKETFD socketFd, int userNum);

参数：

- socketFd：已连接的句柄。
- userNum：用户坐标系编号。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# 设置用户坐标编号

coord_num = 1

# ret = tl.set_user_coord_number(socket_fd, coord_num)

# if ret == 0:

# print(f"用户坐标编号 {coord_num} 设置成功")

# else:
```

```
# print(f"用户坐标编号设置失败，错误码：{ret}")

print("设置用户坐标编号接口使用示例")

print(f"目标坐标编号: {coord_num}")

print("请参考具体SDK文档实现用户坐标编号设置")
```

4.11.3 get_user_coord_para - 获取当前使用的用户坐标的参数

方法：get_user_coord_para(SOCKETFD socketFd,int userNum,std::vector\<double>& pos);

功能描述：读取当前激活工件坐标系的原点偏移与欧拉旋转姿态参数。

参数：

- socketFd：已连接的句柄。
- userNum：用户坐标编号。
- pos 用户坐标参数。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
def test_get_user_coord_para(arm, ctx):

    """获取用户坐标参数"""

    pos = VectorDouble()

    ret = get_user_coord_para(arm.socket_fd, 0, pos)

    if ret == SUCCESS:

        values = [pos[i] for i in range(min(6, len(pos)))]

        return True, f"用户坐标参数: {print_vector(values)}"

    return True, f"获取返回: {ret}"
```

4.11.4 get_user_coord_number - 获取当前使用的用户坐标编号

方法：get_user_coord_number(SOCKETFD socketFd, int& userNum);

功能描述：读取控制器当前生效的工件坐标系索引号。

参数：

- socketFd：已连接的句柄。
- userNum：用户坐标编号。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
def test_get_user_coord_number(arm, ctx):
    """获取当前用户坐标编号"""
    num = -1
    ret, num = get_user_coord_number(arm.socket_fd, num)
    if ret == SUCCESS:
        return True, f"用户坐标编号: {num}"
return True, f"获取返回: {ret}"
```

4.11.5 get_current_coord - 获取当前工作坐标系

方法：get_current_coord(SOCKETFD socketFd, int& coord);

功能描述：获取机器人当前坐标系。

参数：

- socketFd：已连接的句柄。
- coord：输出参数，0：关节 1：直角 2：工具 3：用户。

返回值：

- 0：成功

- 非 0：失败

使用示例：

```
# ----获取当前工作坐标系----

print("[INFO] **********获取当前工作坐标系**********")

coord = -1

ret, coord = get_current_coord(socket_fd, coord)

if ret != SUCCESS:

    print("[ERROR] 获取当前工作坐标系失败，程序退出!")

    sys.exit(-1)

print("[INFO] 当前坐标系编号为：" + str(coord))
```

4.11.6 set_current_coord - 切换示教运动坐标系

方法：set_current_coord(SOCKETFD socketFd, int coord);

功能描述：设置机器人当前坐标系。

参数：

- socketFd：已连接的句柄。
- coord：坐标系类型。
    - 0：关节坐标系。
    - 1：直角坐标系。
    - 2：工具坐标系。
    - 3：用户坐标系。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl
```

```python
# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# 设置当前坐标系为关节坐标系

coord = 0

ret = tl.set_current_coord(socket_fd, coord)

print(f"设置当前坐标系返回值(ret) = {ret}")
```

4.11.7 calibration_oxy - 三点法自动设置用户坐标系

方法：calibration_oxy(SOCKETFD socketFd, int userNum, std::string xyo);

功能描述：通过输入原点、X 轴点、Y 轴点三组坐标，以三点法计算生成工件用户坐标系。

参数：

- socketFd：已连接的句柄。
- userNum：用户坐标编号。
- xyo：值 \'X\' \'Y\' \'O\'。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```python
# ----三点法自动设置用户坐标系----

print("[INFO] **********三点法自动设置用户坐标系**********")

print("[INFO] 请将机械臂移动到用户坐标系原点O后按Enter键继续...")

input()

ret = calibration_oxy(socket_fd, 1, "O")

if ret != SUCCESS:

    print("[ERROR] 标定原点O失败，错误码: " + str(ret))

    sys.exit(-1)
```

```
print("[INFO] 请将机械臂移动到X轴正方向上一点后按Enter键继续...")

input()

ret = calibration_oxy(socket_fd, 1, "X")

if ret != SUCCESS:

    print("[ERROR] 标定X轴方向点失败，错误码: " + str(ret))

    sys.exit(-1)

print("[INFO] 请将机械臂移动到XY平面内Y轴正方向上一点后按Enter键继续...")

input()

ret = calibration_oxy(socket_fd, 1, "Y")

if ret != SUCCESS:

    print("[ERROR] 标定Y轴方向点失败，错误码: " + str(ret))

    sys.exit(-1)

ret = calculate_user_coordinate(socket_fd, 1)

if ret != SUCCESS:

    print("[ERROR] 计算用户坐标系失败，错误码: " + str(ret))

    sys.exit(-1)

print("[INFO] 三点法标定用户坐标系成功")
```

4.11.8 calculate_user_coordinate - 计算用户坐标

方法：calculate_user_coordinate(socketFd, userNumber);

功能描述：计算用户坐标。

参数：

- socketFd：已连接的句柄。
- userNumber：用户坐标编号。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

ret = tl.calculate_user_coordinate(socket_fd, 1)
```

4.11.9 get_target_coord_pos_value - 获取指定坐标下的位置

方法：get_target_coord_pos_value(socketFd, name, targetCoord, targetPos);

功能描述：获取指定点位在目标坐标系下的位置。

参数：

- socketFd：已连接的句柄。
- name：点位名（P 点、GP 点、E 点、GE 点），如 "GP0001"。
- targetCoord：目标坐标系，0 关节、1 基坐标、2 工具、3 用户。
- targetPos：输出参数，14 位点位数组。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

target_pos = VectorDouble()

ret = tl.get_target_coord_pos_value(socket_fd, "GP0001", 0, target_pos)
```

4.11.10 get_local_pos_p_value - 获取本地位置变量 P 值

方法：get_local_pos_p_value(socketFd, name, pos);

功能描述：获取局部 P 点。

参数：

- socketFd：已连接的句柄。
- name：点位名，范围 P0001 - P9999。
- pos：输出参数，14 位点位数组。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

pos = VectorDouble()

ret = tl.get_local_pos_p_value(socket_fd, "P0001", pos)
```

4.11.11 set_default_cartesian_params - 设置笛卡尔参数为默认值

方法：set_default_cartesian_params(SOCKETFD socketFd);

功能描述：设置机械臂末端的笛卡尔参数为默认值。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# ----设置机械臂末端参数为默认值----

print("[INFO] **********设置机械臂末端参数为默认值**********")

ret = set_default_cartesian_params(socket_fd)

if ret != SUCCESS:
```

```
    print("[ERROR] 设置机械臂末端参数为默认值失败，错误码: " + str(ret))
    sys.exit(-1)
print("[INFO] 设置机械臂末端参数为默认值成功")
```

4.12 工具手相关接口

头文件：include/tl_interface.h

4.12.1 get_tool_hand_number - 获取当前使用的工具手编号

方法：get_tool_hand_number(socket_fd, tool_num);

功能描述：获取当前使用的工具手编号。

参数：

- socket_fd：已连接的句柄。
- tool_num：输出参数，用于存储工具号的整数变量。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl
# 建立连接
socket_fd = tl.connect_robot("192.168.1.13", "6001")
# 定义整数变量，用于接收结果
toolNum = 0
# 获取工具号
ret = tl.get_tool_hand_number(socket_fd, toolNum)
print(f"获取工具号返回值(ret) = {ret}")
print(f"当前工具号 = {toolNum}")
```

4.12.2 set_tool_hand_number - 设置工具手编号

方法：set_tool_hand_number(SOCKETFD socketFd, int toolNum);

功能描述：设定末端工具对应工具组编号，用于控制器匹配工具坐标系、负载参数及碰撞检测包络配置。

参数：

- socketFd：已连接的句柄。
- toolNum：工具手编号。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
test_cases = [
    (1,  "正常值"),
    (2,  "正常值"),
    (10, "较大值"),
    (0,  "边界值: 0"),
    (20, "边界值: 20"),
]
for tool_num, desc in test_cases:
    print("\\n  --- 设置 toolNum=%d (%s) ---" % (tool_num, desc))
    ret = set_tool_hand_number(sock, tool_num)
    if ret != SUCCESS:
        print_fail("set_tool_hand_number(%d)" % tool_num, ret)
        fail_count += 1
        continue
    time.sleep(0.1)
```

4.12.3 set_tool_hand_param - 设置工具手参数

方法：set_tool_hand_param(SOCKETFD socketFd, int toolNum, ToolParam param);

功能描述：配置指定编号的工具手坐标系偏移参数（TCP）。

参数：

- socketFd：已连接的句柄。
- toolNum：工具手编号。
- param：要修改的参数 长度6 X轴偏移量 Y轴偏移量 Z轴偏移量 绕A轴旋转量绕B轴旋转量 绕C轴旋转量。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# 注意: set_tool_hand_param/set_user_coordinate_data 涉及配置修改，

#       错误的值可能导致路径偏差，请确认参数正确。

# ----设置工具手参数---

print("[INFO] **********设置工具手参数**********")

tool_num = 1

tool_param = ToolParam()

tool_param.X = 10

tool_param.Y = 10

tool_param.Z = 10

tool_param.A = 0

tool_param.B = 0

tool_param.C = 0

ret = set_tool_hand_param(socket_fd, tool_num, tool_param)

if ret != SUCCESS:
```

```
    print("[ERROR] 设置工具手参数失败，程序退出!")
    sys.exit(-1)
print("[INFO] 设置工具手参数成功")
print()
```

4.12.4 get_tool_hand_param - 获取当前工具手参数

方法：get_tool_hand_param(SOCKETFD socketFd,int toolNum, ToolParam& param);

功能描述：读取当前选定工具手的TCP六维偏移姿态、末端负载重量、转动惯量以及负载质心坐标，为动力学计算与运动平顺性控制提供负载数据。

参数：

- socketFd：已连接的句柄。
- toolNum：工具手编号。
- param：工具手参数，ToolParam结构体。

返回值：

- 0：成功
- 非0：失败

使用示例：

```
def test_get_tool_param(arm, ctx):
    """获取工具手参数"""
    tp = ToolParam()
    try:
        ret = get_tool_hand_param(arm.socket_fd, 0, tp)
        if ret == SUCCESS:
            return True, (f"工具参数: X={tp.X:.3f} Y={tp.Y:.3f} "
                          f"Z={tp.Z:.3f} A={tp.A:.3f} B={tp.B:.3f} C={tp.C:.3f}")
        return True, f"获取返回: {ret}"
    except Exception as e:
```

```
        return True, f"接口异常: {e}"
```

4.12.5 set_tool_coordinate_range - 设置工具包络球参数（工具坐标范围参数）

方法：set_tool_coordinate_range(SOCKETFD socketFd, int tool_number, const ToolCoordinateRange& range_param);

功能描述：配置末端工具包络球体边界参数，用于机械臂运动过程中的碰撞干涉预判与自碰撞检测，限定工具实际占用空间范围。

参数：

- socket_fd：已连接的句柄。
- tool_number：工具编号。
- range_param；工具坐标范围参数结构体。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
    # ----设置工具包络球参数----
print("[INFO] **********设置工具包络球参数**********")
tool_number = 0
range_param = ToolCoordinateRange()
range_param.max_range_x = 200
range_param.min_range_x = -200
range_param.max_range_y = 200
range_param.min_range_y = -200
range_param.max_range_z = 200
range_param.min_range_z = -200
range_param.tool_hand_method = 0
ret = set_tool_coordinate_range(socket_fd, tool_number, range_param)
```

```
if ret != SUCCESS:
    print("[ERROR] 设置工具包络球参数失败，错误码： " + str(ret))
    sys.exit(-1)
print("[INFO] 设置工具包络球参数成功")
```

4.12.6 get_tool_coordinate_range - 获取工具包络球参数（工具坐标范围参数）

方法：get_tool_coordinate_range(SOCKETFD socketFd, int tool_number, ToolCoordinateRange& range_param);

功能描述：读取末端执行器包络球体空间尺寸参数，为机械臂运动碰撞检测、路径干涉校验提供工具实体边界数据。

参数：

- socketFd：已连接的句柄。
- tool_number：工具编号。
- range_param：工具坐标范围参数结构体。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
"""获取工具包络球参数（get_tool_coordinate_range）"""
print("[INFO] **********获取工具包络球参数**********")
tool_number = 0
range_param = ToolCoordinateRange()
ret = get_tool_coordinate_range(socket_fd, tool_number, range_param)
if ret != SUCCESS:
    print("[ERROR] 获取工具包络球参数失败，错误码： " + str(ret))
    return
print("[INFO] X轴范围: [" + str(range_param.min_range_x) + ", " + str(range_param.max_ran
```

```
ge_x)+"]")

print("[INFO] Y轴范围: [" + str(range_param.min_range_y) + ", " + str(range_param.max_ran
ge_y)+"]")

print("[INFO] Z轴范围: [" + str(range_param.min_range_z) + ", " + str(range_param.max_ran
ge_z)+"]")

print("[INFO] 工具握持方式: " + str(range_param.tool_hand_method))

print("")
```

4.13 示教操作接口

头文件：include/tl_interface.h

4.13.1 robot_start_jogging - 点动

方法：robot_start_jogging(SOCKETFD socketFd, int axis, bool dir);

功能描述：使指定轴向指定方向连续运动。注意：该指令需要周期性调用才能维持运动（通常每40ms调用一次），或与robot_stop_jogging 配合使用，支持**直角坐标系**和**关节坐标系**的点动。

参数：

- socketFd：已连接的句柄。
- axis：轴号，1\~7 代表第 1\~7 轴。
- dir:方向。true 正向 false 反向。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# robot_start_jogging 会直接驱动机器人运动，运行前确保安全区域无人

axis = 1
```

```
direction = True

ret = tl.robot_start_jogging(socket_fd, axis, direction)

if ret == 0:

    print("[INFO] 示教点动成功")

else:

    print(f"[ERROR] 示教点动失败，错误码：{ret}")
```

4.13.2 robot_stop_jogging -- 停止点动

方法：robot_stop_jogging(SOCKETFD socketFd, int axis);

功能描述：使指定轴停止点动。

参数：

- socketFd：已连接的句柄。
- axis：轴号（1\~6）。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
print("[INFO] **********停止点动**********")

axis = 1

ret = robot_stop_jogging(arm.socket_fd, axis)

if ret != SUCCESS:

    print("[ERROR] 停止点动失败，程序退出!")

    sys.exit(-1)

print("[INFO] 停止点动成功")
```

4.14 全局路点管理接口

头文件：include/tl_interface.h

4.14.1 set_global_position - 新增全局路点/设置全局路点

方法：set_global_position(SOCKETFD socketFd, std::string posName, std::vector\<double> posInfo);

功能描述：设置全局GP点位。

参数：

- socketNd：已连接的句柄。
- posName：全局路点名称（如 \"GP0001\"）。
- posInfo：位置坐标数据列表，Python列表格式（list）。根据底层运动规划，要求传入 7 维双精度浮点数：
    - 直角坐标系模式：承载 [X, Y, Z, RX, RY, RZ]，分别对应空间三维毫米位姿、欧拉角弧度。
    - 关节坐标系模式：承载 1 \~ 7 轴的各关节目标运行度数。对于标准 6 轴机械臂，第 7 位请传入 0.0 作为物理占位符。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# ----设置全局路点----

print("[INFO] **********设置全局路点**********")

posName = "GP_test0"

posInfo = [0, 1, 2, 3, 4, 5, 6]

ret = set_global_position(socket_fd, posName, posInfo)

if ret != SUCCESS:

    print("[ERROR] 设置全局路点失败，程序退出!")

    sys.exit(-1)
```

```
print("[INFO] 设置全局路点成功")
```

4.14.2 get_global_position - 查询全局 GP 点位

方法：get_global_position(SOCKETFD socketFd, std::string posName, std::vector\<double>& pos);

参数：

- socketFd：已连接的句柄。
- posName：坐标名称。
- pos：输出参数，用于存储坐标值的向量。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# ----查询全局路点----

print("[INFO] **********查询全局路点**********")

queriedPos = VectorDouble()

ret = get_global_position(socket_fd, posName, queriedPos)

if ret != SUCCESS:

    print("[ERROR] 查询全局路点失败，程序退出!")

    sys.exit(-1)

print("[INFO] 查询全局路点成功，位置为：", end="")

for pos in queriedPos:

    print(str(pos) + " ", end="")
```

4.14.3 set_global_string - 设置全局字符串变量

方法：set_global_string(socketFd, varName, varValue);

功能描述：设置字符串全局变量。

参数：

- socketFd：已连接的句柄。
- varName：变量名，如 "GS001"。
- varValue：字符串变量值。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

ret = tl.set_global_string(socket_fd, "GS001", "hello")
```

4.14.4 get_global_string - 获取全局字符串变量

方法：get_global_string(socketFd, varName, value);

功能描述：查询字符串全局变量。

参数：

- socketFd：已连接的句柄。
- varName：变量名，如 "GS001"。
- value：出参占位，传入字符串初始值（如 `""`），查询结果由返回值元组第二项给出。

返回值：

- tuple：(ret, value)，value 为变量值
- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")
```

```
value = ""  # 占位参数（std::string& 出参）

ret, value = tl.get_global_string(socket_fd, "GS001", value)
```

4.14.5 get_global_sync_position - 获取全局同步位置

方法：get_global_sync_position(socketFd, posName, pos);

功能描述：查询全局 GE 点位。

参数：

- socketFd：已连接的句柄。
- posName：全局位置名，如 "GE0001"。
- pos：输出参数，21 位点位数组（前 7 位坐标姿态、中间 7 位机器人位置、后 7 位外部轴位置）。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

pos = VectorDouble()

ret = tl.get_global_sync_position(socket_fd, "GE0001", pos)
```

4.14.6 get_global_variant - 获取全局变量

方法：get_global_variant(socketFd, varName, value);

功能描述：查询全局变量。

参数：

- socketFd：已连接的句柄。
- varName：全局变量名，支持 "GI001" / "GD001" / "GB001" 等形式。
- value：出参占位，传入数值占位值（如 `0.0`），查询结果由返回值元组第二项给出。

返回值：

- tuple：(ret, value)，ret 为执行结果，value 为变量值
- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

value = 0.0  # 占位参数（double& 出参）

ret, value = tl.get_global_variant(socket_fd, "GI001", value)

if ret == 0:

    print(f"全局变量值：{value}")
```

4.15 模式与 IO 控制接口

头文件：include/tl_interface.h

4.15.1 set_current_mode - 设置机器人模式（切换示教模式和运动模式）

方法：set_current_mode(SOCKETFD socketFd, int mode);

功能描述：切换控制器运行模式：0=示教模式（手动），1=远程模式，2=自动模式（运行作业）。

参数：

- socketFd：已连接的句柄。
- mode：模式
  - 0：示教模式。
  - 1：远程模式。
  - 2：运行模式，切换为运行模式后将自动进入伺服上电状态，无须额外上电。

返回值：

- 0：成功

- 非 0：失败

使用示例：

```
import tl_interface as tl

import time

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# 设置机器人当前模式

# mode 参数说明：

# 模式 0：示教 1：远程 2：运行

mode = 2 # 运行模式

ret = tl.set_current_mode(socket_fd, mode)

print(f"设置模式返回值(ret) = {ret}")
```

4.15.2 set_digital_output - 设置数字 IO 输出

方法：set_digital_output(SOCKETFD socketFd, int port, int value);

功能描述：设置指定数字输出端口的状态。value：1高电平，0低电平。

参数：

- socketFd：已连接的句柄。
- port：输出端口号，从1开始到最大端口号。
- value：输出值
    - 1：ON。
    - 0：OFF。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# ----设置数字 IO 输出----

print("[INFO] **********设置数字 IO 输出**********")

io_port = 1

value = 0

ret = set_digital_output(socket_fd, io_port, value)

if ret != SUCCESS:

    print("[ERROR] 设置数字 IO 输出失败，程序退出!")

    return -1

print("[INFO] 设置数字 IO 输出成功")
```

注意：port的值不能设置为0，否则将会报错[0x7180]参数错误

4.15.3 get_digital_output - 获取所有数字 IO 输出

方法：get_digital_output(SOCKETFD socketFd, std::vector\<int>& out);

功能描述：读取所有数字输出端口的电平状态（通常64路）。

参数：

- socketFd：已连接的句柄。
- out：输出参数，用于存储所有数字输出状态的向量。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# ----查询数字 IO 输出----

print("[INFO] **********查询数字 IO 输出**********")

outputValues = VectorInt()

ret = get_digital_output(socket_fd, outputValues)

if ret != SUCCESS:
```

```
    print("[ERROR] 查询数字 IO 输出失败，程序退出!")
    return -1
print("[INFO] 查询数字 IO 输出成功，状态为：", end="")
for state in outputValues:
    print("%d " % state, end="")
```

4.15.4 get_digital_input - 获取所有 IO 输入状态

方法：get_digital_input(SOCKETFD socketFd,std::vector\<int>& in);

参数：

- socketFd：已连接的句柄。
- in：输出参数，用于存储所有数字输入状态的向量。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
print("[INFO] **********获取所有 IO输入状态**********")
inputValues = VectorInt()
ret = get_digital_input(socket_fd, inputValues)
if ret != SUCCESS:
    print("[ERROR] 查询数字 IO 输入失败，程序退出!")
    return -1
print("[INFO] 查询数字 IO 输入成功，状态为：", end="")
for state in inputValues:
    print("%d " % state, end="")
```

4.15.5 set_analog_output - 设置模拟量输出

方法：set_analog_output(socketFd, port, value);

功能描述：设置模拟输出。

参数：

- socketFd：已连接的句柄。
- port：端口号。
- value：数值，参数范围 0≤value≤10。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

ret = tl.set_analog_output(socket_fd, 1, 5.0)
```

4.15.6 get_analog_input - 获取模拟量输入

方法：get_analog_input(socketFd, ain);

功能描述：查询模拟输入。

参数：

- socketFd：已连接的句柄。
- ain：输出参数，模拟输入数组，最大长度 64。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

ain = VectorDouble()
```

```
ret = tl.get_analog_input(socket_fd, ain)
```

4.15.7 get_analog_output - 获取模拟量输出

方法：get_analog_output(socketFd, aout);

功能描述：查询模拟输出。

参数：

- socketFd：已连接的句柄。
- aout：输出参数，模拟输出数组，最大长度 64。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

aout = VectorDouble()

ret = tl.get_analog_output(socket_fd, aout)
```

4.15.8 get_io_type - 获取 IO 类型

方法：get_io_type(socketFd, io_type);

功能描述：IO 型号查询。

参数：

- socketFd：已连接的句柄。
- io_type：IOtype 输出参数（num IO 板数量；type IO 型号；io_port_sum IO 端口数量一维数组：[数字输入、数字输出、模拟输入、模拟输出端口数量]）。

返回值：

- 0：成功

- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

io_type = IOtype()

ret = tl.get_io_type(socket_fd, io_type)
```

4.15.9 get_IO_reset_function - 获取 IO 复位功能

方法：get_IO_reset_function(socketFd, robotNum, type);

功能描述：获取 IO 复位相关参数。

参数：

- socketFd：已连接的句柄。
- robotNum：机器人编号（1-4）。
- type：设置类型，1 远程IO复位 / 2 切模式停止 / 3 程序报错。

返回值：

- tuple：(ret, enable, value)，enable 复位使能容器、value 复位值容器
- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

ret, enable, value = tl.get_IO_reset_function(socket_fd, 1, 1)
```

4.16 算法接口

头文件：include/tl_interface.h

4.16.1 get_origin_coord_to_target_coord -- 坐标转换算法

方法：get_origin_coord_to_target_coord(SOCKETFD socketFd, int originCoord, std::vector\<double> originPos, int targetCoord,std::vector\<double>& targetPos,int form = 0,const std::vector\<double>& reference_pos={});

功能描述：在不同坐标系之间进行空间坐标转换，当从关节坐标系转换为直角坐标系时为正运动学解算，当从直角坐标系转换为关节坐标系时为逆运动学解算（接口仅做纯数学数值运算，不会驱动机械臂运动、不会触发伺服下电；遇到奇异点位姿、目标位姿超出可达空间时，直接返回非 0错误码）;接口依赖机械臂通信链路，建立机械臂连接后才可调用。工具坐标系（TOOL）转换由控制器端实现（语义与控制器端一致，SDK 透传）。

参数：

- socketFd：已连接的句柄。
- originCoord：原坐标系，0 1 2 3 关节 直角 工具 用户。
- originPos：原坐标数据。
- targetCoord：目标坐标系，0 1 2 3 关节 直角 工具 用户
- targetPos：转换后的坐标。
- form：形态。
- reference_pos：参考点。

返回值：

- 0：成功
- 非 0：失败

修正：当前版本逆解不会下电，只会返回错误码

使用示例：

```
# ============================================================
# 3. 坐标系转换（获取机器人当前位姿在不同坐标系下的表示）
# ============================================================
# 单位说明:
#   关节坐标 [J1..J6] --- 度 °
#   直角坐标 [X,Y,Z,Rx,Ry,Rz] --- 位移 mm，姿态 rad
```

```
#   工具/用户坐标 --- 同直角坐标

print_separator("坐标系转换")

print("--- 3. get_origin_coord_to_target_coord ---")

print("  坐标系: 0=关节(°)  1=直角(mm+rad)  2=工具(mm+rad) 3=用户(mm+rad)")

# 从关节坐标系(0) 转换到直角坐标系(1)

# 关节角度: J1=10°, J2=20°, J3=30°, J4=0°, J5=0°, J6=0°

joint_pos = VectorDouble()

joint_pos.push_back(10.0)

joint_pos.push_back(20.0)

joint_pos.push_back(30.0)

joint_pos.push_back(0.0)

joint_pos.push_back(0.0)

joint_pos.push_back(0.0)

joint_pos.push_back(0.0)

cartesian_pos = VectorDouble()

origin_reference_pos = VectorDouble([0]*7)

ret = get_origin_coord_to_target_coord(sock, 0, joint_pos, 1, cartesian_pos, 0, origin_r
eference_pos)

print_result("get_origin_coord_to_target_coord (关节→直角) 正解", ret)

print_vector("输入关节角度 [°]", joint_pos)

if ret == SUCCESS:

    print("  输出直角坐标 --- 前3位为位移[mm]，后3位为姿态[rad]:")

    print_vector("  直角坐标", cartesian_pos)

    if len(cartesian_pos) >= 6:

        print("    位移 X=" + str(cartesian_pos[0]) + "mm, Y=" + str(cartesian_pos[1])

              + "mm, Z=" + str(cartesian_pos[2]) + "mm")

        print("    姿态 Rx=" + str(cartesian_pos[3]) + "rad, Ry=" + str(cartesian_pos[4])
```

```
            + "rad, Rz=" + str(cartesian_pos[5]) + "rad")

        print("    姿态换算(°): Rx=" + str(cartesian_pos[3] *180.0 / math.pi)

              + ", Ry=" + str(cartesian_pos[4] * 180.0 / math.pi)

              + ", Rz=" + str(cartesian_pos[5] * 180.0 / math.pi))

# 逆解: 直角坐标系(1) → 关节坐标系(0)

# 使用正解输出的直角坐标作为输入，验证正反解一致性

if ret == SUCCESS and len(cartesian_pos) >= 6:

    joint_back = VectorDouble()

    ret = get_origin_coord_to_target_coord(sock, 1, cartesian_pos, 0, joint_back, 0, ori
gin_reference_pos)

    print_result("get_origin_coord_to_target_coord (直角→关节) 逆解", ret)

    print("  输入直角坐标:")

    print("    位移 [mm]: X=" + str(cartesian_pos[0]) + ", Y=" + str(cartesian_pos[1])

          + ", Z=" + str(cartesian_pos[2]))

    print("    姿态 [rad]: Rx=" + str(cartesian_pos[3]) + ",Ry=" + str(cartesian_pos[4])

          + ", Rz=" + str(cartesian_pos[5]))

    if ret == SUCCESS:

        print_vector("  逆解关节角度 [°]", joint_back)

        # 对比原始关节角度，计算误差

        err = 0.0

        for i in range(min(len(joint_pos), len(joint_back))):

            err += abs(joint_pos[i] - joint_back[i])

        print("  正反解误差 (应接近0): " + str(err))
```

4.16.2 get_quat2rpy - 四元数 转 欧拉角

方法：get_quat2rpy(SOCKETFD socketFd, std::vector\<double> quat_vector, std::vector\<double>& rpy_res);

功能介绍：将标准四元数（x, y, z, w）转换为Rx-Ry-Rz（绕机体 X、Y、Z 轴依次旋转）顺序的 RPY 欧拉角，输出单位为弧度；接口依赖机械臂通信链路，建立机械臂连接后才可调用。

参数：

- socketFd：已连接的句柄。
- quat_vector: 被转换的四元数。
- rpy_res: 接收欧拉角结果。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# RPY ↔ 四元数 正反验证

rpy_in = tl.VectorDouble([0.2, 0.3, 0.5])   # rad

quat = tl.VectorDouble()

rpy_out = tl.VectorDouble()

ret = tl.get_rpy2quat(socket_fd, rpy_in, quat)

if ret == 0:

    print("输入欧拉角 [rad]: " + str(list(rpy_in)))

    print("中间四元数 [x,y,z,w]: " + str(list(quat)))

ret = tl.get_quat2rpy(socket_fd, quat, rpy_out)

if ret == 0:

    print("还原欧拉角 [rad]: " + str(list(rpy_out)))

else:

    print(f"坐标转换失败，错误码：{ret}")
```

4.16.3 get_rpy2quat - 欧拉角 转 四元数

方法：get_rpy2quat(SOCKETFD socketFd, std::vector\<double> rpy_vector, std::vector\<double>& quat_res);

功能介绍：按Rx-Ry-Rz 机体轴旋转顺序，将 RPY 欧拉角转换为标准四元数（x, y, z, w）；调用前必须完成机械臂 Socket 通信连接，否则接口执行失败。

参数：

- socketFd：已连接的句柄。
- quat_vector：被转换的欧拉角。
- rpy_res：接收四元素的结果。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# 1a. 欧拉角 → 四元数 → 欧拉角（正反验证）

print("--- 1a. RPY ↔ 四元数 正反验证 ---")

rpy_in = [0.2, 0.3, 0.5]  # rad

quat = VectorDouble()

rpy_out = VectorDouble()

ret = get_rpy2quat(sock, rpy_in, quat)

print("get_rpy2quat (RPY→四元数)", ret)

if ret != SUCCESS:

    sys.exit(-1)

print_vector("  输入欧拉角 [rad]", rpy_in)

print_vector("  中间四元数 [x, y, z, w]", quat)

ret = get_quat2rpy(sock, quat, rpy_out)

print("get_quat2rpy (四元数→RPY)", ret)
```

```
if ret != SUCCESS:

    sys.exit(-1)

print_vector("  还原欧拉角 [rad]", rpy_out)
```

4.16.4 get_rpy2r - 欧拉角 转 旋转矩阵

方法：get_rpy2r(SOCKETFD socketFd, std::vector\<double> rpy_vector, std::vector\<double>& r_res);

功能介绍：按照Rx-Ry-Rz 机体轴旋转顺序，将 RPY 欧拉角换算为 3×3 旋转矩阵；需先建立机械臂 Socket 通信连接才可调用接口，矩阵存储采用行主序排布。

参数：

- socketFd：已连接的句柄。
- rpy_vector：被转换的欧拉角。
- r_res：接收旋转矩阵的结果。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# 1b. 欧拉角 → 旋转矩阵

print("\\n--- 1b. get_rpy2r: 欧拉角转旋转矩阵 ---")

rpy = [0.0, 0.0, math.pi / 2]  # rad

rot_mat = VectorDouble()

ret = get_rpy2r(sock, rpy, rot_mat)

print("get_rpy2r", ret)

print_vector("输入欧拉角 [rad]", rpy)

if ret == SUCCESS:

    print_matrix("输出旋转矩阵 (3x3, 行主序, 无量纲)", rot_mat, 3, 3)
```

4.16.5 get_tr2r- 位姿 转 旋转矩阵

方法：get_tr2r(SOCKETFD socketFd, std::vector\<double> tr_matrix, std::vector\<double>& r_res);

功能介绍：从标准 4×4 齐次位姿（TR）矩阵中剥离提取左上角 3×3 旋转子矩阵；调用前必须建立机械臂 Socket 通信连接，位姿平移分量单位为 mm，矩阵整体采用行主序存储。

参数：

- socketFd：已连接的句柄。
- tr_matrix:被转换的位姿矩阵。
- r_res:接收旋转矩阵的结果。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# ============================================================

# 2. 位姿矩阵 ↔ 旋转矩阵 --- 正反验证

# ============================================================

# 单位说明:

# 旋转矩阵 R (3×3) --- 无量纲

# 平移向量 T [Tx,Ty,Tz] --- 毫米 mm

# 位姿矩阵 TR (4×4) --- R 部分无量纲，T 部分单位 mm

# 位姿矩阵 (行主序 16 元素):

# [Rxx Rxy Rxz Tx] 第 0 行

# [Ryx Ryy Ryz Ty] 第 1 行

# [Rzx Rzy Rzz Tz] 第 2 行

# [ 0 0 0 1] 第 3 行

print_separator("位姿矩阵 ↔ 旋转矩阵")

# 2a. TR → R → TR（正反验证）
```

```
print("--- 2a. TR ↔ R 正反验证 ---")

# 构造一个绕 Z 转 90°、平移 (100, 200, 300) mm 的位姿矩阵

cos90 = math.cos(math.pi / 2)

sin90 = math.sin(math.pi / 2)

tr_in = [

cos90, -sin90, 0.0, 100.0,

sin90, cos90, 0.0, 200.0,

0.0, 0.0, 1.0, 300.0,

0.0, 0.0, 0.0, 1.0

]

R = VectorDouble()

tr_out = VectorDouble()

ret = get_tr2r(sock, tr_in, R)

print_result("get_tr2r (TR→R)", ret)

if ret != SUCCESS:

    sys.exit(-1)

print_matrix("输入位姿矩阵 (4x4, R无量纲, T:mm)", tr_in, 4, 4)

print_matrix("中间旋转矩阵 (3x3, 无量纲)", R, 3, 3)

ret = get_r2tr(sock, R, tr_out)

print_result("get_r2tr (R→TR)", ret)

if ret != SUCCESS:

    sys.exit(-1)

print_matrix("还原位姿矩阵 (4x4, R无量纲, T:mm)", tr_out, 4, 4)

# 说明: get_tr2r 只提取旋转矩阵，平移 T 在此丢弃；

# get_r2tr 的输入不含平移，还原出的 TR 平移部分恒为 0。

# SDK 无 "R+T 构造 TR" 接口，完整位姿矩阵需由调用方自行拼装 T。

print(" 注: get_tr2r 仅输出旋转矩阵，平移 T 不参与往返；")
```

```
print(" 还原位姿矩阵的平移部分为 0，需调用方另行拼装。")

# 对比旋转部分（TR→R→TR 链路仅保证旋转信息无损）

# 注意：行主序 4x4 中旋转块位于 0,1,2, 4,5,6, 8,9,10，

# 位置 3/7/11 是平移列，不能按连续 0..8 遍历。

if len(tr_out) >= 16 and len(tr_in) >= 16:

    rot_idx = [0, 1, 2, 4, 5, 6, 8, 9, 10]

    r_err = 0.0

    for k in rot_idx:

        r_err += abs(tr_out[k] - tr_in[k])

    print(" 旋转矩阵误差 (应接近0)： " + str(r_err))
```

4.16.6 get_r2tr - 旋转矩阵 转 位姿矩阵

方法：get_r2tr(SOCKETFD socketFd, std::vector\<double>r_matrix, std::vector\<double>& tr_res);

功能介绍：将 3×3 旋转矩阵补全平移项，生成标准 4×4 齐次位姿（TR）矩阵；默认平移分量全部置 0，矩阵采用行主序存储，平移量单位为 mm，调用前需先建立机械臂 Socket通信连接。

参数：

- socketFd：已连接的句柄。
- r_matrix：被转换的旋转矩阵。
- tr_res：接收位姿矩阵结果。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# ============================================================

# 2. 位姿矩阵 ↔ 旋转矩阵 --- 正反验证

# ============================================================
```

```
# 单位说明:
#   旋转矩阵R (3×3)     --- 无量纲
#   平移向量T [Tx,Ty,Tz] --- 毫米 mm
#   位姿矩阵TR (4×4)  --- R 部分无量纲，T 部分单位 mm
# 位姿矩阵 (行主序 16 元素):
#   [Rxx Rxy Rxz Tx]  第 0 行
#   [Ryx Ryy Ryz Ty]  第 1 行
#   [Rzx Rzy Rzz Tz]  第 2 行
#   [  0   0   0  1]  第 3 行
print_separator("位姿矩阵 ↔ 旋转矩阵")
# 2a. TR → R → TR（正反验证）
print("--- 2a. TR ↔ R 正反验证 ---")
# 构造一个绕 Z 转 90°、平移 (100, 200, 300) mm 的位姿矩阵
cos90 = math.cos(math.pi / 2)
sin90 = math.sin(math.pi / 2)
tr_in = [
    cos90, -sin90, 0.0, 100.0,
    sin90,  cos90, 0.0, 200.0,
    0.0,    0.0,   1.0, 300.0,
    0.0,    0.0,   0.0, 1.0
]
R = VectorDouble()
tr_out = VectorDouble()
ret = get_tr2r(sock, tr_in, R)
print_result("get_tr2r (TR→R)", ret)
if ret != SUCCESS:
    sys.exit(-1)
```

```
print_matrix("输入位姿矩阵 (4x4, R无量纲, T:mm)", tr_in, 4, 4)

print_matrix("中间旋转矩阵 (3x3, 无量纲)", R, 3, 3)

ret = get_r2tr(sock, R, tr_out)

print_result("get_r2tr (R→TR)", ret)

if ret != SUCCESS:

    sys.exit(-1)

print_matrix("还原位姿矩阵 (4x4, R无量纲, T:mm)", tr_out, 4, 4)

# 说明: get_tr2r 只提取旋转矩阵，平移 T 在此丢弃；

# get_r2tr 的输入不含平移，还原出的 TR 平移部分恒为 0。

# SDK 无 "R+T 构造 TR" 接口，完整位姿矩阵需由调用方自行拼装 T。

print("  注: get_tr2r 仅输出旋转矩阵，平移 T 不参与往返；")

print("      还原位姿矩阵的平移部分为 0，需调用方另行拼装。")

# 对比旋转部分（TR→R→TR 链路仅保证旋转信息无损）

# 注意: 行主序 4x4 中旋转块位于 0,1,2, 4,5,6, 8,9,10,

#       位置 3/7/11 是平移列，不能按连续 0..8 遍历。

if len(tr_out) >= 16 and len(tr_in) >= 16:

    rot_idx = [0, 1, 2, 4, 5, 6, 8, 9, 10]

    r_err = 0.0

    for k in rot_idx:

        r_err += abs(tr_out[k] - tr_in[k])

    print("  旋转矩阵误差 (应接近0): " + str(r_err))
```

4.17 modbus通信接口

头文件：include/tl_interface.h

4.17.1 modbus_set_master_parameter - 设置 Modbus RTU 主站参数

方法：modbus_set_master_parameter(SOCKETFD socketFd, int id, const ModbusMasterParameter& param);

功能描述：配置 Modbus RTU 或 TCP主站参数。此接口用于设置通信协议栈参数。

参数：

- socketFd：已连接的句柄。
- master_id：主站 ID（0\~8）。
- param：ModbusMasterParameter 结构体，包含 type（\"RTU\" 或\"TCP\"），startAddress，以及嵌套的 RTU 或 TCP 子结构。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# 2.1 设置RTU 主站参数

param_rtu = ModbusMasterParameter()

param_rtu.type = "RTU"

param_rtu.startAddress = True

param_rtu.RTU.slaveId = 2             # 灵巧手从站 ID

param_rtu.RTU.port = 1                # 控制器串口号（根据接线修改）

param_rtu.RTU.baudrate = 115200       # 波特率

param_rtu.RTU.checkBit = "None"

param_rtu.RTU.dataBit = 8

param_rtu.RTU.stopBit = 1

ret = modbus_set_master_parameter(sock, 1, param_rtu)

print_result("modbus_set_master_parameter (id=1, RTU slaveId=2)", ret)

if ret != SUCCESS:

    disconnect_robot(sock)
```

```
    return -1

print("  从站ID: %d（灵巧手）" % param_rtu.RTU.slaveId)
```

4.17.2 modbus_open_master - 打开 Modbus 主站

方法：modbus_open_master(SOCKETFD socketFd, int id);

功能描述：激活指定的 Modbus 主站通道，开始总线通信。

参数：

- socketFd：已连接的句柄。
- master_id：主站 ID（0\~8）。

返回值：

- 0：成功
- 非 0：失败

使用示例：在打开前先配置Modbus RTU 参数

```
# 2.2 打开 Modbus 主站

ret = modbus_open_master(sock, 1)

print_result("modbus_open_master (id=1)", ret)

if ret != SUCCESS:

    disconnect_robot(sock)

    return -1

time.sleep(0.5)
```

4.17.3 modbus_write_multiple_holding_registers - 写多个保持寄存器

方法：modbus_write_multiple_holding_registers(SOCKETFD socketFd, int id, int address, const std::vector\<int>& data);

参数：

- socketFd：已连接的句柄。
- master_id：主站 ID（0\~8）。

- address：寄存器起始地址。
- data：待写入的数据列表。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

# 功能码 10H：写多个保持寄存器（id=0，起始地址 10）

write_data = tl.VectorInt([100, 200, 300])

ret = tl.modbus_write_multiple_holding_registers(socket_fd, 0, 10, write_data)

if ret == 0:

    print("  写入数据：" + str(list(write_data)))

else:

    print(f"写多个保持寄存器失败，错误码：{ret}")
```

4.17.4 modbus_read_holding_registers - Modbus RTU 读取多个保持寄存器

方法：modbus_read_holding_registers(SOCKETFD socketFd, int id, int address, int quantity, std::vector\<int>& data);

功能描述：使用功能码03读取保持寄存器。

参数：

- socketFd：已连接的句柄。
- id：主站 ID（0\~8）。
- address：寄存器起始地址。
- quantity: 要读取的寄存器个数。
- data: 读取到的数据（输出向量）。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
print_separator("TCP --- 03H 读保持寄存器")

read_data = VectorInt()

ret = modbus_read_holding_registers(sock, 0, 0, 3, read_data)

print_result("modbus_read_holding_registers (addr=0, qty=3)", ret)

if ret == SUCCESS:

    print("  保持寄存器[0\~2]: ", end="")

    for i in range(len(read_data)):

        print("%d" % read_data[i], end=" ")

    print()

    if len(read_data) > 0 and read_data[0] == 5678:

        print("  [验证] 06H 写入与 03H 读取一致")
```

4.17.5 modbus_write_signal_coil_status - Modbus RTU协议写线圈

方法：modbus_write_signal_coil_status(SOCKETFD socketFd, int id, int address, int data);

功能描述：基于 Modbus RTU协议向指定从站通道写入单路线圈状态，通过寄存器地址与写入数值完成数字量输出点位控制。

参数：

- socketFd：已连接的句柄。
- id：主站通道编号（0\~8）。
- address：起始寄存器地址。
- data：写入的参数

返回值：

- 0：成功

- 非 0：失败

使用示例：

```
print_separator("TCP --- 05H 写单线圈")

ret = modbus_write_signal_coil_status(sock, 0, 0, 0xFF00)

print_result("modbus_write_signal_coil_status (addr=0, ON)", ret)

time.sleep(0.2)
```

4.17.6 modbus_write_signal_holding_registers - Modbus RTU协议写保持寄存器

方法：modbus_write_signal_holding_registers(SOCKETFD socketFd, int id, int address, int data);

功能描述： 通过 Modbus RTU总线协议，对指定通讯通道的保持寄存器地址写入单个寄存器数值。

参数：

- socketFd：已连接的句柄。
- id：主站通道编号（0\~8）。
- address：起始寄存器地址。
- data：写入的参数

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
print_separator("Modbus-RTU --- 06H 写单寄存器")

ret = modbus_write_signal_holding_registers(sock, 0, 0, 5678)

print_result("modbus_write_signal_holding_registers (addr=0, val=5678)", ret)
```

4.17.7 modbus_read_coil_status -- Modbus读线圈状态

方法：modbus_read_coil_status(SOCKETFD socketFd, int id, int address, int quantity, std::vector\<int>& data);

功能描述： 通过 Modbus RTU总线协议功能码01H读取线圈（离散输出）状态；每个线圈返回 0 (断开) /1 (闭合)。

参数:

- socketFd：已连接的句柄。
- id：主站通道编号（0\~8）。
- address：起始线圈地址。
- quantity：需要读取的线圈数量。
- data：输出参数，存放读到的线圈状态，每个元素取值为 0 或 1。

返回值:

- 0：成功
- 非 0：失败

使用示例:

```
print_separator("Modbus-RTU --- 01H 读线圈状态")

read_data = VectorInt()

ret = modbus_read_coil_status(sock, 0, 0, 1, read_data)

print_result("modbus_read_coil_status (addr=0, qty=1)", ret)

if ret == SUCCESS and len(read_data) > 0:

    print("  线圈[0] = %d" % read_data[0])
```

4.17.8 modbus_read_input_status - Modbus读取输入状态

方法：modbus_read_input_status(SOCKETFD socketFd, int id, int address, int quantity, std::vector\<int>& data);

功能描述： 通过 Modbus RTU 总线协议，功能码02H，读取离散输入（输入状态），每个点位为比特位，取值0/1；离散输入为从站侧只读点位，不可写入。

参数:

- socketFd：已连接的句柄。
- id：主站通道编号（0\~8）。

- address：起始离散输入地址。
- quantity：待读取的离散输入点位数量。
- data：输出参数，存储读到的离散输入状态，每个元素取值为 0 或 1。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
print_separator("Modbus-RTU --- 02H 读输入状态")

read_data = VectorInt()

ret = modbus_read_input_status(sock, 0, 0, 4, read_data)

print_result("modbus_read_input_status (addr=0, qty=4)", ret)

if ret == SUCCESS:

    print("  输入状态: ", end="")

    for i in range(len(read_data)):

        print("%d" % read_data[i], end=" ")

    print()
```

4.17.9 modbus_read_input_registers - Modbus 读输入寄存器

方法：modbus_read_input_registers(SOCKETFD socketFd, int id, int address, int quantity, std::vector\<int>& data);

功能描述： 通过 Modbus RTU 总线协议，功能码04H，读取从站的输入寄存器。输入寄存器为从站只读 16 位寄存器，常用于采集传感器、模拟量等外部数据。

参数：

- socketFd：已连接的句柄。
- id：主站通道编号（0\~8）。
- address：输入寄存器的起始地址。
- quantity：待读取的输入寄存器数量。

- data：输出参数。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
print_separator("Modbus-RTU --- 04H 读输入寄存器")

read_data = VectorInt()

ret = modbus_read_input_registers(sock, 0, 0, 3, read_data)

print_result("modbus_read_input_registers (addr=0, qty=3)", ret)

if ret == SUCCESS:

    print("  输入寄存器: ", end="")

    for i in range(len(read_data)):

        print("%d" % read_data[i], end=" ")

    print()
```

4.18 队列运动接口

头文件：include/tl_interface.h

4.18.1 queue_motion_set_status - 打开/关闭队列运动模式

方法：queue_motion_set_status(SOCKETFD socketFd, boolstatus);

参数：

- socketFd：已连接的句柄。
- status：bool型True打开，False关闭。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# 1. 打开队列运动模式（会清空远端队列）

ret = queue_motion_set_status(sock, True)

print_result("queue_motion_set_status(true)", ret)

if ret != SUCCESS:

    return False
```

4.18.2 queue_motion_push_back_moveJ队列运动模式的本地队列最后插入一条moveJ运动

方法：queue_motion_push_back_moveJ(SOCKETFD socketFd, MoveCmd moveCmd);

功能描述：向本地队列尾部添加一条关节运动指令。

参数：

- socketFd：已连接的句柄。
- moveCmd：结构体。
- targetPos 目标点坐标 长度7位。
- velocity 参数范围(1,100] 度/s。
- acc，dec 参数范围(1,100]。
- pl 多条轨迹之间平滑参数 参数范围[0,5]。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# ============================================================

# MoveCmd 构建函数

# ============================================================

# @brief 构建 MoveCmd（关节坐标模式）
```

```python
# @param joints 6 个关节角度值，单位 °（度），顺序 [J1, J2, J3, J4, J5, J6]

# @param vel 百分比 % (1,100]，最大关节速度的百分比，实际受全局速度限制

# @param acc 百分比 % (1,100]

# @param dec 百分比 % (1,100]

# @param pl 平滑参数 [0,5]

# @note 实际运动速度 = 全局速度 × (vel / 100)

def make_movej_cmd(joints, vel=30, acc=50, dec=50, pl=0):

    cmd = MoveCmd()

    cmd.coord = 0 # 关节坐标系

    cmd.velocity = vel

    cmd.acc = acc

    cmd.dec = dec

    cmd.pl = pl

    # 填充 targetPosValue（前7位为关节值，后7位为外部轴，补零到14位）

    # 注意: 需要 VectorDouble 类型，不能用 Python list 直接赋值

    target = VectorDouble()

    for i in range(14):

        target.append(joints[i] if i < len(joints) else 0.0)

    cmd.targetPosValue = target

    return cmd

r = queue_motion_push_back_moveJ(sock, make_movej_cmd([0, 20, 0, 0, 0, 0], 30))

print(" push_back_moveJ: J1=0°, J2=20°")

print_result("push_back_moveJ", r)

if r != SUCCESS:

    return False
```

4.18.3 queue_motion_push_back_moveL队列运动模式的本地队列最后插入一条move L运动

方法：queue_motion_push_back_moveL(SOCKETFD socketFd, MoveCmd moveCmd);

功能描述：向本地队列尾部添加一条直线运动指令。

参数：

- socketFd：已连接的句柄。
- moveCmd：结构体。
- targetPos 目标点坐标 长度7位。
- velocity 参数范围(1,1000] mm/s。
- acc，dec 参数范围(1,100]。
- pl 多条轨迹之间平滑参数 参数范围[0,5]。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# @brief 构建 MoveCmd（直角坐标模式）

# @param pose 位姿 [X, Y, Z, Rx, Ry, Rz]，位置单位 mm，姿态单位 °

# @param vel 百分比 % (1,1000]，最大 TCP 线速度的百分比

# @param acc 百分比 % (1,100]

# @param dec 百分比 % (1,100]

# @param pl 平滑参数 [0,5]

# @note 实际运动速度 = 全局速度 × (vel / 100)

def make_movel_cmd(pose, vel=100, acc=50, dec=50, pl=0):

    cmd = MoveCmd()

    cmd.coord = 1 # 直角坐标系，可切换到 2：工具坐标系，3：用户坐标系

    cmd.velocity = vel
```

```
    cmd.acc = acc
    cmd.dec = dec
    cmd.pl = pl
    # 填充 targetPosValue（前7位为位姿值，后7位为外部轴，补零到14位）
    target = VectorDouble()
    for i in range(14):
        target.append(pose[i] if i < len(pose) else 0.0)
    cmd.targetPosValue = target
    return cmd
r = queue_motion_push_back_moveL(sock, make_movel_cmd([400, 100, 300, 3.141, 0, 0], 30))
print("  push_back_moveL:  X=400, Y=100, Z=300")
print_result("push_back_moveL", r)
if r != SUCCESS:
    return False
```

4.18.4 queue_motion_push_back_moveC - 队列运动本地队列末尾插入一条 MoveC 运动

方法：queue_motion_push_back_moveC(socketFd, moveCmd);

功能描述：队列运动模式的本地队列最后插入一条 MoveC 运动。

参数：

- socketFd：已连接的句柄。
- moveCmd：MoveCmd 结构体（targetPosValue 目标点坐标长度 7 位；velocity 参数范围 (1,1000] mm/s；acc、dec 参数范围 (1,100]；pl 平滑参数 [0,5]）。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

cmd = MoveCmd()

cmd.coord = 1

cmd.velocity = 100

cmd.targetPosValue = VectorDouble([100.0, 0.0, 200.0, 0.0, 0.0, 0.0, 0.0])

ret = tl.queue_motion_push_back_moveC(socket_fd, cmd)
```

4.18.5 queue_motion_push_back_moveS - 队列运动本地队列末尾插入一条 MoveS 运动

方法：queue_motion_push_back_moveS(socketFd, moveCmd);

功能描述：队列运动模式的本地队列最后插入一条 MoveS 运动。

参数：

- socketFd：已连接的句柄。
- moveCmd：MoveCmd 结构体（同 MoveC）。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

cmd = MoveCmd()

cmd.coord = 1

cmd.velocity = 100

cmd.targetPosValue = VectorDouble([100.0, 0.0, 200.0, 0.0, 0.0, 0.0, 0.0])

ret = tl.queue_motion_push_back_moveS(socket_fd, cmd)
```

4.18.6 queue_motion_send_to_controller - 将本地队列的前size个数据发送到控制器

方法：queue_motion_send_to_controller(SOCKETFD socketFd,int size,bool isContinue = false);

功能描述：将本地队列的前size个数据发送到控制器。

参数：

- socketFd：已连接的句柄。
- size：要发送的队列大小 size =0时将当前运动队列全部指令发送，范围[0,31]。
- is_continue：是否选择继续发送，true -继续；false-不继续，继续发送时可以继续发送点位，机器人不运动；不继续时机器人接收点位后立刻运动，默认为false。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# 4. 发送队列到控制器并等待执行完成

print(" [信息] 发送队列到控制器...")

ret = queue_motion_send_to_controller(sock, 0, False)

print_result("send_to_controller", ret)

if ret != SUCCESS:

    queue_motion_set_status(sock, False)

    return False

# 注意：控制器接收到队列后，将立马开始运动。调用前请先调用 queue_motion_set_status(true)。
# 是否选择继续发送：true-继续（可继续发送点位，机器人不运动）；false-不继续（机器人接收点位后立刻运动，默认 false）
```

4.18.7 queue_motion_get_status -- 查询队列模式状态

方法：queue_motion_get_status(SOCKETFD socketFd, bool& status);

功能描述：查询控制器当前是否打开队列运动模式。

参数：

- socketFd：已连接的句柄。
- status 用来存储返回值 false-关闭 true-已打开。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

status = False

_, status = tl.queue_motion_get_status(socket_fd, status)

print("  [信息] 队列模式状态: " + ("已开启" if status else "已关闭"))
```

4.18.8 queue_motion_size - 查询当前运动队列的长度

方法：queue_motion_size(SOCKETFD socketFd, int& size);

功能描述：查询当前运动队列的长度。

参数：

- socketFd：已连接的句柄。
- size：返回的队列长度。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# 3. 查询本地队列长度

# 注意: size 为输出参数
```

```
size = 0

ret, size = queue_motion_size(sock, size)

if ret == SUCCESS:

    print("  [信息] 本地队列长度: " + str(size) + " 条指令")
```

4.18.9 queue_motion_get_queuelen - 查询当前运动队列剩余的指令数量

方法：queue_motion_get_queuelen(SOCKETFD socketFd, int& len);

功能描述：查询当前运动队列剩余的指令数量。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# 辅助：等待队列剩余数减少到 target_len

def wait_queue_len(sock, target_len):

    while True:

        time.sleep(0.2)

        length = -1

        _, length = queue_motion_get_queuelen(sock, length)

        if length <= target_len or length == -1:

            break
```

4.18.10 queue_motion_suspend - 暂停连续运动（轨迹暂停）

方法：queue_motion_suspend(SOCKETFD socketFd);

功能描述：暂停队列正在执行的连续运动轨迹，伺服维持使能锁定，调用恢复指令即可从断点继续运行。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
r = queue_motion_suspend(sock)

print("暂停 (suspend)", r)

time.sleep(2)
```

4.18.11 queue_motion_restart - 暂停后再次运行运动队列（继续当前轨迹运动）

方法：queue_motion_restart(SOCKETFD socketFd);

功能描述：恢复已暂停的运动队列任务，从轨迹中断断点处继续执行剩余运动路径。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
r = queue_motion_restart(sock)

print_result("恢复 (restart)", r)

print(" [信息] 等待第 1 条完成...")

wait_queue_len(sock, 3)

print(" [信息] 第 1 条完成")
```

4.18.12 queue_motion_stop - 停止队列运动

方法：queue_motion_stop(socketFd);

功能描述：停止队列运动（暂停后可通过 restart 恢复）。**该接口已废弃**，建议使用 `queue_motion_stop_not_power_off` 替代（后续版本将移除）。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

ret = tl.queue_motion_stop(socket_fd)
```

4.18.13 queue_motion_stop_not_power_off - 停止连续运动保持上电状态（轨迹缓停）

方法：queue_motion_stop_not_power_off(SOCKETFD socketFd);

功能描述：对队列中待执行运动指令执行平滑缓停，伺服驱动器保持上电使能状态，可直接继续下发运动任务。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
r = queue_motion_stop_not_power_off(sock)

print("停止保持上电 (stop_not_power_off)", r)
```

```
time.sleep(1)
```

4.18.14 queue_motion_clear_Data - 清空本地队列数据

方法：queue_motion_clear_Data(socketFd);

功能描述：清空缓存的运动队列数据。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

socket_fd = tl.connect_robot("192.168.1.13", "6001")

ret = tl.queue_motion_clear_Data(socket_fd)
```

4.19 拖拽示教与轨迹记录回放

头文件：include/tl_interface.h

4.19.1 set_darg_mode - 设置拖拽示教的拖拽方式

方法：set_darg_mode(SOCKETFD socketFd, int mode);

功能描述：配置机械臂拖拽示教执行模式，可选择力矩柔顺拖拽、位置锁定拖拽、3D鼠标操控等不同拖动方式，适配现场不同示教操作需求。

参数：

- socketFd：已连接的句柄。
- mode：拖拽模式 0-无 1-3D鼠标 2-力矩模式 3-位置。

返回值：

- 0：成功

- 非 0：失败

使用示例：

```
# mode: 0=无  1=力矩模式  2=位置

print_title("测试2: set_darg_mode(1) 力矩模式")

ret = set_darg_mode(sock, 1)

if ret == SUCCESS:

    print_ok("set_darg_mode(1) 力矩模式已开启")

    pass_count += 1

else:

    print_fail("set_darg_mode(1)", ret)

    fail_count += 1
```

4.19.2 set_teach_type - 设置示教模式类型

方法：set_teach_type(SOCKETFD socketFd, int type);

功能描述：设置机器人本地示教操作类型，可切换为点动单轴微调模式或力矩拖拽直接拖动模式，满足点位精细校准与快速示教两种作业需求。

参数：

- socketFd：已连接的句柄。
- type ： 0 点动 1 拖拽。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
print_title("测试2b: set_teach_type(1) 切换到拖拽")

ret = set_teach_type(sock, 1)

if ret == SUCCESS:

    print_ok("已切换到拖拽模式")
```

```
else:
    print_fail("set_teach_type(1)", ret)
```

4.19.3 get_teach_type -- 查询示教模式类型

方法：get_teach_type(SOCKETFD socketFd, int& type);

功能描述：查询示教模式下是何种操作类型。

参数：

- socketFd：已连接的句柄。
- type： 0 点动 1 拖拽。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# 确认
teach_type = -1
ret, teach_type = get_teach_type(sock, teach_type)
if ret == SUCCESS:
    name = {0: "点动", 1: "拖拽"}.get(teach_type, "未知")
    print_ok("确认: teach_type=%d (%s)" % (teach_type, name))
```

4.19.4 set_current_teach_sensitivity - 设置电流环拖动示教灵敏度

方法：set_current_teach_sensitivity(SOCKETFD socketFd, const std::vector\<double>& friction_offsets);

功能描述：用于配置拖动示教模式下各关节电流环力矩触发阈值，调节手动拖拽示教时关节柔顺度与外力感知灵敏度。

参数：

- socketFd：已连接的句柄。

- friction_offsets：包含6个关节电流环拖动示教灵敏度参数的向量，顺序为J1-J6，范围应在0-3之间。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

friction_offsets = tl.VectorDouble()

friction_offsets.append(1)   # J1

friction_offsets.append(1)   # J2

friction_offsets.append(1)   # J3

friction_offsets.append(1)   # J4

friction_offsets.append(1)   # J5

friction_offsets.append(1)   # J6

ret = tl.set_current_teach_sensitivity(socket_fd, friction_offsets)

if ret == 0:

    print("[INFO] 设置电流环拖动示教灵敏度成功")

else:

    print(f"[ERROR] 设置电流环拖动示教灵敏度失败，错误码: {ret}")
```

4.19.5 get_current_teach_sensitivity - 获取电流环拖动示教灵敏度

方法：get_current_teach_sensitivity(SOCKETFD socketFd,std::vector\<double>& friction_offsets);

功能描述：读取拖动示教模式下各轴电流环力矩检测阈值，获取当前手动拖拽柔顺度与外力触发灵敏度配置参数。

参数：

- socketFd：已连接的句柄。
- friction_offsets：输出参数，用于接收6个关节电流环拖动示教灵敏度参数的向量，顺序为J1-J6。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
import tl_interface as tl

# 建立连接

socket_fd = tl.connect_robot("192.168.1.13", "6001")

friction_offsets = tl.VectorDouble()

ret = tl.get_current_teach_sensitivity(socket_fd, friction_offsets)

if ret == 0:

    print("[INFO] 各关节电流环拖动示教灵敏度: ", end="")

    for i in range(len(friction_offsets)):

        print("J" + str(i + 1) + "=" + str(friction_offsets[i]) + " ", end="")

    print("")

else:

    print(f"[ERROR] 获取电流环拖动示教灵敏度失败，错误码: {ret}")
```

4.19.6 get_drag_thread_is_end -- 查询拖拽是否结束

方法：get_drag_thread_is_end(SOCKETFD socketFd, bool& endFlag);

功能描述：查询拖拽是否已经结束。

参数：

- socketFd：已连接的句柄。
- endFlag：拖拽结束标志位，True 结束 False 未结束。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# 查询拖拽线程是否结束（绑定返回 [ret, endFlag] 列表）

ret, drag_finished = get_drag_thread_is_end(sock, False)

if ret != SUCCESS:

    print("[ERROR] 查询拖拽结束状态失败! ret = " + str(ret))

    break

if drag_finished:

    print("\\n[INFO] 检测到拖拽已结束")

    break
```

4.19.7 track_record_start - 轨迹记录开始

方法：track_record_start(SOCKETFD socketFd, double maxSamplingNum, double samplingInterval);

功能描述：启动机器人运动轨迹采集任务，配置最大采样总点数与点位采样时间间隔，用于记录拖拽示教过程中的连续运动路径。

参数：

- socketFd：已连接的句柄。
- maxSamplingNum：最大采样点数 [200,12000]。
- samplingInterval：采样间隔 [0.03,1]。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
MAX_SAMPLES = 500
```

```
SAMPLE_INTERVAL = 1

ret = track_record_start(sock, MAX_SAMPLES, SAMPLE_INTERVAL)

if ret != SUCCESS:

    print_fail("track_record_start", ret)
```

4.19.8 get_track_record_status - 轨迹记录开启状态查询

方法：get_track_record_status(SOCKETFD socketFd, bool& recordStart);

功能描述：查询轨迹录制功能当前运行状态，通过出参返回轨迹记录是否处于开启采集状态。

参数：

- socketFd：已连接的句柄。
- recordStart：记录状态 true 开启 false 关闭。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
# 查询轨迹记录状态

print_title("测试1: 查询初始记录状态")

recordStart = False

ret, recordStart = get_track_record_status(sock, recordStart)

if ret == SUCCESS:

    print_ok("初始状态: %s (ret=%d)" % ("记录中" if recordStart else "未记录", ret))

else:

    print_fail("get_track_record_status", ret)
```

4.19.9 track_record_stop - 轨迹记录关闭

方法：track_record_stop(SOCKETFD socketFd);

功能描述：终止机器人连续运动轨迹采样录制。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
ret = track_record_stop(sock)

if ret == SUCCESS:

    print_ok("track_record_stop")

else:

    print_fail("track_record_stop", ret)
```

4.19.10 track_record_save - 保存拖动示教轨迹

方法：track_record_save(SOCKETFD socketFd, std::string trajName);

功能描述：将已采集完成的拖拽示教轨迹以指定名称存储至控制器，实现轨迹文件持久化留存以便后续调用回放。

参数：

- socketFd：已连接的句柄。
- trajName：保存名字。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
print_title("测试5: 保存轨迹")

# 按示教器格式生成轨迹名

TRAJ_NAME = time.strftime("轨迹名-%m月%d日%H时%M分%S", time.localtime())
```

```
ret = track_record_save(sock, TRAJ_NAME)

if ret == SUCCESS:

    print_ok("轨迹已保存为: %s" % TRAJ_NAME)

else:

    print_fail("track_record_save('%s')" % TRAJ_NAME, ret)
```

4.19.11 track_record_playback - 轨迹回放

方法：track_record_playback(SOCKETFD socketFd, int vel);

功能描述：调用已录制完成的轨迹文件进行复现运行，可通过传入速度参数设定轨迹回放的整体运行速率。

参数：

- socketFd：已连接的句柄。
- vel：速度。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
print("\\n  [用例A] 低速回放 (vel=20)")

ret = track_record_playback(sock, 20)

if ret == SUCCESS:

    print_ok("低速回放已开始，等待完成...")

    if wait_playback_done(sock, timeout=120):

        print_ok("低速回放完成")

    else:

        print("  [WARN] 低速回放等待超时")
```

4.19.12 track_record_delete - 轨迹清除

方法：track_record_delete(SOCKETFD socketFd);

功能描述：清空控制器内缓存的当前录制轨迹数据，释放轨迹占用存储空间。

参数：

- socketFd：已连接的句柄。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
print_title("测试8: 清除轨迹数据")

ret = track_record_delete(sock)

if ret == SUCCESS:

    print_ok("track_record_delete")

else:

    print_fail("track_record_ delete", ret)
```

4.19.13 drag_trajectory_export - 导出拖动轨迹

方法：drag_trajectory_export(SOCKETFD socketFd, str directoryPath);

功能描述：将控制器 `dragPos` 目录中的全部拖动轨迹（`.json` 文件）导出到本地目录，用于备份或迁移。

参数：

- socketFd：已连接的句柄。
- directoryPath：本地保存目录；末级目录不存在时自动创建，父目录须存在。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
print_title("导出拖动轨迹")

ret = drag_trajectory_export(sock, "/home/user/drag_traj")

if ret == SUCCESS:

    print_ok("drag_trajectory_export")

else:

    print_fail("drag_trajectory_export", ret)
```

4.19.14 drag_trajectory_import - 导入拖动轨迹

方法：drag_trajectory_import(SOCKETFD socketFd, str directoryPath);

功能描述：将本地目录中的全部 `.json` 拖动轨迹导入控制器 `dragPos` 目录。

参数：

- socketFd：已连接的句柄。
- directoryPath：包含 `.json` 轨迹文件的本地目录。

返回值：

- 0：成功
- 非 0：失败

注意：控制器中存在同名轨迹时将被覆盖。

使用示例：

```
print_title("导入拖动轨迹")

ret = drag_trajectory_import(sock, "/home/user/drag_traj")

if ret == SUCCESS:

    print_ok("drag_trajectory_import")

else:

    print_fail("drag_trajectory_import", ret)
```

4.19.15 drag_trajectory_playback - 回放指定拖动轨迹

方法：drag_trajectory_playback(SOCKETFD socketFd, str trajectoryName, int velocity);

功能描述：回放控制器 `dragPos` 目录中指定的拖动轨迹文件。回放前建议先退出拖拽模式。

参数：

- socketFd：已连接的句柄。
- trajectoryName：控制器 `dragPos` 目录中的轨迹文件名，包含 `.json` 后缀。
- velocity：回放速度，范围 [1,100]。

返回值：

- 0：成功
- 非 0：失败

使用示例：

```
print_title("回放拖动轨迹")

ret = drag_trajectory_playback(sock, "traj_001.json", 50)

if ret == SUCCESS:

    print_ok("drag_trajectory_playback")

else:

    print_fail("drag_trajectory_playback", ret)
```

4.19.16 drag_trajectory_delete - 删除指定拖动轨迹

方法：drag_trajectory_delete(SOCKETFD socketFd, str trajectoryName);

功能描述：删除控制器 `dragPos` 目录中指定的拖动轨迹文件。

参数：

- socketFd：已连接的句柄。
- trajectoryName：控制器 `dragPos` 目录中的轨迹文件名，包含 `.json` 后缀。

返回值：

- 0：成功
- 非 0：失败

注意：删除后不可恢复。

使用示例：

```
print_title("删除拖动轨迹")

ret = drag_trajectory_delete(sock, "traj_001.json")

if ret == SUCCESS:

    print_ok("drag_trajectory_delete")

else:

    print_fail("drag_trajectory_delete", ret)
```

五．7000端口接口

5.1 以下功能在连接7000端口后使用

头文件：include/tl_interface.h

相关示例代码：examples/py/ ex_servoj.py

5.1.1 open_servoJ - 打开关节跟踪模式

方法：open_servoJ(SOCKETFD socketFd, std::vector\<double> vmax, std::vector\<double> amax, std::vector\<double> jmax);

功能描述：激活高速流式伺服跟踪模式（ServoJ）。需提供各轴的最大速度、加速度、加加速度限制。

参数：

- socketFd：已连接的机械臂高速实时跟踪控制通道（通常固定为 7000端口）句柄，整型格式（int）。
- vmax：关节最大速度约束（固定 7 元素，°/s）。
- amax：关节最大加速度约束（固定 7 元素，$°/s^2$）。
- jmax：关节最大加加速度约束（固定 7 元素，$°/s^3$）。

返回值：

- **0**：激活成功。代表控制器成功进入高速流式透传就绪态。
- - **非 0 整型值**：激活失败。代表系统当前有未清除的报警，或者当前运行模式不允许进入高频控制。

- - **-2 / -1**：断连异常或未捕获的运行时异常。

使用示例

```
import tl_interface as tl

# 7000 端口建立连接（servoJ 走 7000）

sock_servo = tl.connect_robot("192.168.1.13", "7000")

# 速度/加速度/加加速度约束（7 元素，单位 °/s, °/s², °/s³）

vmax = tl.VectorDouble([30.0] * 7)

amax = tl.VectorDouble([60.0] * 7)

jmax = tl.VectorDouble([100.0] * 7)

ret = tl.open_servoJ(sock_servo, vmax, amax, jmax)

if ret == 0:

    print("[信息] open_servoJ 成功")

else:

    print(f"[ERROR] open_servoJ 失败，错误码：{ret}")
```

说明：

启动关节位置实时跟踪模式，必须先调用该接口，才能使用 set_servoJ_pos发送实时关节位置。

5.1.2 close_servoJ - 关闭关节跟踪模式

方法：close_servoJ(SOCKETFD socketFd);

功能描述：退出伺服跟踪模式，恢复常规轨迹规划。

参数：

- socketFd：已连接的机械臂高速实时跟踪控制通道（通常固定为 7000端口）句柄，整型格式（int）。

返回值：

- 0：成功
- 非 0：失败

使用示例

```
# ---- 关闭 servoJ 模式 ----

ret = close_servoJ(sock_servo)

print_result("close_servoJ", ret)
```

说明:

关闭关节实时跟踪模式，退出实时控制状态。

5.1.3 set_servoJ_pos - 设置实时跟踪关节位置

方法：set_servoJ_pos(SOCKETFD socketFd, std::vector\<double>q);

功能描述：在跟踪模式下实时发送目标关节角度。需以固定周期（如10ms，最快4ms）持续调用。

参数:

- socketFd：已连接的机械臂高速实时跟踪控制通道（通常固定为 7000端口）句柄，整型格式（int）。
- q：目标关节角度（单位：°），固定 7 元素（6 轴机器人第 7 元素（外部轴）填 0，7 轴机器人第 7 元素为真实 J7 关节角）。

返回值:

- 0：成功
- 非 0：失败

说明:

在**已打开跟踪模式**下，实时发送关节角度，机械臂会立即跟踪到目标位置。

```
# /

#  ex_servoj.py

#  \@brief 关节实时跟踪（servoJ）示例程序

#  \@attention

#  - 确保机械臂供电正常、网络通信正常

#  - 需要双端口连接：6001（控制）+ 7000（伺服高频透传）

#  - servoJ 周期约 10ms，回调内不要做耗时操作

#  - 默认控制器 IP 为 192.168.1.13，可根据实际环境修改
```

```
#  - 运行模式:透传模式（实时的将关节数据下发给控制器，控制器直接响应。连接7000端口后，并调用
open_servoJ即可进入透传模式。在此之前建议set_current_mode(sock,

# 2)切换到运行模式）

#  \*   - 开始前需: 双端口连接(6001+7000) → 上电 → open_servoJ

#  \*   - 退出前需: close_servoJ → set_current_mode(0) → disconnect_robot(双端口)

#  \* \@note 运行步骤

#  \*       运行: python3 ex_servoj.py

#  \*/

import sys

import os

import platform

# 基于脚本所在目录构建 SDK Python 绑定的绝对路径

_script_dir = os.path.dirname(os.path.abspath(__file__))

# 自动检测平台，添加对应 SDK Python 绑定路径

if platform.system() == \"Windows\":

    sys.path.append(os.path.join(_script_dir, \"../../lib/windows/python\"))

elif platform.machine() in (\"aarch64\", \"arm64\"):

    sys.path.append(os.path.join(_script_dir, \"../../lib/linux/arm64/python\"))

else:

    sys.path.append(os.path.join(_script_dir, \"../../lib/linux/x86/python\"))

# Windows: 将 tl_host.dll 所在目录加入 DLL 搜索路径

if platform.system() == \"Windows\":

    _dll_dir = os.path.join(_script_dir, \"../../lib/windows\")

    os.add_dll_directory(_dll_dir)

from tl_interface import *

import time

# ============================================================
```

```
# 工具函数

# ============================================================

def print_result(msg, ret):

    \"\"\"输出操作结果

    @param msg 操作描述

    @param ret 返回码（Result枚举值）

    \"\"\"

    if ret == SUCCESS:

        print(\"  \[成功\] \" + msg)

    else:

        print(\"  \[失败\] \" + msg + \", 错误码: \" + str(ret))

def print_separator(title):

    \"\"\"输出分隔标题\"\"\"

    print(\"\\n========== \" + title + \" ==========\")

# ============================================================

# 主函数

# ============================================================

def main():

    # \-\-\-- 获取 SDK 版本 \-\-\--

    version = get_library_version()

    print(\"\[信息\] SDK 版本: \" + version)

    # \-\-\-- 双端口连接 \-\-\--

    print_separator(\"双端口连接\")

    sock = connect_robot(\"192.168.1.13\", \"6001\")

    if sock <= 0:

        print(\"\[ERROR\] 连接 6001 端口失败\")
```

```
        return -1

    sock_servo = connect_robot(\"192.168.1.13\", \"7000\")

    if sock_servo <= 0:

        print(\"\[ERROR\] 连接 7000 端口失败\")

        disconnect_robot(sock)

        return -1

    print(\"\[信息\] 双端口连接成功 (6001=\" + str(sock) + \", 7000=\" + str(sock_servo) +
\")\")

    # \-\-\-- 清错 \-\-\--

    clear_error(sock)

    time.sleep(0.3)

    # \-\-\-- 下电复位 \-\-\--

    set_servo_poweroff(sock)

    time.sleep(0.3)

    # \-\-\-- 切换示教模式并设置伺服就绪 \-\-\--

    print_separator(\"切换示教模式\")

    ret = set_current_mode(sock, 0)

    if ret != SUCCESS:

        print(f\"\[ERROR\] 切换示教模式失败: {ret}\")

        disconnect_robot(sock)

        disconnect_robot(sock_servo)

        return -1

    print(\"  \[信息\] 已切换为示教模式\")

    time.sleep(0.3)

    print(\"  \[信息\] 设置伺服就绪\...\")

    ret = set_servo_state(sock, 1)

    if ret != SUCCESS and ret != OPERATION_NOT_ALLOWED:
```

```
        print(f\"\[ERROR\] 设置伺服就绪失败: {ret}\")

        disconnect_robot(sock)

        disconnect_robot(sock_servo)

        return -1

    print_result(\"set_servo_state(1)\", ret)

    time.sleep(0.5)

    # \-\-\-- 切换为运行模式 \-\-\--

    print_separator(\"切换运行模式\")

    ret = set_current_mode(sock, 2)

    print_result(\"set_current_mode(2)\", ret)

    time.sleep(2)

    # ============================================================

    # 打开 servoJ 模式

    # ============================================================

    print_separator(\"servoJ 关节实时跟踪\")

    # 速度/加速度/加加速度约束（7 元素，单位 °/s, °/s², °/s³）

    # 注意: SDK 底层协议要求 servoJ 相关向量固定为 7 元素，传 6 元素会导致控制器协议解析错乱并断开
7000 连接

    vmax = [30.0, 30.0, 30.0, 30.0, 30.0, 30.0, 30.0]

    amax = [60.0, 60.0, 60.0, 60.0, 60.0, 60.0, 60.0]

    jmax = [100.0, 100.0, 100.0, 100.0, 100.0, 100.0, 100.0]

    ret = open_servoJ(sock_servo, vmax, amax, jmax)

    print_result(\"open_servoJ\", ret)

    if ret != SUCCESS:

        set_servo_poweroff(sock)

        disconnect_robot(sock)

        disconnect_robot(sock_servo)
```

```
        return -1

    # \-\-\-- 动作 1: J1 从 0° 平滑转到 30° \-\-\--

    print(\"\\n\-\-- 动作 1: J1 转到 30° \-\--\")

    # q 必须为 7 元素，与 open_servoJ 一致

    q = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

    for i in range(200):

        q[0] += (30.0 / 200.0)       # 每次增加 0.15°

        set_servoJ_pos(sock_servo, q)

        time.sleep(0.01)              # 100Hz

    print(\"  \[信息\] J1=30° 到位\")

    time.sleep(1.0)

    # \-\-\-- 动作 2: J1 回到 0°，同时 J2 转到 20° \-\-\--

    print(\"\\n\-\-- 动作 2: J1 回零, J2 转到 20° \-\--\")

    # q 必须为 7 元素

    q = [30.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

    for i in range(200):

        q[0] -= (30.0 / 200.0)       # J1 逐步回到 0°

        q[1] += (20.0 / 200.0)       # J2 逐步增加到 20°

        set_servoJ_pos(sock_servo, q)

        time.sleep(0.01)

    print(\"  \[信息\] J1=0°, J2=20° 到位\")

    time.sleep(1.0)

    # \-\-\-- 动作 3: J1=0°, J2 回到 0°, J3 转到 15° \-\-\--

    print(\"\\n\-\-- 动作 3: J2 回零, J3 转到 15° \-\--\")

    # q 必须为 7 元素

    q = [0.0, 20.0, 0.0, 0.0, 0.0, 0.0, 0.0]

    for i in range(200):
```

```
    q[1] -= (20.0 / 200.0)       # J2 逐步回到 0°
    q[2] += (15.0 / 200.0)       # J3 逐步增加到 15°
    set_servoJ_pos(sock_servo, q)
    time.sleep(0.01)
print(\"  \[信息\] J2=0°, J3=15° 到位\")
time.sleep(1.0)
# \-\-\-- 动作 4: 所有轴回到零位 \-\-\--
print(\"\\n\-\-- 动作 4: 所有轴回零 \-\--\")
# q 必须为 7 元素
q = [0.0, 0.0, 15.0, 0.0, 0.0, 0.0, 0.0]
for i in range(150):
    q[2] -= (15.0 / 150.0)       # J3 逐步回到 0°
    set_servoJ_pos(sock_servo, q)
    time.sleep(0.01)
print(\"  \[信息\] 所有轴回零到位\")
time.sleep(1.0)
# \-\-\-- 关闭 servoJ 模式 \-\-\--
ret = close_servoJ(sock_servo)
print_result(\"close_servoJ\", ret)
print(\"\\n  \[信息\] servoJ 运动全部完成\")
# ============================================================
# 切回示教模式（自动下电）& 断开
# ============================================================
print_separator(\"切回示教模式\")
ret = set_current_mode(sock, 0)
print_result(\"set_current_mode(0) 切回示教模式（自动下电）\", ret)
```

```
    time.sleep(0.3)

    disconnect_robot(sock)

    disconnect_robot(sock_servo)

    print(\"\[信息\] 已断开连接\")

    print(\"\\n\[信息\] servoJ 示例程序运行完毕\")

    return 0

if __name__ == \"\_\_main\_\_\":

    main()
```

5.1.4 open_servo_csv_mode - 打开周期同步速度模式

方法：open_servo_csv_mode(SOCKETFD socketFd);

功能描述：打开周期同步速度模式（CSV），之后可按固定周期调用 set_servo_csv_velocity 下发关节目标速度。

参数：

- socketFd：已连接的机械臂高速实时跟踪控制通道（通常固定为 7000端口）句柄，整型格式（int）。

返回值：

- 0：成功
- 非 0：失败

注意：需控制器 24.03 及以上版本；需先连接 7000 端口，并与 close_servo_csv_mode 成对调用。

使用示例

```
ret = open_servo_csv_mode(sock_servo)

print_result("open_servo_csv_mode", ret)
```

5.1.5 close_servo_csv_mode - 关闭周期同步速度模式

方法：close_servo_csv_mode(SOCKETFD socketFd);

功能描述：关闭周期同步速度模式，停止速度下发。

参数：

- socketFd：已连接的机械臂高速实时跟踪控制通道（通常固定为 7000端口）句柄，整型格式（int）。

返回值：

- 0：成功
- 非 0：失败

使用示例

```
ret = close_servo_csv_mode(sock_servo)

print_result("close_servo_csv_mode", ret)
```

5.1.6 set_servo_csv_velocity - 下发周期同步速度

方法：set_servo_csv_velocity(SOCKETFD socketFd, std::vector\<double> targetVelocity);

功能描述：在周期同步速度模式下，按固定周期下发各关节目标速度。

参数：

- socketFd：已连接的机械臂高速实时跟踪控制通道（通常固定为 7000端口）句柄，整型格式（int）。
- targetVelocity：各关节目标速度（°/s），固定 7 元素；6 轴机器人第 7 元素（索引 6）补 0（外部轴），7 轴机器人第 7 元素为真实 J7 速度。

返回值：

- 0：成功
- 非 0：失败（向量长度 ≠ 7 时返回参数错误）

使用示例

```
# 以 10°/s 正向旋转 J1，其余轴保持 0

velocity = VectorDouble([10.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0])

ret = set_servo_csv_velocity(sock_servo, velocity)

print_result("set_servo_csv_velocity", ret)
```

5.1.7 open_servo_cst_mode - 打开周期同步扭矩模式

方法：open_servo_cst_mode(SOCKETFD socketFd);

功能描述：打开周期同步扭矩模式（CST），之后可按固定周期调用 set_servo_cst_torque 下发关节目标扭矩。

参数：

- socketFd：已连接的机械臂高速实时跟踪控制通道（通常固定为 7000端口）句柄，整型格式（int）。

返回值：

- 0：成功
- 非 0：失败

注意：需控制器 24.03 及以上版本；需先连接 7000 端口，并与 close_servo_cst_mode 成对调用。

使用示例

```
ret = open_servo_cst_mode(sock_servo)

print_result("open_servo_cst_mode", ret)
```

5.1.8 close_servo_cst_mode - 关闭周期同步扭矩模式

方法：close_servo_cst_mode(SOCKETFD socketFd);

功能描述：关闭周期同步扭矩模式，停止扭矩下发。

参数：

- socketFd：已连接的机械臂高速实时跟踪控制通道（通常固定为 7000端口）句柄，整型格式（int）。

返回值：

- 0：成功
- 非 0：失败

使用示例

```
ret = close_servo_cst_mode(sock_servo)

print_result("close_servo_cst_mode", ret)
```

5.1.9 set_servo_cst_torque - 下发周期同步扭矩

方法：set_servo_cst_torque(SOCKETFD socketFd, std::vector\<double> targetTorque);

功能描述：在周期同步扭矩模式下，按固定周期下发各关节目标扭矩。

参数：

- socketFd：已连接的机械臂高速实时跟踪控制通道（通常固定为 7000端口）句柄，整型格式（int）。
- targetTorque：各关节目标扭矩，固定 7 元素；单位为额定扭矩的 0.1%，范围 [-1000,1000]；6 轴机器人第 7 元素（索引 6）补 0（外部轴），7 轴机器人第 7 元素为真实 J7 扭矩。

返回值：

- 0：成功
- 非 0：失败（向量长度 ≠ 7 时返回参数错误）

使用示例

```
# 给 J1 施加 5% 额定扭矩（50 × 0.1%）

torque = VectorDouble([50.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0])

ret = set_servo_cst_torque(sock_servo, torque)

print_result("set_servo_cst_torque", ret)
```

六． 使用示例

在下面sdk结构下.test.py里面直接粘贴即可运行，python测试版本为3.10.12

SDK结构如下：

├── include/

│ └── include2/

│ ├── c/

│ │ ├── interface/ # C语言接口头文件

│ │ └── parameter/ # C语言参数定义头文件

│ └── cpp/

```
│  ├──── interface/ # C++语言接口头文件
│  └──── parameter/ # C++语言参数定义头文件
├──── _tl_host.so # 核心底层库（C/C++编译生成）
├──── tl_interface.py # Python接口封装文件（所有接口入口）
└──── test.py # 测试脚本
```

6.1Modbus 主站通信综合示例

```
# /**
#  * ex_modbus.py
#  * @brief Modbus 主站通信综合示例（TCP + RTU 双模式）
#  * @attention
#  *   - 确保控制器网络通信正常
#  *   - TCP 模式需在 PC 上启动虚拟 Modbus 从站模拟器（端口 5000）：
#  *     python3 -c "from pymodbus.server import StartTcpServer
#  *     from pymodbus.datastore import ModbusSlaveContext, ModbusSequentialDataBlock
#  *     store = ModbusSlaveContext(di=ModbusSequentialDataBlock(0, [0]*100),
#  *         co=ModbusSequentialDataBlock(0, [0]*100),
#  *         hr=ModbusSequentialDataBlock(0, [0]*100),
#  *         ir=ModbusSequentialDataBlock(0, [0]*100))
#  *     StartTcpServer(context=store, address=('0.0.0.0', 5000))"
#  *   - RTU 模式用于灵巧手（通过 RS485 串口连接控制器）
#  *       从站 ID = 2，波特率 115200，串口号根据实际接线修改
#  *   - 灵巧手寄存器地址（4X 保持寄存器）：
#  *       力阈值: 1115\~1119（5 指） 速度: 1125\~1130（6 关节）
#  *       目标位置: 1135\~1140（6 关节） 实际位置反馈: 1155\~1160
#  * @note 运行步骤
```

```
#  *       1. TCP 模式需先启动虚拟从站模拟器
#  *       2. 运行: python3 ex_modbus.py
#  */
import sys
import os
import platform
import time
# 基于脚本所在目录构建 SDK Python 绑定的绝对路径
_script_dir = os.path.dirname(os.path.abspath(__file__))
# 自动检测平台，添加对应 SDK Python 绑定路径
if platform.system() == "Windows":
    sys.path.append(os.path.join(_script_dir, "../../lib/windows/python"))
elif platform.machine() in ("aarch64", "arm64"):
    sys.path.append(os.path.join(_script_dir, "../../lib/linux/arm64/python"))
else:
    sys.path.append(os.path.join(_script_dir, "../../lib/linux/x86/python"))
# Windows: 将 tl_host.dll 所在目录加入 DLL 搜索路径
if platform.system() == "Windows":
    _dll_dir = os.path.join(_script_dir, "../../lib/windows")
    os.add_dll_directory(_dll_dir)
from tl_interface import *
# 灵巧手寄存器地址（Modbus 4X 保持寄存器）
REG_FORCE_LIMIT     = 1115   # 手指力阈值（5 个）
REG_SPEED           = 1125   # 手指速度（6 个）
REG_POS_TARGET      = 1135   # 目标位置（6 个）
REG_POS_FEEDBACK    = 1155   # 实际位置反馈（6 个）
# ============================================================
```

```
# 工具函数
# ============================================================
def print_result(msg, ret):
    """@brief 打印操作结果"""
    if ret == SUCCESS:
        print("  [成功] %s" % msg)
    else:
        print("  [失败] %s，错误码: %d" % (msg, ret))
def print_separator(title):
    """@brief 打印分隔标题"""
    print("\\n========== %s ==========" % title)
def read_with_retry(sock, id, addr, qty, desc):
    """@brief 带重试的读取：最多尝试 3 次，每次间隔 500ms"""
    for attempt in range(1, 4):
        data = VectorInt()
        ret = modbus_read_holding_registers(sock, id, addr, qty, data)
        if ret == SUCCESS:
            print("  [成功] %s（第%d次）" % (desc, attempt))
            return ret, data
        print("  [重试] %s 第%d次失败，500ms后重试..." % (desc, attempt))
        time.sleep(0.5)
    print("  [失败] %s，错误码: %d（重试3次均失败）" % (desc, ret))
    return ret, VectorInt()
# ============================================================
# 主函数
# ============================================================
```

```
def main():
    # ---- 获取 SDK 版本 ----
    version = get_library_version()
    print("[信息] SDK 版本: %s" % version)
    # ---- 连接控制器 ----
    print_separator("连接控制器")
    sock = connect_robot("192.168.1.13", "6001")
    if sock < 0:
        print("[ERROR] 连接控制器失败")
        return -1
    print("[信息] 连接成功 (socket = %d)" % sock)
    # ============================================================
    # 第一部分：Modbus TCP 模式（master ID = 0）
    #   覆盖功能码 01H\~06H + 10H
    # ============================================================
    print_separator("第一部分：Modbus TCP 模式")
    # 1.1 设置 TCP 主站参数
    param = ModbusMasterParameter()
    param.type = "TCP"
    param.startAddress = True               # 起始地址 0
    param.TCP.IP = "192.168.1.100"          # 从站 IP（PC 端 Modbus 模拟器）
    param.TCP.port = 5000                   # 从站端口
    ret = modbus_set_master_parameter(sock, 0, param)
    print_result("modbus_set_master_parameter (id=0, TCP)", ret)
    if ret != SUCCESS:
        disconnect_robot(sock)
        return -1
```

```
print("  类型: %s, IP: %s, 端口: %d" % (param.type, param.TCP.IP, param.TCP.port))

# 1.2 打开Modbus 主站

ret = modbus_open_master(sock, 0)

print_result("modbus_open_master (id=0)", ret)

if ret != SUCCESS:

    disconnect_robot(sock)

    return -1

time.sleep(0.5)

# 1.3 功能码 05H: 写单线圈

print_separator("TCP --- 05H 写单线圈")

ret = modbus_write_signal_coil_status(sock, 0, 0, 0xFF00)

print_result("modbus_write_signal_coil_status (addr=0, ON)", ret)

time.sleep(0.2)

# 1.4 功能码 01H: 读线圈状态（验证 05H 写入）

print_separator("TCP --- 01H 读线圈状态")

read_data = VectorInt()

ret = modbus_read_coil_status(sock, 0, 0, 1, read_data)

print_result("modbus_read_coil_status (addr=0, qty=1)", ret)

if ret == SUCCESS and len(read_data) > 0:

    print("  线圈[0] = %d" % read_data[0])

# 1.5 功能码 02H: 读输入状态

print_separator("TCP --- 02H 读输入状态")

read_data = VectorInt()

ret = modbus_read_input_status(sock, 0, 0, 4, read_data)

print_result("modbus_read_input_status (addr=0, qty=4)", ret)

if ret == SUCCESS:
```

```
        print("  输入状态: ", end="")
        for i in range(len(read_data)):
            print("%d" % read_data[i], end=" ")
        print()
    # 1.6 功能码 06H: 写单寄存器
    print_separator("TCP --- 06H 写单寄存器")
    ret = modbus_write_signal_holding_registers(sock, 0, 0, 5678)
    print_result("modbus_write_signal_holding_registers (addr=0, val=5678)", ret)
    # 1.7 功能码 03H: 读保持寄存器（验证 06H 写入）
    print_separator("TCP --- 03H 读保持寄存器")
    read_data = VectorInt()
    ret = modbus_read_holding_registers(sock, 0, 0, 3, read_data)
    print_result("modbus_read_holding_registers (addr=0, qty=3)", ret)
    if ret == SUCCESS:
        print("  保持寄存器[0\~2]: ", end="")
        for i in range(len(read_data)):
            print("%d" % read_data[i], end=" ")
        print()
        if len(read_data) > 0 and read_data[0] == 5678:
            print("  [验证] 06H 写入与 03H 读取一致")
    # 1.8 功能码 04H: 读输入寄存器
    print_separator("TCP --- 04H 读输入寄存器")
    read_data = VectorInt()
    ret = modbus_read_input_registers(sock, 0, 0, 3, read_data)
    print_result("modbus_read_input_registers (addr=0, qty=3)", ret)
    if ret == SUCCESS:
        print("  输入寄存器: ", end="")
```

```
    for i in range(len(read_data)):
        print("%d" % read_data[i], end=" ")
    print()
# 1.9 功能码 10H: 写多个保持寄存器
print_separator("TCP --- 10H 写多个保持寄存器")
write_data = [100, 200, 300]
ret = modbus_write_multiple_holding_registers(sock, 0, 10, write_data)
print_result("modbus_write_multiple_holding_registers (addr=10)", ret)
if ret == SUCCESS:
    print("  写入数据: %d, %d, %d" % (write_data[0], write_data[1], write_data[2]))
# 1.10 03H 回读验证批量写入
read_data = VectorInt()
ret = modbus_read_holding_registers(sock, 0, 10, 3, read_data)
print_result("modbus_read_holding_registers (addr=10, qty=3) 回读验证", ret)
if ret == SUCCESS and len(read_data) >= 3:
    match = (read_data[0] == 100 and read_data[1] == 200 and read_data[2] == 300)
    print("  批量读写一致性验证: %s" % ("通过" if match else "失败"))
# ============================================================
# 第二部分：Modbus RTU 模式（master ID = 1）
#   灵巧手控制示例
# ============================================================
print_separator("第二部分：Modbus RTU 模式（灵巧手控制）")
# 2.1 设置RTU 主站参数
param_rtu = ModbusMasterParameter()
param_rtu.type = "RTU"
param_rtu.startAddress = True
```

```
    param_rtu.RTU.slaveId = 2              # 灵巧手从站 ID

    param_rtu.RTU.port = 1                 # 控制器串口号（根据接线修改）

    param_rtu.RTU.baudrate = 115200        # 波特率

    param_rtu.RTU.checkBit = "None"

    param_rtu.RTU.dataBit = 8

    param_rtu.RTU.stopBit = 1

    ret = modbus_set_master_parameter(sock, 1, param_rtu)

    print_result("modbus_set_master_parameter (id=1, RTU slaveId=2)", ret)

    if ret != SUCCESS:

        disconnect_robot(sock)

        return -1

    print("  从站ID: %d（灵巧手）" % param_rtu.RTU.slaveId)

    # 2.2 打开Modbus 主站

    ret = modbus_open_master(sock, 1)

    print_result("modbus_open_master (id=1)", ret)

    if ret != SUCCESS:

        disconnect_robot(sock)

        return -1

    time.sleep(0.5)

    # 2.3 03H 读灵巧手寄存器（目标位置 + 实际位置）

    print_separator("RTU --- 03H 读保持寄存器")

    ret_tgt, tgt_data = read_with_retry(sock, 1, REG_POS_TARGET, 6, "读目标位置
1135\~1140")

    if ret_tgt == SUCCESS:

        print("  目标位置: ", end="")

        for i in range(len(tgt_data)):

            print("%d" % tgt_data[i], end=" ")
```

```
        print()

    ret_fb, fb_data = read_with_retry(sock, 1, REG_POS_FEEDBACK, 6, "读实际位置
1155\~1160")

    if ret_fb == SUCCESS:

        print("  实际位置: ", end="")

        for i in range(len(fb_data)):

            print("%d" % fb_data[i], end=" ")

        print()

    # 2.4 06H 设置手指力阈值

    print_separator("RTU --- 06H 写单寄存器（力阈值）")

    ret = modbus_write_signal_holding_registers(sock, 1, REG_FORCE_LIMIT, 500)

    print_result("写力阈值 (addr=%d, val=500)" % REG_FORCE_LIMIT, ret)

    time.sleep(0.2)

    # 2.5 06H 设置手指速度

    print_separator("RTU --- 06H 写单寄存器（速度）")

    ret = modbus_write_signal_holding_registers(sock, 1, REG_SPEED, 200)

    print_result("写速度 (addr=%d, val=200)" % REG_SPEED, ret)

    time.sleep(0.2)

    # 2.6 05H 写单线圈（测试）

    print_separator("RTU --- 05H 写单线圈")

    ret = modbus_write_signal_coil_status(sock, 1, 0, 0xFF00)

    print_result("modbus_write_signal_coil_status (addr=0, ON)", ret)

    time.sleep(0.2)

    # 2.7 01H 读线圈状态（验证）

    print_separator("RTU --- 01H 读线圈状态")

    coil_data = VectorInt()

    for at in range(3):
```

```
        ret = modbus_read_coil_status(sock, 1, 0, 1, coil_data)
        if ret == SUCCESS:
            break
        time.sleep(0.5)
    print_result("modbus_read_coil_status (addr=0, qty=1)", ret)
    if ret == SUCCESS and len(coil_data) > 0:
        print("  线圈[0] = %d" % coil_data[0])
    # 2.8 02H 读输入状态
    print_separator("RTU --- 02H 读输入状态")
    in_status = VectorInt()
    for at in range(3):
        ret = modbus_read_input_status(sock, 1, 0, 4, in_status)
        if ret == SUCCESS:
            break
        time.sleep(0.5)
    print_result("modbus_read_input_status (addr=0, qty=4)", ret)
    if ret == SUCCESS:
        print("  输入状态: ", end="")
        for i in range(len(in_status)):
            print("%d" % in_status[i], end=" ")
        print()
    # 2.9 04H 读输入寄存器
    print_separator("RTU --- 04H 读输入寄存器")
    in_reg = VectorInt()
    for at in range(3):
        ret = modbus_read_input_registers(sock, 1, 0, 3, in_reg)
        if ret == SUCCESS:
```

```
        break
    time.sleep(0.5)
print_result("modbus_read_input_registers (addr=0, qty=3)", ret)
if ret == SUCCESS:
    print("  输入寄存器: ", end="")
    for i in range(len(in_reg)):
        print("%d" % in_reg[i], end=" ")
    print()
# 2.10 06H 设置手指目标位置（张开：所有手指 0）
print_separator("RTU --- 06H 设置目标位置（张开）")
# 手势"张开"位置 [0, 0, 0, 0, 0, 728]
pos = [0, 0, 0, 0, 0, 728]
for i in range(6):
    ret = modbus_write_signal_holding_registers(sock, 1, REG_POS_TARGET + i, pos[i])
    if ret != SUCCESS:
        print("  [失败] 写位置[%d] 错误码: %d" % (i, ret))
    time.sleep(0.05)
print("  [成功] 手指已张开（位置全0，拇指728）")
# ============================================================
# 第三部分：六维力传感器通讯参数
# ============================================================
print_separator("第三部分：六维力传感器通讯参数")
force_param = SixDimensionalForceCommunicationParams()
force_param.sensorCommunicationType = 0       # 0: EtherCAT
force_param.startupAutoConnectSensor = True   # 启动自动连接传感器
force_param.sensorDragEnable = True           # 传感器拖拽使能
```

```
    force_param.etherCat_mapNum = 1

    force_param.YDirection = -1

    force_param.ZDirection = -1

    ret = set_six_dimensional_force_communication_params(sock, force_param)

    print_result("set_six_dimensional_force_communication_params", ret)

    if ret == SUCCESS:

        print("  传感器通信类型: EtherCAT")

        print("  拖拽使能: %s" % ("开启" if force_param.sensorDragEnable else "关闭"))

    # ============================================================

    # 断开连接

    # ============================================================

    print_separator("断开连接")

    disconnect_robot(sock)

    print("[信息] 已断开连接")

    print("\\n[信息] Modbus 综合示例程序运行完毕")

    return 0

if __name__ == "__main__":

    main()
```

6.2关节实时跟踪servoJ使用示例

```
# /**

#  * ex_servoj.py

#  * @brief 关节实时跟踪（servoJ）示例程序

#  * @attention

#  *   - 确保机械臂供电正常、网络通信正常

#  *   - 需要双端口连接: 6001（控制）+ 7000（伺服高频透传）
```

```
#  *    - servoJ 周期约 10ms，回调内不要做耗时操作

#  *    - 默认控制器 IP 为 192.168.1.13，可根据实际环境修改

#  *    - 运行模式：透传模式（实时的将关节数据下发给控制器，控制器直接响应。连接7000端口后，并调用open_servoJ即可进入透传模式。在此之前建议set_current_mode(sock, 2)切换到运行模式）

#  *    - 开始前需：双端口连接(6001+7000) → 上电 → open_servoJ

#  *    - 退出前需：close_servoJ → set_current_mode(0) → disconnect_robot(双端口)

#  * @note 运行步骤

#  *       运行：python3 ex_servoj.py

#  */

import sys

import os

import platform

# 基于脚本所在目录构建 SDK Python 绑定的绝对路径

_script_dir = os.path.dirname(os.path.abspath(__file__))

# 自动检测平台，添加对应 SDK Python 绑定路径

if platform.system() == "Windows":

    sys.path.append(os.path.join(_script_dir, "../../lib/windows/python"))

elif platform.machine() in ("aarch64", "arm64"):

    sys.path.append(os.path.join(_script_dir, "../../lib/linux/arm64/python"))

else:

    sys.path.append(os.path.join(_script_dir, "../../lib/linux/x86/python"))

# Windows: 将 tl_host.dll 所在目录加入 DLL 搜索路径

if platform.system() == "Windows":

    _dll_dir = os.path.join(_script_dir, "../../lib/windows")

    os.add_dll_directory(_dll_dir)

from tl_interface import *

import time
```

```python
# ============================================================

# 工具函数

# ============================================================

def print_result(msg, ret):

    """输出操作结果

    @param msg 操作描述

    @param ret 返回码（Result枚举值）

    """

    if ret == SUCCESS:

        print("  [成功] " + msg)

    else:

        print("  [失败] " + msg + ", 错误码: " + str(ret))

def print_separator(title):

    """输出分隔标题"""

    print("\\n========== " + title + " ==========")

# ============================================================

# 主函数

# ============================================================

def main():

    # ---- 获取 SDK 版本 ----

    version = get_library_version()

    print("[信息] SDK 版本: " + version)

    # ---- 双端口连接 ----

    print_separator("双端口连接")

    sock = connect_robot("192.168.1.13", "6001")

    if sock <= 0:

        print("[ERROR] 连接 6001 端口失败")
```

```
    return -1
sock_servo = connect_robot("192.168.1.13", "7000")
if sock_servo <= 0:
    print("[ERROR] 连接 7000 端口失败")
    disconnect_robot(sock)
    return -1
print("[信息] 双端口连接成功 (6001=" + str(sock) + ", 7000=" + str(sock_servo) + ")")
# ---- 清错 ----
clear_error(sock)
time.sleep(0.3)
# ---- 下电复位 ----
set_servo_poweroff(sock)
time.sleep(0.3)
# ---- 切换示教模式并设置伺服就绪 ----
print_separator("切换示教模式")
ret = set_current_mode(sock, 0)
if ret != SUCCESS:
    print(f"[ERROR] 切换示教模式失败: {ret}")
    disconnect_robot(sock)
    disconnect_robot(sock_servo)
    return -1
print("  [信息] 已切换为示教模式")
time.sleep(0.3)
print("  [信息] 设置伺服就绪...")
ret = set_servo_state(sock, 1)
if ret != SUCCESS and ret != OPERATION_NOT_ALLOWED:
```

```
        print(f"[ERROR] 设置伺服就绪失败: {ret}")
        disconnect_robot(sock)
        disconnect_robot(sock_servo)
        return -1
    print_result("set_servo_state(1)", ret)
    time.sleep(0.5)
    # ---- 切换为运行模式 ----
    print_separator("切换运行模式")
    ret = set_current_mode(sock, 2)
    print_result("set_current_mode(2)", ret)
    time.sleep(2)
    # ============================================================
    # 打开 servoJ 模式
    # ============================================================
    print_separator("servoJ 关节实时跟踪")
    # 速度/加速度/加加速度约束（7 元素，单位 °/s, °/s², °/s³）
    # 注意: SDK 底层协议要求 servoJ 相关向量固定为 7 元素，传 6 元素会导致
    #       控制器协议解析错乱并断开 7000 连接
    vmax = [30.0, 30.0, 30.0, 30.0, 30.0, 30.0, 30.0]
    amax = [60.0, 60.0, 60.0, 60.0, 60.0, 60.0, 60.0]
    jmax = [100.0, 100.0, 100.0, 100.0, 100.0, 100.0, 100.0]
    ret = open_servoJ(sock_servo, vmax, amax, jmax)
    print_result("open_servoJ", ret)
    if ret != SUCCESS:
        set_servo_poweroff(sock)
        disconnect_robot(sock)
        disconnect_robot(sock_servo)
```

```
    return -1
# ---- 动作 1: J1 从 0° 平滑转到 30° ----
print("\\n--- 动作 1: J1 转到 30° ---")
# q 必须为 7 元素，与 open_servoJ 一致
q = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
for i in range(200):
    q[0] += (30.0 / 200.0)       # 每次增加 0.15°
    set_servoJ_pos(sock_servo, q)
    time.sleep(0.01)              # 100Hz
print("  [信息] J1=30° 到位")
time.sleep(1.0)
# ---- 动作 2: J1 回到 0°，同时 J2 转到 20° ----
print("\\n--- 动作 2: J1 回零, J2 转到 20° ---")
# q 必须为 7 元素
q = [30.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
for i in range(200):
    q[0] -= (30.0 / 200.0)       # J1 逐步回到 0°
    q[1] += (20.0 / 200.0)       # J2 逐步增加到 20°
    set_servoJ_pos(sock_servo, q)
    time.sleep(0.01)
print("  [信息] J1=0°, J2=20° 到位")
time.sleep(1.0)
# ---- 动作 3: J1=0°, J2 回到 0°, J3 转到 15° ----
print("\\n--- 动作 3: J2 回零, J3 转到 15° ---")
# q 必须为 7 元素
q = [0.0, 20.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

```
    for i in range(200):
        q[1] -= (20.0 / 200.0)       # J2 逐步回到 0°
        q[2] += (15.0 / 200.0)       # J3 逐步增加到 15°
        set_servoJ_pos(sock_servo, q)
        time.sleep(0.01)
    print("  [信息] J2=0°, J3=15° 到位")
    time.sleep(1.0)
    # ---- 动作 4: 所有轴回到零位 ----
    print("\\n--- 动作 4: 所有轴回零 ---")
    # q 必须为 7 元素
    q = [0.0, 0.0, 15.0, 0.0, 0.0, 0.0, 0.0]
    for i in range(150):
        q[2] -= (15.0 / 150.0)       # J3 逐步回到 0°
        set_servoJ_pos(sock_servo, q)
        time.sleep(0.01)
    print("  [信息] 所有轴回零到位")
    time.sleep(1.0)
    # ---- 关闭 servoJ 模式 ----
    ret = close_servoJ(sock_servo)
    print_result("close_servoJ", ret)
    print("\\n  [信息] servoJ 运动全部完成")
    # ============================================================
    # 切回示教模式（自动下电）& 断开
    # ============================================================
    print_separator("切回示教模式")
    ret = set_current_mode(sock, 0)
    print_result("set_current_mode(0) 切回示教模式（自动下电）", ret)
```

```
    time.sleep(0.3)

    disconnect_robot(sock)

    disconnect_robot(sock_servo)

    print("[信息] 已断开连接")

    print("\\n[信息] servoJ 示例程序运行完毕")

    return 0

if __name__ == "__main__":

    main()
```

6.3关节运动到目标点robot_movej直线运动到目标位置robot_movel综合使用示例

```
# /**

#  * ex_move_control.py

#  * @brief 运动控制相关接口（包括MoveJ、MoveL）

#  * @attention

#  *   - 运行模式：示教模式

#  *   - 开始前需: connect_robot → set_servo_state(1) → set_servo_poweron

#  *   - 退出前需: set_servo_poweroff → disconnect_robot

#  *   - 确保机械臂供电正常、网络通信正常

#  * @note 运行步骤

#  *       运行: python3 ex_move_control.py

#  */

import sys

import os

import platform

# 基于脚本所在目录构建 SDK Python 绑定的绝对路径
```

```python
_script_dir = os.path.dirname(os.path.abspath(__file__))
# 自动检测平台，添加对应 SDK Python 绑定路径
if platform.system() == "Windows":
    sys.path.append(os.path.join(_script_dir, "../../lib/windows/python"))
elif platform.machine() in ("aarch64", "arm64"):
    sys.path.append(os.path.join(_script_dir, "../../lib/linux/arm64/python"))
else:
    sys.path.append(os.path.join(_script_dir, "../../lib/linux/x86/python"))
# Windows: 将 tl_host.dll 所在目录加入 DLL 搜索路径
if platform.system() == "Windows":
    _dll_dir = os.path.join(_script_dir, "../../lib/windows")
    os.add_dll_directory(_dll_dir)
from tl_interface import *
import time
# ============================================================
# 工具函数
# ============================================================
def print_result(msg, ret):
    """输出操作结果
    @param msg 操作描述
    @param ret 返回码（Result枚举值）
    """
    if ret == SUCCESS:
        print("  [成功] " + msg)
    else:
        print("  [失败] " + msg + ", 错误码: " + str(ret))
```

```
def print_separator(title):

    """输出分隔标题"""

    print("\\n========== " + title + " ==========")

def enable_servo(socket_fd):

    """使能上电 ------
先清错，再检查伺服状态，报警则报错退出，最后上电

    @param socket_fd 6001端口 socket

    """

    print("--- 检查伺服状态 ---")

    # 1. 先清除控制器错误和报警，再下电复位确保状态同步

    clear_error(socket_fd)

    time.sleep(0.3)

    set_servo_poweroff(socket_fd)

    time.sleep(0.3)

    # 2. 检查伺服状态

    state = -1

    ret, state = get_servo_state(socket_fd, state)

    if ret != SUCCESS:

        print(f"[ERROR] 获取伺服状态失败: {ret}")

        return

    print(f"  [信息] 当前伺服状态: {state} (0=停止 1=就绪 2=报警 3=运行)")

    if state == 2:

        print("[ERROR] 伺服处于报警状态，无法上电，请排查报警原因后重试")

        return

    if state == 3:

        print("[INFO] 机械臂已在使能上电状态")

        return
```

```
    # 3. 上电流程
    if state == 0:
        print("  [信息] 伺服处于停止状态，设置就绪...")
        ret = set_servo_state(socket_fd, 1)
        if ret != SUCCESS and ret != OPERATION_NOT_ALLOWED:
            print(f"[ERROR] 设置伺服就绪失败: {ret}")
            return
        time.sleep(0.5)
    print("  [信息] 上电使能...")
    ret = set_servo_poweron(socket_fd)
    if ret != SUCCESS:
        print(f"[ERROR] 上电失败: {ret}")
        return
    time.sleep(2)
    # 4. 确认结果
    _, state = get_servo_state(socket_fd, state)
    if state == 3:
        print("[INFO] 机械臂使能上电成功（servo_state = " + str(state) + "）")
    else:
        print("[ERROR] 机械臂使能上电失败（servo_state = " + str(state) + "）")
def disable_servo(socket_fd):
    """使能下电
    @param socket_fd 6001端口 socket
    """
    state = -1
    _, state = get_servo_state(socket_fd, state)
    if state == 1:
```

```
        print("[INFO] 机械臂已使能下电")
    elif state == 3:
        set_servo_poweroff(socket_fd)
        _, state = get_servo_state(socket_fd, state)
        print("[INFO] 机械臂使能下电成功（servo_state = " + str(state) + "）")
        return
    print("[INFO] 机械臂使能下电失败（servo_state = " + str(state) + "）")
def wait_motion_done(socket_fd, timeout_sec):
    """轮询机器人运行状态等待运动完成，替代固定延时
    @param socket_fd TCP socket
    @param timeout_sec 超时秒数
    @return True 运动完成 / False 超时
    """
    deadline = time.time() + timeout_sec
    last_print = 0
    time.sleep(0.1)
    while time.time() < deadline:
        ret, state = get_robot_running_state(socket_fd, -1)
        if ret == SUCCESS and state == 0:
            print("  [信息] 运动完成")
            return True
        time.sleep(0.2)
        elapsed = int(time.time() - (deadline - timeout_sec))
        if elapsed - last_print >= 5:
            print(f"  [信息] 等待运动中... (已过 {elapsed}s)")
            last_print = elapsed
```

```
    print(f"  [WARN] 等待运动超时 ({timeout_sec}s)")
    return False

# ============================================================
# 主函数
# ============================================================
def main():
    # ---- 定义IP地址和端口 ----
    ip = "192.168.1.13"
    port = "6001"
    # ---- 获取 SDK 版本 ----
    version = get_library_version()
    print("[INFO] SDK 版本: " + version)
    # ---- 连接控制器 ----
    print("[INFO] 连接控制器...")
    socket_fd = connect_robot(ip, port)
    if socket_fd < 0:
        print("[ERROR] 连接失败，程序退出!")
        return -1
    print("[INFO] 连接成功 (socket=" + str(socket_fd) + ")")
    # ---- 切换为示教模式 ----
    print("--- 切换示教模式 ---")
    ret = set_current_mode(socket_fd, 0)
    if ret != SUCCESS:
        print("[ERROR] 切换示教模式失败，程序退出!")
        return -1
    print("[INFO] 已切换为示教模式")
    print()
```

```
    # ---- 使能上电 ----
    enable_servo(socket_fd)
    # ---- 设置全局速度 ----
    ret = set_speed(socket_fd, 30)
    print_result("set_speed", ret)
    time.sleep(0.2)
    # ---- MoveJ运动控制 ----
    print("[INFO] **********MoveJ运动控制**********")
    cmd = MoveCmd()
    cmd.targetPosType = PosType_PType
    cmd.targetPosName = ""
    cmd.coord = 0               # 关节坐标系
    cmd.velocity = 40           # 关节速度百分比 (1,100]
    cmd.acc = 10
    cmd.dec = 10
    cmd.pl = 0                  # 平滑参数 [0,5]
    cmd.targetPosValue = VectorDouble([
        20.0, 10.0, -10.0,
        0.0, 0.0, 0.0, 0.0])  # 关节坐标下单位为度
    ret = robot_movej(socket_fd, cmd)
    if ret != SUCCESS:
        print("[ERROR] MoveJ运动控制失败，程序退出!")
        return -1
    # 等待运动到位
    wait_motion_done(socket_fd, 30)
    print("[INFO] MoveJ运动控制成功")
```

```
    print()
    # ---- MoveL运动控制 ----
    print("[INFO] **********MoveL运动控制**********")
    cmd.targetPosName = ""
    cmd.coord = 1               # 直角坐标系
    cmd.targetPosValue = VectorDouble([
        280, 90.0, 270,
        3.14, 0.0, 0.0])  # 直角坐标下位移单位是mm，旋转单位是弧度
    ret = robot_movel(socket_fd, cmd)
    if ret != SUCCESS:
        print("[ERROR] MoveL运动控制失败，程序退出!")
        return -1
    # 等待运动到位
    wait_motion_done(socket_fd, 30)
    print("[INFO] MoveL运动控制成功")
    print()
    # ---- 使能下电 ----
    disable_servo(socket_fd)
    # ---- 断开控制器连接 ----
    print("[INFO] 断开控制器连接...")
    ret = disconnect_robot(socket_fd)
    if ret != SUCCESS:
        print("[ERROR] 断开连接失败，程序退出!")
        return -1
    print("[INFO] 断开连接成功")
    return 0
if __name__ == "__main__":
```

```
    main()
```

6.4点动使用示例

```
# /**
#  * ex_teach_control.py
#  * @brief 示教控制相关接口
#  * @attention
#  *   - 运行模式: 示教模式
#  *   - 开始前需: connect_robot → set_servo_state(1) → set_servo_poweron
#  *   - 退出前需: set_servo_poweroff → disconnect_robot
#  *   - 确保机械臂供电正常、网络通信正常
#  * @note 运行步骤
#  *       运行: python3 ex_teach_control.py
#  */
import sys
import os
import platform
# 基于脚本所在目录构建 SDK Python 绑定的绝对路径
_script_dir = os.path.dirname(os.path.abspath(__file__))
# 自动检测平台，添加对应 SDK Python 绑定路径
if platform.system() == "Windows":
    sys.path.append(os.path.join(_script_dir, "../../lib/windows/python"))
elif platform.machine() in ("aarch64", "arm64"):
    sys.path.append(os.path.join(_script_dir, "../../lib/linux/arm64/python"))
else:
    sys.path.append(os.path.join(_script_dir, "../../lib/linux/x86/python"))
```

```
# Windows: 将 tl_host.dll 所在目录加入 DLL 搜索路径
if platform.system() == "Windows":
    _dll_dir = os.path.join(_script_dir, "../../lib/windows")
    os.add_dll_directory(_dll_dir)
from tl_interface import *
import time
# ----工具函数----
def print_result(msg, ret):
    """打印操作结果"""
    if ret == SUCCESS:
        print("  [成功] " + msg)
    else:
        print("  [失败] " + msg + ", 错误码: " + str(ret))
def print_separator(title):
    """打印分隔标题"""
    print("\\n========== " + title + " ==========")
def enable_servo(socket_fd):
    """使能上电 ------
先清错，再检查伺服状态，报警则报错退出，最后上电"""
    print("--- 检查伺服状态 ---")
    # 1. 先清除控制器错误和报警，再下电复位确保状态同步
    ret = clear_error(socket_fd)
    time.sleep(0.3)
    set_servo_poweroff(socket_fd)
    time.sleep(0.3)
    # 2. 检查伺服状态
    state = -1
```

```
ret, state = get_servo_state(socket_fd, state)
if ret != SUCCESS:
    print(f"[ERROR] 获取伺服状态失败: {ret}")
    return
print(f"  [信息] 当前伺服状态: {state} (0=停止 1=就绪 2=报警 3=运行)")
if state == 2:
    print("[ERROR] 伺服处于报警状态，无法上电，请排查报警原因后重试")
    return
if state == 3:
    print("[INFO] 机械臂已在使能上电状态")
    return
# 3. 上电流程
if state == 0:
    print("  [信息] 伺服处于停止状态，设置就绪...")
    ret = set_servo_state(socket_fd, 1)
    if ret != SUCCESS and ret != OPERATION_NOT_ALLOWED:
        print(f"[ERROR] 设置伺服就绪失败: {ret}")
        return
    time.sleep(0.5)
print("  [信息] 上电使能...")
ret = set_servo_poweron(socket_fd)
if ret != SUCCESS:
    print(f"[ERROR] 上电失败: {ret}")
    return
time.sleep(2)
# 4. 确认结果
```

```
    _, state = get_servo_state(socket_fd, state)
    if state == 3:
        print("[INFO] 机械臂使能上电成功（servo_state = " + str(state) + "）")
    else:
        print("[ERROR] 机械臂使能上电失败（servo_state = " + str(state) + "）")
def disable_servo(socket_fd):
    """使能下电 ------ 安全关闭伺服"""
    state = -1
    _, state = get_servo_state(socket_fd, state)
    if state == 1:
        print("[INFO] 机械臂已使能下电")
    elif state == 3:
        set_servo_poweroff(socket_fd)
        _, state = get_servo_state(socket_fd, state)
        print("[INFO] 机械臂使能下电成功（servo_state = " + str(state) + "）")
        return
    else:
        pass
    print("[INFO] 机械臂使能下电失败（servo_state = " + str(state) + "）")
def demo_set_current_teach_sensitivity(socket_fd):
    """设置电流环拖动示教灵敏度"""
    print("[INFO] **********设置电流环拖动示教灵敏度**********")
    friction_offsets = VectorDouble()
    friction_offsets.append(1)   # J1
    friction_offsets.append(1)   # J2
    friction_offsets.append(1)   # J3
    friction_offsets.append(1)   # J4
```

```
    friction_offsets.append(1)   # J5
    friction_offsets.append(1)   # J6
    ret = set_current_teach_sensitivity(socket_fd, friction_offsets)
    if ret != SUCCESS:
        print("[ERROR] 设置电流环拖动示教灵敏度失败，错误码: " + str(ret))
        return
    print("[INFO] 设置电流环拖动示教灵敏度成功")
    print()
# ============================================================
# 主程序
# ============================================================
if __name__ == "__main__":
    # 定义IP地址和端口
    ip = "192.168.1.13"
    port = "6001"
    # 获取 SDK 版本
    version = get_library_version()
    print("[INFO] SDK 版本: " + version)
    # 连接控制器
    print("[INFO] 连接控制器...")
    socket_fd = connect_robot(ip, port)
    if socket_fd < 0:
        print("[ERROR] 连接失败，程序退出!")
        sys.exit(-1)
    print("[INFO] 连接成功 (socket=" + str(socket_fd) + ")")
    # 使能上电
```

```python
enable_servo(socket_fd)

# ----切换为示教模式----

# 示教点动需要在示教模式下执行（0=示教）

print("--- 切换示教模式 ---")

ret = set_current_mode(socket_fd, 0)

if ret != SUCCESS:

    print("[ERROR] 切换示教模式失败，程序退出!")

    sys.exit(-1)

print("[INFO] 已切换为示教模式")

# ----示教点动（注意安全）----

# ⚠️  robot_start_jogging 会直接驱动机器人运动，运行前确保安全区域无人

print("[INFO] **********示教点动**********")

axis = 1

direction = True

ret = robot_start_jogging(socket_fd, axis, direction)

if ret != SUCCESS:

    print("[ERROR] 示教点动失败，程序退出!")

    sys.exit(-1)

print("[INFO] 示教点动成功")

time.sleep(5)

print("[INFO] **********停止点动**********")

axis = 1

ret = robot_stop_jogging(arm.socket_fd, axis)

if ret != SUCCESS:

    print("[ERROR] 停止点动失败，程序退出!")

    sys.exit(-1)

print("[INFO] 停止点动成功")
```

```
    # ----设置电流环拖动示教灵敏度----

    demo_set_current_teach_sensitivity(socket_fd)

    # 使能下电

    disable_servo(socket_fd)

    # 断开控制器连接

    print("[INFO] 断开控制器连接...")

    ret = disconnect_robot(socket_fd)

    if ret != SUCCESS:

        print("[ERROR] 断开连接失败，程序退出!")

        sys.exit(-1)

    print("[INFO] 断开连接成功")
```

6.5作业文件指令使用示例

```
# /**

#  * ex_job_control.py

#  * @brief 作业控制相关接口（包括作业文件插入MoveJ、MoveL、MoveC，打开作业文件，创建作业文件，获取全部作业文件名，运行作业文件）

#  * @attention

#  *    - 插入作业之前需要先打开一个作业文件

#  *    - 运行一个作业文件需要切换到运行模式

#  *    - 确保机械臂供电正常、网络通信正常

#  * @note 运行步骤

#  *       运行: python3 ex_job_control.py

#  */

import sys

import os
```

```
import platform

# 基于脚本所在目录构建 SDK Python 绑定的绝对路径

_script_dir = os.path.dirname(os.path.abspath(__file__))

# 自动检测平台，添加对应 SDK Python 绑定路径

if platform.system() == "Windows":

    sys.path.append(os.path.join(_script_dir, "../../lib/windows/python"))

elif platform.machine() in ("aarch64", "arm64"):

    sys.path.append(os.path.join(_script_dir, "../../lib/linux/arm64/python"))

else:

    sys.path.append(os.path.join(_script_dir, "../../lib/linux/x86/python"))

# Windows: 将 tl_host.dll 所在目录加入 DLL 搜索路径

if platform.system() == "Windows":

    _dll_dir = os.path.join(_script_dir, "../../lib/windows")

    os.add_dll_directory(_dll_dir)

from tl_interface import *

import time

def enable_servo(socket_fd):

    """使能上电 ------
先清错，再检查伺服状态，报警则报错退出，最后上电"""

    print("--- 检查伺服状态 ---")

    # 1. 先清除控制器错误和报警，再下电复位确保状态同步

    ret = clear_error(socket_fd)

    time.sleep(0.3)

    set_servo_poweroff(socket_fd)

    time.sleep(0.3)

    # 2. 检查伺服状态

    state = -1
```

```
    ret, state = get_servo_state(socket_fd, state)
    if ret != SUCCESS:
        print(f"[ERROR] 获取伺服状态失败: {ret}")
        return
    print(f"  [信息] 当前伺服状态: {state} (0=停止 1=就绪 2=报警 3=运行)")
    if state == 2:
        print("[ERROR] 伺服处于报警状态，无法上电，请排查报警原因后重试")
        return
    if state == 3:
        print("[INFO] 机械臂已在使能上电状态")
        return
    # 3. 上电流程
    if state == 0:
        print("  [信息] 伺服处于停止状态，设置就绪...")
        ret = set_servo_state(socket_fd, 1)
        if ret != SUCCESS and ret != OPERATION_NOT_ALLOWED:
            print(f"[ERROR] 设置伺服就绪失败: {ret}")
            return
        time.sleep(0.5)
    print("  [信息] 上电使能...")
    ret = set_servo_poweron(socket_fd)
    if ret != SUCCESS:
        print(f"[ERROR] 上电失败: {ret}")
        return
    time.sleep(2)
    # 4. 确认结果
    _, state = get_servo_state(socket_fd, state)
```

```
    if state == 3:
        print("[INFO] 机械臂使能上电成功（servo_state = %d）" % state)
    else:
        print("[ERROR] 机械臂使能上电失败（servo_state = %d）" % state)
def disable_servo(socket_fd):
    """@brief 使能下电"""
    state = -1
    _, state = get_servo_state(socket_fd, state)
    if state == 1:
        print("[INFO] 机械臂已使能下电")
        return
    elif state == 3:
        set_servo_poweroff(socket_fd)
        _, state = get_servo_state(socket_fd, state)
        print("[INFO] 机械臂使能下电成功（servo_state = %d）" % state)
        return
    print("[INFO] 机械臂使能下电失败（servo_state = %d）" % state)
def main():
    # 定义IP地址和端口
    ip = "192.168.1.13"
    port = "6001"
    # 获取 SDK 版本
    version = get_library_version()
    print("[INFO] SDK 版本: %s" % version)
    # 连接控制器
    print("[INFO] 连接控制器...")
```

```
    socket_fd = connect_robot(ip, port)
    if socket_fd < 0:
        print("[ERROR] 连接失败，程序退出!")
        return -1
    print("[INFO] 连接成功 (socket=%d)" % socket_fd)
    # 使能上电
    enable_servo(socket_fd)
    # ----创建作业文件----
    jobName = "TEST_PY"
    print("[INFO] **********创建作业文件**********")
    ret = job_create(socket_fd, jobName)
    if ret != SUCCESS:
        print("[ERROR] 创建作业文件失败，程序退出!")
        return -1
    print("[INFO] 创建作业文件成功")
    print()
    # ----打开作业文件----
    print("[INFO] **********打开作业文件**********")
    ret = job_open(socket_fd, jobName)
    if ret != SUCCESS:
        print("[ERROR] 打开作业文件失败，程序退出!")
        return -1
    print("[INFO] 打开作业文件成功")
    print()
    # ----作业文件插入MoveJ----
    print("[INFO] **********作业文件插入MoveJ**********")
    line = 1
```

```
cmd = MoveCmd()
cmd.targetPosType  = PosType_PType
cmd.targetPosName  = "P0001"
cmd.coord          = 0
cmd.velocity       = 40
cmd.acc            = 10
cmd.dec            = 10
cmd.pl             = 0
cmd.targetPosValue = VectorDouble([0.0] * 14)
ret = job_insert_moveJ(socket_fd, line, cmd)
if ret != SUCCESS:
    print("[ERROR] 作业文件插入MoveJ失败，程序退出!")
    return -1
print("[INFO] 作业文件插入MoveJ成功")
print()
# ----作业文件插入MoveL----
print("[INFO] **********作业文件插入MoveL**********")
line = 2
cmd.targetPosName  = "P0002"
cmd.coord          = 1
cmd.targetPosValue = VectorDouble([280, 0.0, 380, 3.14, 0.0, 0.0])
ret = job_insert_moveL(socket_fd, line, cmd)
if ret != SUCCESS:
    print("[ERROR] 作业文件插入MoveL失败，程序退出!")
    return -1
print("[INFO] 作业文件插入MoveL成功")
```

```
    print()
    # ----作业文件插入MoveC（姿态1）----
    print("[INFO] **********作业文件插入MoveC**********")
    line = 3
    cmd.targetPosName  = "P0003"
    cmd.coord          = 2
    cmd.targetPosValue = VectorDouble([270, 0.0, 380, 3.14, 0.0, 0.0])
    ret = job_insert_moveC(socket_fd, line, cmd)
    if ret != SUCCESS:
        print("[ERROR] 作业文件插入MoveC失败，程序退出!")
        return -1
    print("[INFO] 作业文件插入MoveC成功")
    print()
    # ----作业文件插入MoveC（姿态2）----
    print("[INFO] **********作业文件插入MoveC**********")
    line = 3
    cmd.targetPosName  = "P0003"
    cmd.coord          = 2
    cmd.targetPosValue = VectorDouble([240, 0.0, 380, 3.14, 0.0, 0.0])
    ret = job_insert_moveC(socket_fd, line, cmd)
    if ret != SUCCESS:
        print("[ERROR] 作业文件插入MoveC失败，程序退出!")
        return -1
    print("[INFO] 作业文件插入MoveC成功")
    print()
    # ----获取所有作业文件名----
    print("[INFO] **********获取所有作业文件名**********")
```

```
AllJobNames = VectorVectorString()

ret = job_get_all_jobfile_name(socket_fd, AllJobNames)

if ret != SUCCESS:

    print("[ERROR] 获取作业文件名失败，程序退出!")

    return -1

i = 1

for JobNames in AllJobNames:

    for name in JobNames:

        print("[INFO] 作业文件 %d 名称: %s" % (i, name))

        i += 1

# ----切换为运行模式----

# 运行作业文件需要在运行模式下执行（2=运行）

print("--- 切换运行模式 ---")

ret = set_current_mode(socket_fd, 2)

if ret != SUCCESS:

    print("[ERROR] 切换运行模式失败，程序退出!")

    return -1

print("[INFO] 已切换为运行模式")

# ----运行作业文件----

print("[INFO] **********运行作业文件**********")

ret = job_run(socket_fd, jobName)

if ret != SUCCESS:

    print("[ERROR] 运行作业文件失败，程序退出!")

    return -1

print("[INFO] 运行作业文件成功")

print()
```

```
    # 使能下电
    disable_servo(socket_fd)
    # 断开控制器连接
    print("[INFO] 断开控制器连接...")
    ret = disconnect_robot(socket_fd)
    if ret != SUCCESS:
        print("[ERROR] 断开连接失败，程序退出!")
        return -1
    print("[INFO] 断开连接成功")
    return 0
if __name__ == "__main__":
    main()
```

6.6队列运动指令使用示例

```
# /**
#  * ex_queue_motion.py
#  * @brief 队列运动示例程序 ------ 展示队列运动完整工作流
#  * @attention
#  *   - 确保机械臂供电正常、网络通信正常
#  *   - 默认控制器 IP 为 192.168.1.13，可根据实际环境修改
#  *   - 队列运动模式下，所有指令先在本地缓存，调用 send_to_controller 后一次性下发执行
#  *   - 演示四种队列运动：纯 MoveJ、纯 MoveL、MoveJ+MoveL 混合、暂停/恢复/停止
#  *   - 运行模式:队列模式（在queue_motion_set_status(true)之后，即可进入队列模式，在此之前建议
set_current_mode(sock, 2)切换到运行模式）
#  *   - 开始前需: connect_robot → 上电 → queue_motion_set_status(true)
#  *   - 退出前需: queue_motion_set_status(false) → set_current_mode(0) →
set_servo_poweroff → disconnect_robot
```

```
#  * @note 运行步骤

#  *       运行: python3 ex_queue_motion.py

#  */

import sys

import os

import platform

# 基于脚本所在目录构建 SDK Python 绑定的绝对路径

_script_dir = os.path.dirname(os.path.abspath(__file__))

# 自动检测平台，添加对应 SDK Python 绑定路径

if platform.system() == "Windows":

    sys.path.append(os.path.join(_script_dir, "../../lib/windows/python"))

elif platform.machine() in ("aarch64", "arm64"):

    sys.path.append(os.path.join(_script_dir, "../../lib/linux/arm64/python"))

else:

    sys.path.append(os.path.join(_script_dir, "../../lib/linux/x86/python"))

# Windows: 将 tl_host.dll 所在目录加入 DLL 搜索路径

if platform.system() == "Windows":

    _dll_dir = os.path.join(_script_dir, "../../lib/windows")

    os.add_dll_directory(_dll_dir)

from tl_interface import *

import time

# ============================================================

# 工具函数

# ============================================================

def print_result(msg, ret):

    """输出操作结果

    @param msg 操作描述
```

```
    @param ret 返回码（Result枚举值）
    """
    if ret == SUCCESS:
        print("  [成功] " + msg)
    else:
        print("  [失败] " + msg + ", 错误码: " + str(ret))
def print_separator(title):
    """输出分隔标题"""
    print("\\n========== " + title + " ==========")
# ============================================================
# MoveCmd 构建函数
# ============================================================
# @brief 构建 MoveCmd（关节坐标模式）
# @param joints 6 个关节角度值，单位 °（度），顺序 [J1, J2, J3, J4, J5, J6]
# @param vel    百分比 % (1,100]，最大关节速度的百分比，实际受全局速度限制
# @param acc    百分比 % (1,100]
# @param dec    百分比 % (1,100]
# @param pl     平滑参数 [0,5]
# @note 实际运动速度 = 全局速度 × (vel / 100)
def make_movej_cmd(joints, vel=30, acc=50, dec=50, pl=0):
    cmd = MoveCmd()
    cmd.coord = 0        # 关节坐标系
    cmd.velocity = vel
    cmd.acc = acc
    cmd.dec = dec
    cmd.pl = pl
```

```
    # 填充 targetPosValue（前7位为关节值，后7位为外部轴，补零到14位）
    # 注意: SWIG 要求 VectorDouble 类型，不能用 Python list 直接赋值
    target = VectorDouble()
    for i in range(14):
        target.append(joints[i] if i < len(joints) else 0.0)
    cmd.targetPosValue = target
    return cmd
# @brief 构建 MoveCmd（直角坐标模式）
# @param pose 位姿 [X, Y, Z, Rx, Ry, Rz]，位置单位 mm，姿态单位 °
# @param vel    百分比 % (1,1000]，最大 TCP 线速度的百分比
# @param acc    百分比 % (1,100]
# @param dec    百分比 % (1,100]
# @param pl     平滑参数 [0,5]
# @note 实际运动速度 = 全局速度 × (vel / 100)
def make_movel_cmd(pose, vel=100, acc=50, dec=50, pl=0):
    cmd = MoveCmd()
    cmd.coord = 1        # 直角坐标系，可切换到 2：工具坐标系，3：用户坐标系
    cmd.velocity = vel
    cmd.acc = acc
    cmd.dec = dec
    cmd.pl = pl
    # 填充 targetPosValue（前7位为位姿值，后7位为外部轴，补零到14位）
    target = VectorDouble()
    for i in range(14):
        target.append(pose[i] if i < len(pose) else 0.0)
    cmd.targetPosValue = target
    return cmd
```

```python
# ============================================================

# 队列运动执行器

# ============================================================

# @brief 执行一轮队列运动

# @param sock       socket 句柄

# @param name       队列名称（用于打印）

# @param fn_push    回调函数，在此函数中调用 queue_motion_push_back_*

#                   回调会在队列模式开启后、发送前被调用

# @return True 成功 / False 失败

def run_queue_motion(sock, name, fn_push):

    print_separator("队列运动：" + name)

    # 1. 打开队列运动模式（会清空远端队列）

    ret = queue_motion_set_status(sock, True)

    print_result("queue_motion_set_status(true)", ret)

    if ret != SUCCESS:

        return False

    # 确认队列模式已开启

    # 注意: status 为输出参数，SWIG Python 绑定中传递 bool 类型的可变对象

    status = False

    _, status = queue_motion_get_status(sock, status)

    print("  [信息] 队列模式状态: " + ("已开启" if status else "已关闭"))

    # 2. 插入运动指令

    print("  [信息] 插入运动指令...")

    if not fn_push(sock):

        print("[ERROR] 插入运动指令失败")

        queue_motion_set_status(sock, False)
```

```
        return False
    # 3. 查询本地队列长度
    # 注意: size 为输出参数
    size = 0
    ret, size = queue_motion_size(sock, size)
    if ret == SUCCESS:
        print("  [信息] 本地队列长度: " + str(size) + " 条指令")
    # 4. 发送队列到控制器并等待执行完成
    print("  [信息] 发送队列到控制器...")
    ret = queue_motion_send_to_controller(sock, 0, False)
    print_result("send_to_controller", ret)
    if ret != SUCCESS:
        queue_motion_set_status(sock, False)
        return False
    print("  [信息] 等待运动执行完成...")
    while True:
        time.sleep(0.1)
        # 注意: running_state 为输出参数
        running_state = -1
        ret, running_state = get_robot_running_state(sock, running_state)
        if ret == SUCCESS and running_state == 0:
            break
        time.sleep(0.5)
    print("  [信息] 队列运动执行完毕")
    time.sleep(1.0)
    # 5. 关闭队列运动模式
    queue_motion_set_status(sock, False)
```

```
    return True

# ============================================================

# 主函数

# ============================================================

def main():

    # ---- 获取 SDK 版本 ----

    version = get_library_version()

    print("[信息] SDK 版本: " + version)

    # ---- 连接控制器 ----

    print_separator("连接控制器")

    sock = connect_robot("192.168.1.13", "6001")

    if sock < 0:

        print("[ERROR] 连接控制器失败")

        return -1

    print("[信息] 连接成功 (socket = " + str(sock) + ")")

    # ---- 清错 ----

    clear_error(sock)

    time.sleep(0.3)

    set_servo_poweroff(sock)

    time.sleep(0.3)

    # ---- 检查模式并确保伺服运行 ----

    # 先获取当前模式

    current_mode = -1

    ret, current_mode = get_current_mode(sock, current_mode)

    if ret != SUCCESS or current_mode < 0:

        print("[ERROR] 获取当前模式失败: " + str(ret))
```

```
        disconnect_robot(sock)
        return -1
    print("  [信息] 当前模式: " + str(current_mode) + " (0=示教 1=远程 2=运行)")
    if current_mode == 0:
        # 示教模式 → 先确保伺服就绪上电，再切换为运行模式
        servo_state = -1
        ret, servo_state = get_servo_state(sock, servo_state)
        if ret != SUCCESS:
            print("[ERROR] 获取伺服状态失败: " + str(ret))
            disconnect_robot(sock)
            return -1
        print("  [信息] 伺服状态: " + str(servo_state) + " (0=停止 1=就绪 2=报警 3=运行)")
        if servo_state != 3:
            # 伺服未运行 → 执行上电流程
            print("  [信息] 伺服未运行，执行上电流程...")
            ret = set_servo_state(sock, 1)
            if ret != SUCCESS and ret != OPERATION_NOT_ALLOWED:
                print("[ERROR] 设置伺服就绪失败: " + str(ret))
                disconnect_robot(sock)
                return -1
            time.sleep(0.5)
            ret = set_servo_poweron(sock)
            if ret != SUCCESS:
                print("[ERROR] 上电失败: " + str(ret))
                disconnect_robot(sock)
                return -1
            print("  [信息] 上电成功")
```

```
            time.sleep(2)
        else:
            print("  [信息] 伺服已在运行状态")
        # 切换为运行模式
        print("  [信息] 切换为运行模式...")
        ret = set_current_mode(sock, 2)
        if ret != SUCCESS:
            print("[ERROR] 切换运行模式失败: " + str(ret))
            disconnect_robot(sock)
            return -1
        print("  [信息] 已切换为运行模式")
        time.sleep(1.0)
    elif current_mode == 2:
        # 已在运行模式，直接检查伺服状态
        servo_state = -1
        ret, servo_state = get_servo_state(sock, servo_state)
        if ret != SUCCESS or servo_state != 3:
            print("[ERROR] 伺服未在运行状态 (servo_state=" + str(servo_state) + ")")
            disconnect_robot(sock)
            return -1
        print("  [信息] 已在运行模式，伺服运行中 (servo_state=3)")
    else:
        print("[ERROR] 不支持的模式: " + str(current_mode) + "，期望示教(0)或运行(2)模式")
        disconnect_robot(sock)
        return -1
    # 延时2秒，保证状态完全就绪
```

```
    time.sleep(2.0)

    # ---- 设置全局速度 ----

    # 控制器默认全局速度为 5%，设为 30% 提升运动效率

    print_separator("设置全局速度")

    ret = set_speed(sock, 30)

    print_result("set_speed(30)", ret)

    # ============================================================

    # 演示 1: 纯 MoveJ 队列运动

    # ============================================================

    def push_pure_movej(sock):

        r = queue_motion_push_back_moveJ(sock, make_movej_cmd([30, 0, 0, 0, 0, 0], 30))

        print("  push_back_moveJ:  J1=30°")

        print_result("push_back_moveJ", r)

        if r != SUCCESS:

            return False

        r = queue_motion_push_back_moveJ(sock, make_movej_cmd([30, 20, 0, 0, 0, 0], 30))

        print("  push_back_moveJ:  J1=30°, J2=20°")

        print_result("push_back_moveJ", r)

        if r != SUCCESS:

            return False

        r = queue_motion_push_back_moveJ(sock, make_movej_cmd([0, 20, 0, 0, 0, 0], 30))

        print("  push_back_moveJ:  J1=0°, J2=20°")

        print_result("push_back_moveJ", r)

        if r != SUCCESS:

            return False

        r = queue_motion_push_back_moveJ(sock, make_movej_cmd([0, 0, 0, 0, 0, 0], 20))

        print("  push_back_moveJ:  回零")
```

```
        print_result("push_back_moveJ", r)

        if r != SUCCESS:

            return False

        return True

    run_queue_motion(sock, "纯MoveJ", push_pure_movej)

    # ============================================================

    # 演示 2: 纯MoveL 队列运动

    # ============================================================

    def push_pure_movel(sock):

        r = queue_motion_push_back_moveL(sock, make_movel_cmd([300, 0, 300, 3.141, 0, 0]
, 30))

        print("  push_back_moveL:  X=300, Y=0, Z=300")

        print_result("push_back_moveL", r)

        if r != SUCCESS:

            return False

        r = queue_motion_push_back_moveL(sock, make_movel_cmd([400, 100, 300, 3.141, 0,
0], 30))

        print("  push_back_moveL:  X=400, Y=100, Z=300")

        print_result("push_back_moveL", r)

        if r != SUCCESS:

            return False

        r = queue_motion_push_back_moveL(sock, make_movel_cmd([300, 200, 300, 3.141, 0,
0], 30))

        print("  push_back_moveL:  X=300, Y=200, Z=300")

        print_result("push_back_moveL", r)

        if r != SUCCESS:

            return False

        r = queue_motion_push_back_moveL(sock, make_movel_cmd([229.882, 0, 358.680, 3.14
```

```
1, 0, 0], 20))

        print("  push_back_moveL:  回起始点")

        print_result("push_back_moveL", r)

        if r != SUCCESS:

            return False

        return True

    run_queue_motion(sock, "纯MoveL", push_pure_movel)

    # ============================================================

    # 演示 3: MoveJ + MoveL 混合队列运动

    # ============================================================

    def push_mixed(sock):

        # 先用MoveJ 调整姿态

        r = queue_motion_push_back_moveJ(sock, make_movej_cmd([45, 0,0, 0, 0, 0], 30))

        print("  push_back_moveJ:  J1=45°（调整关节姿态）")

        print_result("push_back_moveJ", r)

        if r != SUCCESS:

            return False

        # 再用MoveL 走直线

        r = queue_motion_push_back_moveL(sock, make_movel_cmd([400, 0, 300, 3.141, 0, 0]
, 30))

        print("  push_back_moveL:  X=400, Y=0, Z=300")

        print_result("push_back_moveL", r)

        if r != SUCCESS:

            return False

        # MoveJ 换向

        r = queue_motion_push_back_moveJ(sock, make_movej_cmd([-45, 0, 0, 0, 0, 0], 30))

        print("  push_back_moveJ:  J1=-45°（换向）")
```

```
        print_result("push_back_moveJ", r)

        if r != SUCCESS:

            return False

        # MoveL 走另一条直线

        r = queue_motion_push_back_moveL(sock, make_movel_cmd([300, 200, 300, 3.141, 0,
0], 30))

        print("  push_back_moveL:  X=300, Y=200, Z=300")

        print_result("push_back_moveL", r)

        if r != SUCCESS:

            return False

        # MoveJ 回零

        r = queue_motion_push_back_moveJ(sock, make_movej_cmd([0, 0, 0, 0, 0, 0], 20))

        print("  push_back_moveJ:  回零")

        print_result("push_back_moveJ", r)

        if r != SUCCESS:

            return False

        return True

    run_queue_motion(sock, "MoveJ + MoveL 混合", push_mixed)

    # ============================================================

    # 演示 4: 队列运动控制（暂停/恢复/停止保持上电）

    # ============================================================

    print_separator("队列运动控制：暂停/恢复/停止")

    # 辅助：等待队列剩余数减少到 target_len

    def wait_queue_len(sock, target_len):

        while True:

            time.sleep(0.2)

            length = -1
```

```
            _, length = queue_motion_get_queuelen(sock, length)
            if length <= target_len or length == -1:
                break
    # 1. 开启队列模式
    r = queue_motion_set_status(sock, True)
    print_result("queue_motion_set_status(true)", r)
    if r != SUCCESS:
        print("[ERROR] 开启队列模式失败")
        set_current_mode(sock, 0)
        disconnect_robot(sock)
        return -1
    time.sleep(2)
    # 2. 插入 4 条 J5 摆动指令（简单清晰，便于观察控制效果）
    print("  [信息] 插入 4 条 MoveJ 指令（J5 摆动）")
    targets = [
        [0, 0, 0, 0, -30, 0, 0],   # J5 → -30°
        [0, 0, 0, 0,   0, 0, 0],   # J5 → 0°
        [0, 0, 0, 0, -30, 0, 0],   # J5 → -30°
        [0, 0, 0, 0,   0, 0, 0],   # J5 → 0°
    ]
    for i, target in enumerate(targets):
        cmd = make_movej_cmd(target, 30)
        r = queue_motion_push_back_moveJ(sock, cmd)
        print("  第" + str(i + 1) + "条: push_back_moveJ (J5→" + str(target[4]) + "°)  "
+ ("成功" if r == SUCCESS else "失败"))
    # 3. 发送队列到控制器
    print("  [信息] 发送队列到控制器...")
```

```
    r = queue_motion_send_to_controller(sock, 0, False)
    print_result("send_to_controller", r)
    if r != SUCCESS:
        print("[ERROR] 发送队列失败")
        queue_motion_set_status(sock, False)
        set_current_mode(sock, 0)
        disconnect_robot(sock)
        return -1
    # 4. 第 1 条执行中 → 暂停
    print("  [信息] 第 1 条执行中，等待 2 秒后暂停...")
    time.sleep(2)
    r = queue_motion_suspend(sock)
    print_result("暂停 (suspend)", r)
    time.sleep(2)
    # 5. 恢复
    r = queue_motion_restart(sock)
    print_result("恢复 (restart)", r)
    print("  [信息] 等待第 1 条完成...")
    wait_queue_len(sock, 3)
    print("  [信息] 第 1 条完成")
    # 6. 等待第 2 条完成
    print("  [信息] 等待第 2 条完成...")
    wait_queue_len(sock, 2)
    print("  [信息] 第 2 条完成")
    # 7. 第 3 条执行中 → 暂停
    time.sleep(1)
    r = queue_motion_suspend(sock)
```

```
    print_result("再次暂停 (suspend)", r)
    time.sleep(2)
    # 8. 恢复
    r = queue_motion_restart(sock)
    print_result("再次恢复 (restart)", r)
    print("  [信息] 等待第 3 条完成...")
    wait_queue_len(sock, 1)
    print("  [信息] 第 3 条完成")
    # 9. 第 4 条执行中 → 停止（保持上电）
    time.sleep(2)
    r = queue_motion_stop_not_power_off(sock)
    print_result("停止保持上电 (stop_not_power_off)", r)
    time.sleep(1)
    # 10. 关闭队列模式
    queue_motion_set_status(sock, False)
    print("  [信息] 队列运动控制演示完毕")
    # ---- 切回示教模式并断开连接 ----
    print_separator("切回示教模式 + 断开连接")
    ret = set_current_mode(sock, 0)
    print_result("set_current_mode(0) 切回示教模式", ret)
    time.sleep(0.3)
    disconnect_robot(sock)
    print("[信息] 已断开连接")
    print("\\n[信息] 队列运动示例程序运行完毕")
    return 0
if __name__ == "__main__":
```

```
    main()
```

6.7坐标变换算法示例

```
# /**
#  * ex_coord_transform.py
#  * @brief 坐标变换算法示例程序 ------ 展示常用的坐标系转换接口
#  * @attention
#  *   - 需要控制器连接正常、已上电
#  *   - 默认控制器 IP 为 192.168.1.13，可根据实际环境修改
#  * @note 运行步骤
#  *       运行: python3 ex_coord_transform.py
#  */
import sys
import os
import platform
# 基于脚本所在目录构建 SDK Python 绑定的绝对路径
_script_dir = os.path.dirname(os.path.abspath(__file__))
# 自动检测平台，添加对应 SDK Python 绑定路径
if platform.system() == "Windows":
    sys.path.append(os.path.join(_script_dir, "../../lib/windows/python"))
elif platform.machine() in ("aarch64", "arm64"):
    sys.path.append(os.path.join(_script_dir, "../../lib/linux/arm64/python"))
else:
    sys.path.append(os.path.join(_script_dir, "../../lib/linux/x86/python"))
# Windows: 将 tl_host.dll 所在目录加入 DLL 搜索路径
if platform.system() == "Windows":
```

```
    _dll_dir = os.path.join(_script_dir, "../../lib/windows")
    os.add_dll_directory(_dll_dir)
from tl_interface import *
import time
import math
# ============================================================
# 工具函数
# ============================================================
def print_result(msg, ret):
    """打印操作结果"""
    if ret == SUCCESS:
        print("  [成功] " + msg)
    else:
        print("  [失败] " + msg + ", 错误码: " + str(ret))
def print_separator(title):
    """打印分隔标题"""
    print("\\n========== " + title + " ==========")
def print_vector(name, v):
    """打印向量 ------ 格式: name [size] = {v1, v2, ...}, 保留4位小数"""
    formatted = ["{:.4f}".format(x) for x in v]
    print("  " + name + " [" + str(len(v)) + "] = {" + ", ".join(formatted) + "}")
def print_matrix(name, m, rows, cols):
    """打印矩阵（行主序），保留4位小数"""
    print("  " + name + " [" + str(len(m)) + "]:")
    for r in range(rows):
        line = "    "
```

```
        for c in range(cols):
            idx = r * cols + c
            if idx < len(m):
                line += "{:10.4f} ".format(m[idx])
        print(line)
# ============================================================
# 主程序
# ============================================================
if __name__ == "__main__":
    # ---- 获取 SDK 版本 ----
    version = get_library_version()
    print("[信息] SDK 版本: " + version)
    # ---- 双端口连接 ----
    print_separator("连接控制器")
    sock = connect_robot("192.168.1.13", "6001")
    if sock < 0:
        print("[ERROR] 连接 6001 端口失败")
        sys.exit(-1)
    sock_servo = connect_robot("192.168.1.13", "7000")
    if sock_servo < 0:
        print("[ERROR] 连接 7000 端口失败")
        disconnect_robot(sock)
        sys.exit(-1)
    print("[信息] 双端口连接成功 (6001=" + str(sock) + ", 7000=" + str(sock_servo) + ")")
    # ---- 上电 (servoJ 需要上电状态) ----
    print_separator("清错与上电")
    clear_error(sock)
```

```
    time.sleep(0.3)
    set_servo_poweroff(sock)
    time.sleep(0.3)
    servo_state = -1
    ret, servo_state = get_servo_state(sock, servo_state)
    if ret != SUCCESS:
        print(f"[ERROR] 获取伺服状态失败: {ret}")
        disconnect_robot(sock)
        disconnect_robot(sock_servo)
        sys.exit(-1)
    print(f"  [信息] 当前伺服状态: {servo_state} (0=停止 1=就绪 2=报警 3=运行)")
    if servo_state == 2:
        print("[ERROR] 伺服处于报警状态，无法上电")
        disconnect_robot(sock)
        disconnect_robot(sock_servo)
        sys.exit(-1)
    if servo_state != 3:
        print("  [信息] 伺服未运行，执行上电流程...")
        ret = set_servo_state(sock, 1)
        if ret != SUCCESS and ret != OPERATION_NOT_ALLOWED:
            print(f"[ERROR] 设置伺服就绪失败: {ret}")
            disconnect_robot(sock)
            disconnect_robot(sock_servo)
            sys.exit(-1)
        time.sleep(0.5)
        ret = set_servo_poweron(sock)
```

```
        if ret != SUCCESS:
            print(f"[ERROR] 上电失败: {ret}")
            disconnect_robot(sock)
            disconnect_robot(sock_servo)
            sys.exit(-1)
        time.sleep(2)
        print("  [信息] 上电成功")
    else:
        print("  [信息] 伺服已运行 (servo_state=3)")
    # ============================================================
    # 1. 四元数 ↔ 欧拉角 (RPY) --- 正反验证
    # ============================================================
    # 单位说明:
    #   四元数 [x, y, z, w]       --- 无量纲（归一化）
    #   欧拉角 [roll, pitch, yaw] --- 弧度 rad
    #   旋转矩阵 (3×3 行主序)     --- 无量纲
    print_separator("四元数 ↔ 欧拉角")
    # 1a. 欧拉角 → 四元数 → 欧拉角（正反验证）
    print("--- 1a. RPY ↔ 四元数 正反验证 ---")
    rpy_in = [0.2, 0.3, 0.5]  # rad
    quat = VectorDouble()
    rpy_out = VectorDouble()
    ret = get_rpy2quat(sock, rpy_in, quat)
    print_result("get_rpy2quat (RPY→四元数)", ret)
    if ret != SUCCESS:
        sys.exit(-1)
    print_vector("  输入欧拉角 [rad]", rpy_in)
```

```
print_vector("  中间四元数 [x, y, z, w]", quat)

ret = get_quat2rpy(sock, quat, rpy_out)

print_result("get_quat2rpy (四元数→RPY)", ret)

if ret != SUCCESS:

    sys.exit(-1)

print_vector("  还原欧拉角 [rad]", rpy_out)

if len(rpy_out) >= 3 and len(rpy_in) >= 3:

    err = abs(rpy_out[0] - rpy_in[0]) \\

        + abs(rpy_out[1] - rpy_in[1]) \\

        + abs(rpy_out[2] - rpy_in[2])

    print("  正反变换误差 (应接近0): " + str(err))

# 1b. 欧拉角 → 旋转矩阵

print("\\n--- 1b. get_rpy2r: 欧拉角转旋转矩阵 ---")

rpy = [0.0, 0.0, math.pi / 2]  # rad

rot_mat = VectorDouble()

ret = get_rpy2r(sock, rpy, rot_mat)

print_result("get_rpy2r", ret)

print_vector("输入欧拉角 [rad]", rpy)

if ret == SUCCESS:

    print_matrix("输出旋转矩阵 (3x3, 行主序, 无量纲)", rot_mat, 3, 3)

# ============================================================

# 2. 位姿矩阵 ↔ 旋转矩阵 --- 正反验证

# ============================================================

# 单位说明:

#   旋转矩阵 R (3×3)     --- 无量纲

#   平移向量 T [Tx,Ty,Tz] --- 毫米 mm
```

```
#   位姿矩阵 TR (4×4)  --- R 部分无量纲，T 部分单位 mm
# 位姿矩阵 (行主序 16 元素):
#   [Rxx Rxy Rxz Tx]  第 0 行
#   [Ryx Ryy Ryz Ty]  第 1 行
#   [Rzx Rzy Rzz Tz]  第 2 行
#   [  0   0   0  1]  第 3 行
print_separator("位姿矩阵 ↔ 旋转矩阵")
# 2a. TR → R → TR（正反验证）
print("--- 2a. TR ↔ R 正反验证 ---")
# 构造一个绕 Z 转 90°、平移 (100, 200, 300) mm 的位姿矩阵
cos90 = math.cos(math.pi / 2)
sin90 = math.sin(math.pi / 2)
tr_in = [
    cos90, -sin90, 0.0, 100.0,
    sin90,  cos90, 0.0, 200.0,
    0.0,    0.0,   1.0, 300.0,
    0.0,    0.0,   0.0, 1.0
]
R = VectorDouble()
tr_out = VectorDouble()
ret = get_tr2r(sock, tr_in, R)
print_result("get_tr2r (TR→R)", ret)
if ret != SUCCESS:
    sys.exit(-1)
print_matrix("输入位姿矩阵 (4x4, R无量纲, T:mm)", tr_in, 4, 4)
print_matrix("中间旋转矩阵 (3x3, 无量纲)", R, 3, 3)
ret = get_r2tr(sock, R, tr_out)
```

```
print_result("get_r2tr (R→TR)", ret)

if ret != SUCCESS:

    sys.exit(-1)

print_matrix("还原位姿矩阵 (4x4, R无量纲, T:mm)", tr_out, 4, 4)

# 说明: get_tr2r 只提取旋转矩阵，平移 T 在此丢弃；

# get_r2tr 的输入不含平移，还原出的 TR 平移部分恒为 0。

# SDK 无 "R+T 构造 TR" 接口，完整位姿矩阵需由调用方自行拼装 T。

print("  注: get_tr2r 仅输出旋转矩阵，平移 T 不参与往返；")

print("      还原位姿矩阵的平移部分为 0，需调用方另行拼装。")

# 对比旋转部分（TR→R→TR 链路仅保证旋转信息无损）

# 注意: 行主序 4x4 中旋转块位于 0,1,2, 4,5,6, 8,9,10,

#       位置 3/7/11 是平移列，不能按连续 0..8 遍历。

if len(tr_out) >= 16 and len(tr_in) >= 16:

    rot_idx = [0, 1, 2, 4, 5, 6, 8, 9, 10]

    r_err = 0.0

    for k in rot_idx:

        r_err += abs(tr_out[k] - tr_in[k])

    print("  旋转矩阵误差 (应接近0): " + str(r_err))

# ============================================================

# 3. 坐标系转换（获取机器人当前位姿在不同坐标系下的表示）

# ============================================================

# 单位说明:

#   关节坐标 [J1..J6] --- 度 °

#   直角坐标 [X,Y,Z,Rx,Ry,Rz] --- 位移 mm，姿态 rad

#   工具/用户坐标 --- 同直角坐标

print_separator("坐标系转换")
```

```
    print("--- 3. get_origin_coord_to_target_coord ---")

    print("  坐标系: 0=关节(°)  1=直角(mm+rad)  2=工具(mm+rad) 3=用户(mm+rad)")

    # 从关节坐标系(0) 转换到直角坐标系(1)

    # 关节角度: J1=10°, J2=20°, J3=30°, J4=0°, J5=0°, J6=0°

    joint_pos = VectorDouble()

    joint_pos.push_back(10.0)

    joint_pos.push_back(20.0)

    joint_pos.push_back(30.0)

    joint_pos.push_back(0.0)

    joint_pos.push_back(0.0)

    joint_pos.push_back(0.0)

    joint_pos.push_back(0.0)

    cartesian_pos = VectorDouble()

    origin_reference_pos = VectorDouble([0]*7)

    ret = get_origin_coord_to_target_coord(sock, 0, joint_pos, 1, cartesian_pos, 0, orig
in_reference_pos)

    print_result("get_origin_coord_to_target_coord (关节→直角) 正解", ret)

    print_vector("输入关节角度 [°]", joint_pos)

    if ret == SUCCESS:

        print("  输出直角坐标 --- 前3位为位移[mm]，后3位为姿态[rad]:")

        print_vector("  直角坐标", cartesian_pos)

        if len(cartesian_pos) >= 6:

            print("    位移 X=" + str(cartesian_pos[0]) + "mm, Y=" + str(cartesian_pos[1]
)

                  + "mm, Z=" + str(cartesian_pos[2]) + "mm")

            print("    姿态 Rx=" + str(cartesian_pos[3]) + "rad, Ry=" +
str(cartesian_pos[4])

                  + "rad, Rz=" + str(cartesian_pos[5]) + "rad")
```

```
            print("    姿态换算(°): Rx=" + str(cartesian_pos[3] * 180.0 / math.pi)
                  + ", Ry=" + str(cartesian_pos[4] * 180.0 / math.pi)
                  + ", Rz=" + str(cartesian_pos[5] * 180.0 / math.pi))
    # 逆解: 直角坐标系(1) → 关节坐标系(0)
    # 使用正解输出的直角坐标作为输入，验证正反解一致性
    if ret == SUCCESS and len(cartesian_pos) >= 6:
        joint_back = VectorDouble()
        ret = get_origin_coord_to_target_coord(sock, 1, cartesian_pos, 0, joint_back,
0, origin_reference_pos)
        print_result("get_origin_coord_to_target_coord (直角→关节) 逆解", ret)
        print("  输入直角坐标:")
        print("    位移 [mm]: X=" + str(cartesian_pos[0]) + ", Y=" + str(cartesian_pos[1]
)
              + ", Z=" + str(cartesian_pos[2]))
        print("    姿态 [rad]: Rx=" + str(cartesian_pos[3]) + ", Ry=" +
str(cartesian_pos[4])
              + ", Rz=" + str(cartesian_pos[5]))
        if ret == SUCCESS:
            print_vector("  逆解关节角度 [°]", joint_back)
            # 对比原始关节角度，计算误差
            err = 0.0
            for i in range(min(len(joint_pos), len(joint_back))):
                err += abs(joint_pos[i] - joint_back[i])
            print("  正反解误差 (应接近0): " + str(err))
    # ============================================================
    # 4. 逆解关节坐标 → servoJ 下发运动
    # ============================================================
    # 演示流程: 给定直角坐标 → 逆解到关节角 → 通过 servoJ 发送到控制器
```

```
    print_separator("逆解 + servoJ 运动")

    # 取一个直角坐标点，逆解为关节角

    target_pose = [300, 0, 300, 3.141, 0, 0]  # X=300mm, Y=0, Z=300mm

    target_joints = VectorDouble()

    origin_reference_pos = VectorDouble([0]*7)

    ret = get_origin_coord_to_target_coord(sock, 1, target_pose, 0, target_joints, 0, or
igin_reference_pos)

    print_result("逆解: 直角→关节", ret)

    if ret == SUCCESS:

        print_vector("目标直角坐标 [mm, rad]", target_pose)

        print_vector("逆解关节角度 [°]", target_joints)

        # 通过 servoJ 发送关节角度到控制器

        # servoJ 需要连接 7000 端口，已在开头连接

        print("\\n  --- 通过 servoJ 下发逆解结果 ---")

        # servoJ 约束参数（6 轴）

        vmax = [30.0, 30.0, 30.0, 30.0, 30.0, 30.0]

        amax = [60.0, 60.0, 60.0, 60.0, 60.0, 60.0]

        jmax = [100.0, 100.0, 100.0, 100.0, 100.0, 100.0]

        ret = open_servoJ(sock_servo, vmax, amax, jmax)

        print_result("open_servoJ", ret)

        if ret == SUCCESS:

            # 逐步发送逆解出的关节角度，确保 servoJ 跟踪到位

            # 分 50 步从当前角度平滑过渡到目标角度

            current = [0, 0, 0, 0, 0, 0]

            steps = 50

            for i in range(steps):

                q = []
```

```
                for j in range(6):
                    q.append(current[j] + (target_joints[j] - current[j]) * (i + 1) / st
eps)
                set_servoJ_pos(sock_servo, q)
                time.sleep(0.01)
            # servoJ 是实时跟踪模式，等待固定时间确保运动到位
            print("  [信息] 等待 servoJ 运动到位...")
            time.sleep(5)
            close_servoJ(sock_servo)
            print_result("close_servoJ", ret)
    # ============================================================
    # 5. 不同坐标系转换（正解/逆解）--- 多机器人版本
    # ============================================================
    print_separator("坐标系转换（多机器人版本）")
    print("--- 5. get_origin_coord_to_target_coord ---")
    print("  坐标系: 0=关节(°)  1=直角(mm+rad)  2=工具(mm+rad) 3=用户(mm+rad)")
    # 正解: 关节坐标系(0) → 直角坐标系(1)
    print("\\n  --- 正解: 关节→直角 ---")
    joint_pos = VectorDouble()
    joint_pos.push_back(0.0)
    joint_pos.push_back(0.0)
    joint_pos.push_back(0.0)
    joint_pos.push_back(0.0)
    joint_pos.push_back(0.0)
    joint_pos.push_back(0.0)
    joint_pos.push_back(0.0)
    joint_pos.push_back(0.0)
    cartesian_pos = VectorDouble()
```

```
    origin_reference_pos = VectorDouble([0]*7)

    robot_num = 1

    ret = get_origin_coord_to_target_coord(sock, 0, joint_pos, 1, cartesian_pos, 0, orig
in_reference_pos)

    print_result("关节→直角 (robot_num=1) 正解", ret)

    print_vector("输入关节角度 [°]", joint_pos)

    if ret == SUCCESS:

        print_vector("输出直角坐标", cartesian_pos)

        if len(cartesian_pos) >= 6:

            print("    位移 [mm]: X=" + str(cartesian_pos[0]) + ", Y=" + str(cartesian_po
s[1])

                  + ", Z=" + str(cartesian_pos[2]))

            print("    姿态 [rad]: Rx=" + str(cartesian_pos[3]) + ", Ry=" + str(cartesian
_pos[4])

                  + ", Rz=" + str(cartesian_pos[5]))

    # 逆解: 直角坐标系(1) → 关节坐标系(0)

    if ret == SUCCESS and len(cartesian_pos) >= 6:

        print("\\n  --- 逆解: 直角→关节 ---")

        joint_back = VectorDouble()

        ret = get_origin_coord_to_target_coord(sock, 1, cartesian_pos, 0, joint_back,
0, origin_reference_pos)

        print_result("直角→关节 (robot_num=1) 逆解", ret)

        if ret == SUCCESS:

            print_vector("逆解关节角度 [°]", joint_back)

            err = 0.0

            for i in range(min(len(joint_pos), len(joint_back))):

                err += abs(joint_pos[i] - joint_back[i])

            print("  正反解误差 (应接近0): " + str(err))

    # ============================================================
```

```
    # 4. 逆解关节坐标 → servoJ 下发运动

    # ============================================================

    # 演示流程: 给定直角坐标 → 逆解到关节角 → 通过 servoJ 发送到控制器

    print_separator("逆解 + servoJ 运动")

    # 取一个直角坐标点，逆解为关节角

    target_pose = [300, 0, 300, 3.141, 0, 0]  # X=300mm, Y=0, Z=300mm

    target_joints = VectorDouble()

    origin_reference_pos = VectorDouble([0]*7)

    ret = get_origin_coord_to_target_coord(sock, 1, target_pose, 0, target_joints, 0, or
igin_reference_pos)

    print_result("逆解: 直角→关节", ret)

    if ret == SUCCESS:

        print_vector("目标直角坐标 [mm, rad]", target_pose)

        print_vector("逆解关节角度 [°]", target_joints)

        # 通过 servoJ 发送关节角度到控制器

        # servoJ 需要连接 7000 端口，已在开头连接

        print("\\n  --- 通过 servoJ 下发逆解结果 ---")

        # servoJ 约束参数（7 元素: 6 本体轴 + 1 外部轴）

        # 注意: SDK 底层协议要求 servoJ 相关向量固定为 7 元素

        vmax = [30.0, 30.0, 30.0, 30.0, 30.0, 30.0, 30.0]

        amax = [60.0, 60.0, 60.0, 60.0, 60.0, 60.0, 60.0]

        jmax = [100.0, 100.0, 100.0, 100.0, 100.0, 100.0, 100.0]

        ret = open_servoJ(sock_servo, vmax, amax, jmax)

        print_result("open_servoJ", ret)

        if ret == SUCCESS:

            # 逐步发送逆解出的关节角度，确保 servoJ 跟踪到位

            # 分 50 步从当前角度平滑过渡到目标角度
```

```
            # q 固定 7 元素，长度不够补 0（已在初始化时置零）
            axis_count = len(target_joints)
            current = [0.0] * axis_count
            steps = 50
            for i in range(steps):
                q = [0.0] * 7
                n = min(axis_count, len(q))
                for j in range(n):
                    q[j] = current[j] + (target_joints[j] - current[j]) * (i + 1) / step
s
                set_servoJ_pos(sock_servo, q)
                time.sleep(0.01)
            # servoJ 是实时跟踪模式，等待固定时间确保运动到位
            print("  [信息] 等待 servoJ 运动到位...")
            time.sleep(5)
            close_servoJ(sock_servo)
            print_result("close_servoJ", ret)
    # ---- 断开连接 ----
    print_separator("断开连接")
    disconnect_robot(sock)
    disconnect_robot(sock_servo)
    print("[信息] 已断开连接，机械臂保持运行模式")
print("\\n[信息] 坐标变换示例程序运行完毕")
```

6.8模式与 IO 控制接口示例

```
# /**
```

```python
# * ex_digital_io.py

# * @brief 模式与 IO 控制接口（包括设置机械臂模式、设置数字 IO 输出、查询数字 IO 输出、获取所有 IO 输入状态）

# * @attention 确保机械臂供电正常、网络通信正常

# * @note 运行步骤

# * 运行: python3 ex_digital_io.py

# */

import sys

import os

import platform

# 基于脚本所在目录构建 SDK Python 绑定的绝对路径

_script_dir = os.path.dirname(os.path.abspath(__file__))

# 自动检测平台，添加对应 SDK Python 绑定路径

if platform.system() == "Windows":

sys.path.append(os.path.join(_script_dir, "../../lib/windows/python"))

elif platform.machine() in ("aarch64", "arm64"):

sys.path.append(os.path.join(_script_dir, "../../lib/linux/arm64/python"))

else:

sys.path.append(os.path.join(_script_dir, "../../lib/linux/x86/python"))

# Windows: 将 tl_host.dll 所在目录加入 DLL 搜索路径

if platform.system() == "Windows":

_dll_dir = os.path.join(_script_dir, "../../lib/windows")

os.add_dll_directory(_dll_dir)

from tl_interface import *

import time

def enable_servo(socket_fd):

"""使能上电 ------
先清错，再检查伺服状态，报警则报错退出，最后上电"""
```

```
print("--- 检查伺服状态 ---")
# 1. 先清除控制器错误和报警，再下电复位确保状态同步
ret = clear_error(socket_fd)
time.sleep(0.3)
set_servo_poweroff(socket_fd)
time.sleep(0.3)
# 2. 检查伺服状态
state = -1
ret, state = get_servo_state(socket_fd, state)
if ret != SUCCESS:
print(f"[ERROR] 获取伺服状态失败: {ret}")
return
print(f" [信息] 当前伺服状态: {state} (0=停止 1=就绪 2=报警 3=运行)")
if state == 2:
print("[ERROR] 伺服处于报警状态，无法上电，请排查报警原因后重试")
return
if state == 3:
print("[INFO] 机械臂已在使能上电状态")
return
# 3. 上电流程
if state == 0:
print(" [信息] 伺服处于停止状态，设置就绪...")
ret = set_servo_state(socket_fd, 1)
if ret != SUCCESS and ret != OPERATION_NOT_ALLOWED:
print(f"[ERROR] 设置伺服就绪失败: {ret}")
return
```

```
time.sleep(0.5)

print(" [信息] 上电使能...")

ret = set_servo_poweron(socket_fd)

if ret != SUCCESS:

print(f"[ERROR] 上电失败: {ret}")

return

time.sleep(2)

# 4. 确认结果

_, state = get_servo_state(socket_fd, state)

if state == 3:

print("[INFO] 机械臂使能上电成功（servo_state = %d）" % state)

else:

print("[ERROR] 机械臂使能上电失败（servo_state = %d）" % state)

def disable_servo(socket_fd):

"""@brief 使能下电"""

state = -1

_, state = get_servo_state(socket_fd, state)

if state == 1:

print("[INFO] 机械臂已使能下电")

return

elif state == 3:

set_servo_poweroff(socket_fd) _, state = get_servo_state(socket_fd, state)

print("[INFO] 机械臂使能下电成功（servo_state = %d）" % state)

return

print("[INFO] 机械臂使能下电失败（servo_state = %d）" % state)

def demo_get_robot_running_state(socket_fd):

"""查询流程图编程运行状态"""
```

```
print("[INFO] **********查询流程图编程运行状态**********")

status = 0

ret, status = get_robot_running_state(socket_fd, status)

if ret != SUCCESS:

print("[ERROR] 查询流程图编程运行状态失败，错误码: %d" % ret)

return

if status == 0:

print("[INFO] 机械臂处于停止状态 (status = %d)" % status)

elif status == 1:

print("[INFO] 机械臂处于暂停状态 (status = %d)" % status)

elif status == 2:

print("[INFO] 机械臂处于运行状态 (status = %d)" % status)

print()

def main():

# 定义IP地址和端口

# 解析命令行参数: python3 ex_digital_io.py [-ip 192.168.1.13]，默认
192.168.1.13

ip = "192.168.1.13"

if len(sys.argv) >= 3 and sys.argv[1] in ("-ip", "--ip"):

ip = sys.argv[2]

port = "6001"

# 获取 SDK 版本

version = get_library_version()

print("[INFO] SDK 版本: %s" % version)

# 连接控制器

print("[INFO] 连接控制器...")

socket_fd = connect_robot(ip, port)
```

```
if socket_fd < 0:

print("[ERROR] 连接失败，程序退出!")

return -1

print("[INFO] 连接成功 (socket=%d)" % socket_fd)

# 使能上电

enable_servo(socket_fd)

# ----设置机器人模式----

print("[INFO] **********设置机器人模式**********")

mode = 1

ret = set_current_mode(socket_fd, mode)

if ret != SUCCESS:

print("[ERROR] 设置机器人模式失败，程序退出!")

return -1

print("[INFO] 设置机器人模式成功")

print()

# ----设置数字 IO 输出----

print("[INFO] **********设置数字 IO 输出**********")

io_port = 1

value = 0

ret = set_digital_output(socket_fd, io_port, value)

if ret != SUCCESS:

    print("[ERROR] 设置数字 IO 输出失败，程序退出!")

    return -1

print("[INFO] 设置数字 IO 输出成功")

print()

# ----查询数字 IO 输出----
```

```
print("[INFO] **********查询数字 IO 输出**********")

outputValues = VectorInt()

ret = get_digital_output(socket_fd, outputValues)

if ret != SUCCESS:

print("[ERROR] 查询数字 IO 输出失败，程序退出!")

return -1

print("[INFO] 查询数字 IO 输出成功，状态为：", end="")

for state in outputValues:

print("%d " % state, end="")

print()

# ----获取所有 IO 输入状态----

print("[INFO] **********获取所有 IO 输入状态**********")

inputValues = VectorInt()

ret = get_digital_input(socket_fd, inputValues)

if ret != SUCCESS:

print("[ERROR] 查询数字 IO 输入失败，程序退出!")

return -1

print("[INFO] 查询数字 IO 输入成功，状态为：", end="")

for state in inputValues:

print("%d " % state, end="")

print()

# ----查询流程图编程运行状态----

demo_get_robot_running_state(socket_fd)

# 使能下电

disable_servo(socket_fd)

# 断开控制器连接

print("[INFO] 断开控制器连接...")
```

```
ret = disconnect_robot(socket_fd)

if ret != SUCCESS:

print("[ERROR] 断开连接失败，程序退出!")

return -1

print("[INFO] 断开连接成功")

return 0

if __name__ == "__main__":

main()
```

6.9拖拽示教与轨迹记录/回放示例

```
#!/usr/bin/env python3

#

# ex_track_queue_drag.py

# @brief 测试 track_record_playback 接口 ------ 轨迹记录与回放

# @attention

# - 确保机械臂供电正常、网络通信正常

# - 默认控制器 IP 为 192.168.1.13

# - 需要在安全环境下运行，回放时机械臂会自动运动

# - 操作流程:

# 示教模式 → set_darg_mode(1)力矩 → set_teach_type(1)拖拽

# → track_record_start → 手拖机械臂 → track_record_stop

# → set_teach_type(0)恢复 → track_record_save → track_record_playback

# - 运行: python3 ex_track_queue_drag.py

#

import sys

import os
```

```
import platform

import time

_script_dir = os.path.dirname(os.path.abspath(__file__))

if platform.system() == "Windows":

sys.path.append(os.path.join(_script_dir, "../../lib/windows/python"))

_dll_dir = os.path.join(_script_dir, "../../lib/windows")

os.add_dll_directory(_dll_dir)

else:

sys.path.append(os.path.join(_script_dir, "../../lib/linux/x86/python"))

from tl_interface import *

# ============================================================

# 工具函数

# ============================================================

def print_ok(msg):

print(" [OK] " + msg)

def print_fail(msg, ret):

print(" [FAIL] " + msg + ", 错误码: " + str(ret))

def print_title(title):

print("\\n" + "=" * 50)

print(" " + title)

print("=" * 50)

def enable_servo(sock):

"""伺服上电（含状态检查，对齐 liuxin1.cpp 的 enable_servo）"""

print("--- 检查伺服状态 ---")

clear_error(sock)

time.sleep(0.3)
```

```
set_servo_poweroff(sock)

time.sleep(0.3)

state = -1

ret, state = get_servo_state(sock, state)

if ret != SUCCESS:

print(" [ERROR] 获取伺服状态失败: %d" % ret)

return False

print(" [信息] 当前伺服状态: %d (0=停止 1=就绪 2=报警 3=运行)" % state)

if state == 2:

print("[ERROR] 伺服处于报警状态，无法上电，请排查报警原因后重试")

return False

if state == 3:

print("[INFO] 机械臂已在使能上电状态")

return True

if state == 0:

print(" [信息] 伺服处于停止状态，设置就绪...")

ret = set_servo_state(sock, 1)

if ret != SUCCESS and ret != OPERATION_NOT_ALLOWED:

print("[ERROR] 设置伺服就绪失败: %d" % ret)

return False

time.sleep(0.5)

print(" [信息] 上电使能...")

ret = set_servo_poweron(sock)

if ret != SUCCESS:

print("[ERROR] 上电失败: %d" % ret)

return False

time.sleep(2)
```

```
ret, state = get_servo_state(sock, state)

if state == 3:

print("[INFO] 机械臂使能上电成功（servo_state = %d）" % state)

return True

else:

print("[ERROR] 机械臂使能上电失败（servo_state = %d）" % state)

return False

def disable_servo(sock):

"""伺服下电（对齐 liuxin1.cpp 的 disable_servo）"""

state = -1

get_servo_state(sock, state)

if state == 1:

print("[INFO] 机械臂已使能下电")

elif state == 3:

set_servo_poweroff(sock)

get_servo_state(sock, state)

print("[INFO] 机械臂使能下电成功（servo_state = %d）" % state)

else:

print("[INFO] 机械臂使能下电失败（servo_state = %d）" % state)

def wait_playback_done(sock, timeout=120):

"""轮询等待回放完成（用 get_robot_running_state 替代，x86 SWIG 不支持 bool&）"""

time.sleep(0.1)

for _ in range(timeout * 2):

running_state = -1

ret, running_state = get_robot_running_state(sock, running_state)

if ret == SUCCESS and running_state == 0: # 0=停止，回放完成
```

```
    return True
time.sleep(0.5)
return False
# ============================================================
# 主函数
# ============================================================
def main():
print_title("track_record_playback 接口测试（对齐 liuxin1.cpp）")
print("""
流程概览:
1. 上电 → 示教模式
2. set_darg_mode(1)力矩 → set_teach_type(1)拖拽
3. 开启轨迹记录
4. 手动拖拽机械臂走一段轨迹
5. 停止记录 → set_teach_type(0)恢复 → 保存轨迹
6. 回放轨迹（不同速度，轮询等待完成）
7. 清除轨迹数据
""")
# ---- 连接 ----
sock = connect_robot("192.168.1.13", "6001")
if sock <= 0:
print("[ERROR] 连接失败")
return -1
print("[INFO] 连接成功 (socket=%d)" % sock)
# ---- 清错 + 上电 ----
print_title("清错 & 上电")
if not enable_servo(sock):
```

```
print("[ERROR] 伺服上电失败，程序退出")

disconnect_robot(sock)

return -1

# ---- 示教模式 ----

print_title("切换示教模式")

ret = set_current_mode(sock, 0)

print_ok("set_current_mode(0) 示教模式") if ret == SUCCESS else print_fail("set_current_m
ode", ret)

time.sleep(0.3)

pass_count = 0

fail_count = 0

# ================================================================

# 测试1: 查询初始记录状态

# ================================================================

print_title("测试1: 查询初始记录状态")

recordStart = False

ret, recordStart = get_track_record_status(sock, recordStart)

# 注意: x86 Linux SWIG 可能无法正确传出 bool& 参数，以 ret 为准

if ret == SUCCESS:

print_ok("初始状态: %s (ret=%d)" % ("记录中" if recordStart else "未记录", ret))

pass_count += 1

else:

print_fail("get_track_record_status", ret)

fail_count += 1

# ================================================================

# 测试2: 开启拖拽模式（对齐 liuxin1.cpp: darg_mode=1 力矩，需在 teach_type 前）

# ================================================================
```

```
# mode: 0=无 1=力矩模式 2=位置（参考 liuxin1.cpp 实际映射）

print_title("测试2: set_darg_mode(1) 力矩模式")

ret = set_darg_mode(sock, 1)

if ret == SUCCESS:

print_ok("set_darg_mode(1) 力矩模式已开启")

pass_count += 1

else:

print_fail("set_darg_mode(1)", ret)

fail_count += 1

# set_teach_type 必须在 set_darg_mode 之后（对齐 liuxin1.cpp）

print_title("测试2b: set_teach_type(1) 切换到拖拽")

ret = set_teach_type(sock, 1)

if ret == SUCCESS:

print_ok("已切换到拖拽模式")

pass_count += 1

else:

print_fail("set_teach_type(1)", ret)

fail_count += 1

# 确认

teach_type = -1

ret, teach_type = get_teach_type(sock, teach_type)

if ret == SUCCESS:

name = {0: "点动", 1: "拖拽"}.get(teach_type, "未知")

print_ok("确认: teach_type=%d (%s)" % (teach_type, name))

# ============================================================

# 测试3: track_record_start（开始记录）
```

```
# ============================================================

print_title("测试3: 开始轨迹记录")

# 参数对齐 liuxin1.cpp: maxSamplingNum=500, samplingInterval=0.1

MAX_SAMPLES = 500

SAMPLE_INTERVAL = 1

ret = track_record_start(sock, MAX_SAMPLES, SAMPLE_INTERVAL)

if ret != SUCCESS:

print_fail("track_record_start", ret)

fail_count += 1

# 尝试备选参数

print(" [信息] 尝试备选参数 (1000, 0.1)...")

ret = track_record_start(sock, 1000, 1)

if ret != SUCCESS:

print_fail("track_record_start (备选参数)", ret)

set_teach_type(sock, 0)

set_darg_mode(sock, 0)

set_servo_poweroff(sock)

disconnect_robot(sock)

return -1

print_ok("轨迹记录已开始 (maxSamples=%d, interval=%.2fs)" % (MAX_SAMPLES, SAMPLE_INTERVAL)
)

# 验证记录状态（注意: x86 SWIG 可能无法正确传出 bool&，以 track_record_start 返回值判断）

recordStart = False

ret, recordStart = get_track_record_status(sock, recordStart)

if ret == SUCCESS:

print_ok("记录状态验证: %s" % ("正在记录" if recordStart else "状态查询成功(ret=%d)，以上一步
start 返回值为准" % ret))

pass_count += 1
```

```
else:

print_fail("记录状态验证失败", ret)

fail_count += 1

# ============================================================

# 用户手动拖拽

# ============================================================

print("\\n" + "=" * 50)

print(" [操作] 机械臂已进入拖拽状态，请手动拖拽机械臂走一段轨迹")

print(" [操作] 完成后按回车键停止记录...")

print("=" * 50)

try:

input()

except (EOFError, KeyboardInterrupt):

pass

# ============================================================

# 测试4: track_record_stop（停止记录）

# ============================================================

print_title("测试4: 停止轨迹记录")

ret = track_record_stop(sock)

if ret == SUCCESS:

print_ok("track_record_stop")

pass_count += 1

else:

print_fail("track_record_stop", ret)

fail_count += 1

# 验证停止状态
```

```
recordStart = True

ret, recordStart = get_track_record_status(sock, recordStart)

if ret == SUCCESS:

print_ok("记录状态验证: %s" % ("已停止" if not recordStart else "状态查询成功
(ret=%d)，以上一步 stop 返回值为准" % ret))

pass_count += 1

else:

print_fail("记录状态验证失败", ret)

fail_count += 1

# ---- 退出拖拽模式，切回点动（对齐
# liuxin1.cpp，回放前需切回点动）----

print_title("退出拖拽模式")

ret = set_teach_type(sock, 0)

print_ok("set_teach_type(0) 已切回点动模式") if ret == SUCCESS else print_fail("切回点动", r
et)

time.sleep(0.3)

# ================================================================

# 测试5: track_record_save（保存轨迹）

# ================================================================

print_title("测试5: 保存轨迹")

# 按示教器格式生成轨迹名（对齐 liuxin1.cpp）

TRAJ_NAME = time.strftime("轨迹名-%m月%d日%H时%M分%S", time.localtime())

ret = track_record_save(sock, TRAJ_NAME)

if ret == SUCCESS:

print_ok("轨迹已保存为: %s" % TRAJ_NAME)

pass_count += 1

else:

print_fail("track_record_save('%s')" % TRAJ_NAME, ret)
```

```
fail_count += 1

# ================================================================

# 测试6: track_record_playback（回放）------ 核心测试

# ================================================================

print_title("测试6: 轨迹回放 (track_record_playback)")

print(" [警告] 机械臂将自动回放记录的轨迹！请确保周围安全！")

print(" 按回车键开始回放...")

try:

input()

except (EOFError, KeyboardInterrupt):

pass

# --- 用例A: 低速回放 vel=20 ---

print("\\n [用例A] 低速回放 (vel=20)")

ret = track_record_playback(sock, 20)

if ret == SUCCESS:

print_ok("低速回放已开始，等待完成...")

if wait_playback_done(sock, timeout=120):

print_ok("低速回放完成")

pass_count += 1

else:

print(" [WARN] 低俗回放等待超时")

fail_count += 1

else:

print_fail("低俗回放", ret)

fail_count += 1

# --- 用例B: 中速回放 vel=50（对齐 liuxin1.cpp 的默认速度）---

print("\\n [用例B] 中速回放 (vel=50)")
```

```
print(" 按回车键继续...")

try:

input()

except (EOFError, KeyboardInterrupt):

pass

ret = track_record_playback(sock, 50)

if ret == SUCCESS:

print_ok("中速回放已开始，等待完成...")

if wait_playback_done(sock, timeout=120):

print_ok("中速回放完成")

pass_count += 1

else:

print(" [WARN] 中速回放等待超时")

fail_count += 1

else:

print_fail("中速回放", ret)

fail_count += 1

# --- 用例C: 高速回放 vel=100 ---

print("\\n [用例C] 高速回放 (vel=100)")

print(" 按回车键继续...")

try:

input()

except (EOFError, KeyboardInterrupt):

pass

ret = track_record_playback(sock, 100)

if ret == SUCCESS:
```

```
print_ok("高速回放已开始，等待完成...")

if wait_playback_done(sock, timeout=120):

print_ok("高速回放完成")

pass_count += 1

else:

print(" [WARN] 高速回放等待超时")

fail_count += 1

else:

print_fail("高速回放", ret)

fail_count += 1

# --- 用例D: 边界值测试 ---

print_title("测试7: playback 边界值测试")

for vel, desc in [(0, "vel=0"), (150, "vel=150"), (200,
"vel=200")]:

ret = track_record_playback(sock, vel)

if ret == SUCCESS:

print(" vel=%d (%s): 返回成功" % (vel, desc))

else:

print(" vel=%d (%s): 返回错误码 %d（可能是正常范围外）" % (vel, desc, ret))

time.sleep(1)

# ================================================================

# 测试8: track_record_delete（清除轨迹缓冲区）

# ================================================================

print_title("测试8: 清除轨迹数据")

ret = track_record_delete(sock)

if ret == SUCCESS:

print_ok("track_record_delete")
```

```
pass_count += 1

else:

print_fail("track_record_delete", ret)

fail_count += 1

# ================================================================

# 收尾

# ================================================================

print_title("收尾")

set_darg_mode(sock, 0)

set_teach_type(sock, 0)

disable_servo(sock)

disconnect_robot(sock)

print("[INFO] 测试完毕")

# ---- 报告 ----

print_title("测试报告")

total = pass_count + fail_count

print(" 通过: %d, 失败: %d, 总计: %d" % (pass_count, fail_count, total))

return 0 if fail_count == 0 else -1

if __name__ == "__main__":

exit(main())
```

附录（一）

返回值说明

返回值	常量	说明
0	SUCCESS	成功
-1	RECEIVE_FAILED	接收失败

返回值	常量	说明
-2	DISCONNECT	未连接
-3	PARAM_ERR	参数错误
-4	OPERATION_NOT_ALLOWED	操作不被允许
-5	EXCEPTION	异常
-6	TIMEOUT	超时

部分接口可以看cpp 文件的代码注释

服务热线：**18090080872**

四川天链机器人股份有限公司

地址：四川省绵阳高新区火炬西街南段10号

http://www.tlibot.com

E-mail:TLIBOT@163.com

本产品的额定功率、规格、外部尺寸等

如需改良而进行变更，恕不另行通告。

技术数据和插图仅作为供货参考，保留更改权利。